**Department of Electronic and Telecommunication Engineering**

**University of Moratuwa, Sri Lanka**




# Battery-Aware Return-to-Home for a Delivery Drone Using an Extended Kalman Filter

**Basnayake B.M.N.S**

Index No.: 220072R

**EN4594-Autonomous Systems**

Submitted in partial fulfillment of the requirements for course module EN4594

**October 20, 2026**

# Abstract

A delivery drone must decide when to abandon its route and fly home, yet it cannot measure its battery's remaining charge directly: its current sensor is noisy and biased, and its voltage sags under load. This report develops an Extended Kalman Filter (EKF) that estimates the state of charge (SoC) and its uncertainty from these two signals, and a risk-aware return-to-home rule that turns back when the probability of landing with a 10% reserve falls below 95%. Both are tested in a MATLAB simulator whose battery deliberately differs from the filter's equivalent-circuit model. A design-stage investigation showed that resistance mismatch dominated the SoC error, so the filter learns the cell resistance during flight; this reduced the SoC RMSE on a nominal flight to 0.97%, against 4.36% for charge counting. A five-state filter that also estimates sensor bias and capacity kept honest uncertainty bands in every nominal, biased and aged-battery scenario and was chosen for decisions. In 500 random missions, the risk-aware rule had no battery failures and broke the reserve in 2.8% of missions, against 38.0% for the same filter without its uncertainty and 30.4% battery failures for charge counting. Its breach rate stayed within the promised level at every threshold tested, showing that its probabilities were calibrated.

# Contents

**Abstract** **i**

**1 Introduction** **1**

**2 Background** **3**

**3 Methodology** **7**

**4 Implementation** **14**

**5 Results** **29**

**6 Discussion** **54**

**7 Conclusion** **61**

**References** **63**

**A MATLAB Code** **65**

# Chapter 1

# Introduction

## 1.1 Problem and motivation

A delivery drone must decide, during flight, when to stop delivering and return home. It cannot measure its remaining charge, the *state of charge* (SoC), directly. It only has a noisy, slightly biased current sensor and a voltage sensor whose reading dips under heavy load, so a low voltage does not always mean an empty battery.

The decision can fail in two ways. Turning back too late risks losing the drone and its payload. Turning back too early wastes battery and deliveries. A fixed safety margin trades one failure for the other. A better decision needs both an SoC estimate and a measure of how uncertain it is, so that the margin can shrink when the estimate is precise and grow when it is not.

## 1.2 Why an Extended Kalman Filter

A Kalman filter combines a model-based prediction with a measurement, weights each by how much it can be trusted, and reports its own uncertainty. The standard filter assumes linear equations, but a battery's resting voltage is a curved function of its SoC (Section 2.1). The Extended Kalman Filter (EKF) handles this by using the tangent of the curve at the current estimate. The curve is smooth and only mildly bent, so the EKF fits well, and it is the classic approach to SoC estimation in battery management systems [1, 2]. A particle filter would also work, but at a much higher computational cost for little benefit here [3].

## 1.3 Objectives

1. Build a MATLAB simulator of a delivery drone and its battery that gives the true SoC and realistic, imperfect sensor readings.

2. Design an EKF that estimates the SoC and its uncertainty.

3. Design a return-to-home (RTH) rule that uses both the estimate and its uncertainty.
4. Compare the EKF and the RTH rule with simpler methods (charge counting and a voltage threshold) in single flights and over many random missions.

## 1.4 Contributions

- An EKF that learns the battery's internal resistance during flight. Tests showed that a wrong resistance value, not the faulty current sensor, caused most of the SoC error (Section 3.4.5).
- A choice of the filter used for decisions based on how honest its uncertainty is, not only on its accuracy. The most accurate filter was confidently wrong on an aged battery (Section 3.4.5).
- An RTH rule that turns back when the probability of getting home with a safety reserve drops below 95%, so the margin grows automatically when the filter is unsure (Section 3.5).
- A safety-versus-wasted-battery trade-off that shows what each extra level of safety costs (Section 5.5).

# Chapter 2

# Background

## 2.1 Battery basics

The *capacity* $C_n$ is the charge a full battery can deliver, in ampere-hours (Ah); it falls as the battery ages. The *state of charge* (SoC) is the fraction of the capacity that remains, from 1 (full) to 0 (empty).

When a battery rests, its voltage depends only on its SoC. This resting voltage is the *open-circuit voltage* (OCV). The OCV curve (Figure 2.1) is curved, so a straight-line model cannot describe it, and its slope changes: about 25 mV per 1% SoC near empty, but only 5 mV per 1% between 20% and 40%. Where the curve is flat, the voltage says little about the SoC, so a good filter should become less certain there.

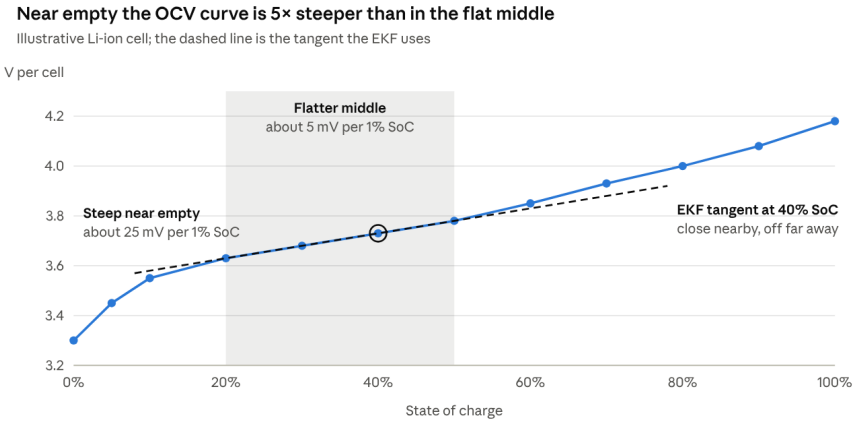



Figure 2.1: OCV curve of one cell. It is steep near empty and flat between about 20% and 50% SoC. The dashed tangent at 40% SoC is what the EKF uses: close to the curve nearby, wrong far away.

Under load the voltage drops in two ways. An instant drop $R_0 I$ comes from the internal resistance $R_0$, where $I$ is the current (positive for discharge):

$$\Delta V_{R_0} = R_0\, I. \tag{2.1}$$

At 30 A take-off current and $R_0 = 8\,\mathrm{m\Omega}$ this is 0.24 V per cell, so the battery looks emptier than it is. A slower drop $V_1$ builds up over tens of seconds, modelled by a resistor $R_1$ in parallel with a capacitor $C_1$:

$$C_1 \frac{dV_1}{dt} = I - \frac{V_1}{R_1}, \tag{2.2}$$

with time constant $\tau_1 = R_1 C_1$ (30 s here). Together these form the equivalent circuit of Figure 2.2. The sensor reads

$$V = \mathrm{OCV}(\mathrm{SoC}) - R_0\, I - V_1. \tag{2.3}$$

Only $V$ and $I$ are measured; the SoC and $V_1$ must be estimated.

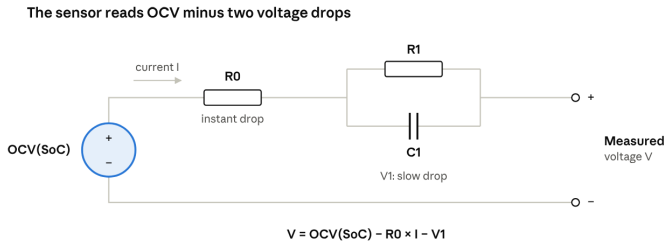



Figure 2.2: One-RC equivalent circuit used by the EKF. The simulated "true" battery adds a second, slower RC pair (Section 3.2).

The simplest SoC tracker is *charge counting*: start from a known SoC and subtract the charge drawn. With time step $\Delta t$ (s) and current $I_k$ (A),

$$\mathrm{SoC}_k = \mathrm{SoC}_{k-1} - \frac{I_k\, \Delta t}{3600\, C_n}. \tag{2.4}$$

It never corrects itself, so a wrong start stays wrong, a wrong capacity makes it drift, and a current-sensor bias $b$ adds an error that grows with time:

$$e(t) = -\frac{b\, t}{3600\, C_n}. \tag{2.5}$$

A 0.5 A bias over 15 minutes is 2.5% of a 5 Ah battery. Charge counting is used as a baseline.

## 2.2 From the Kalman filter to the Extended Kalman Filter

A Kalman filter balances a prediction against a measurement. For the drone, charge counting (2.4) is the prediction and the voltage (2.3) is the measurement. The hidden quantities form the state $x_k$; the measured current is the input $u_k$ and the measured voltage is the measurement $z_k$:

$$x_k = f(x_{k-1}, u_k) + w_k, \qquad z_k = h(x_k, u_k) + v_k, \tag{2.6}$$

where $w_k$ and $v_k$ are zero-mean, bell-shaped (Gaussian) errors with covariances $Q$ (model error) and $R$ (sensor error). The filter keeps an estimate $\hat{x}_k$ and its uncertainty (covariance) $P_k$.

For a linear model, $f = Fx + Bu$ and $h = Hx$, each step has two parts. **Predict** with the model (uncertainty grows), then **update** with the measurement (uncertainty shrinks):

$$\hat{x}_k^- = F\hat{x}_{k-1} + Bu_k, \qquad P_k^- = FP_{k-1}F^\top + Q, \tag{2.7}$$

$$\tilde{y}_k = z_k - H\hat{x}_k^-, \qquad S_k = HP_k^-H^\top + R, \tag{2.8}$$

$$K_k = P_k^-H^\top S_k^{-1}, \qquad \hat{x}_k = \hat{x}_k^- + K_k\tilde{y}_k, \tag{2.9}$$

$$P_k = (\mathbb{I} - K_kH)P_k^-. \tag{2.10}$$

Here "$-$" means before the measurement, $\tilde{y}_k$ is the *innovation* (the filter's surprise), $S_k$ its expected variance, and $K_k$ the *Kalman gain*. The gain is large when the prediction is uncertain and small when the sensor is noisy or the measurement says little about the state, as on the flat part of the OCV curve. $Q$ and $R$ are the tuning knobs: a large $Q$ makes the filter fast but jumpy, a large $R$ smooth but slow.

The battery model is not linear, because of OCV(SoC). The EKF replaces $f$ and $h$, at every step, by their tangent at the current estimate. The slopes form the *Jacobians*

$$F_k = \left.\frac{\partial f}{\partial x}\right|_{\hat{x}_{k-1},\,u_k}, \qquad H_k = \left.\frac{\partial h}{\partial x}\right|_{\hat{x}_k^-,\,u_k}, \tag{2.11}$$

which replace $F$ and $H$ in (2.7)–(2.10), while the state and expected measurement use the full nonlinear $f$ and $h$. For the battery the key entry of $H_k$ is the OCV slope at the current SoC estimate, the tangent in Figure 2.1. This works well when the curve is smooth and the uncertainty small, as here [3, 4].

## 2.3 A worked example

One EKF step with SoC as the only state (`learning/day1_ekf_by_hand.m`) shows the gain at work. The filter starts at $80\% \pm 5\%$. Drawing 20 A for 10 s predicts 78.89%. The model then expects 3.832 V, but the sensor reads 3.80 V, a surprise of $-32$ mV. With an OCV slope of 7 mV per 1% SoC and a 10 mV sensor, the gain is $K = 1.32\,\mathrm{V}^{-1}$, and the estimate moves to $74.6\% \pm 1.4\%$: one voltage reading cut the uncertainty from 5% to 1.4%. With a noisy 50 mV sensor, or a confident $\pm 1\%$ start, the gain falls to about 0.47 and the estimate moves much less.

## 2.4 Related work

Plett's three-part paper [1, 2, 5] is the classic reference for EKF battery estimation, including adding unknown parameters such as resistance or capacity to the state, as done here; his textbook [6] covers equivalent-circuit models in depth. MathWorks provides a MATLAB EKF battery example [7]. Simon [3] covers the KF, EKF, unscented Kalman filter (UKF) and particle filter, and Welch and Bishop [4] give a short introduction.

For drones, an EKF has been used for UAV battery SoC estimation [8], and prediction of remaining flying time, the quantity an RTH decision depends on, has been verified for electric unmanned vehicles [9]. Dibbern et al. [10] compared an EKF, a UKF and charge counting on delivery and inspection missions, and found the UKF more accurate and parameter identification more important than the load profile. Public flight and cell datasets [11–14] are available for future validation; this project uses simulation so that the true SoC is always known.

# Chapter 3

# Methodology

## 3.1 System overview

Figure 3.1 shows the system. The simulator flies the drone and computes the true battery state. The sensors turn the true current and voltage into imperfect readings. The EKF and the two baselines (charge counting and a voltage threshold) see only these readings, and each feeds a turn-back decision. Only the simulator knows the truth, so every estimate and every decision can be scored exactly.

The drone carries a six-cell series lithium-ion pack of 5 Ah. The same current flows through every cell, so one cell is modelled; all voltages and battery parameters are per cell. Everything runs once per second ($\Delta t = 1\,\mathrm{s}$); $I_k$ is the current during step $k$ and every state with index $k$ is its value at the end of that step.

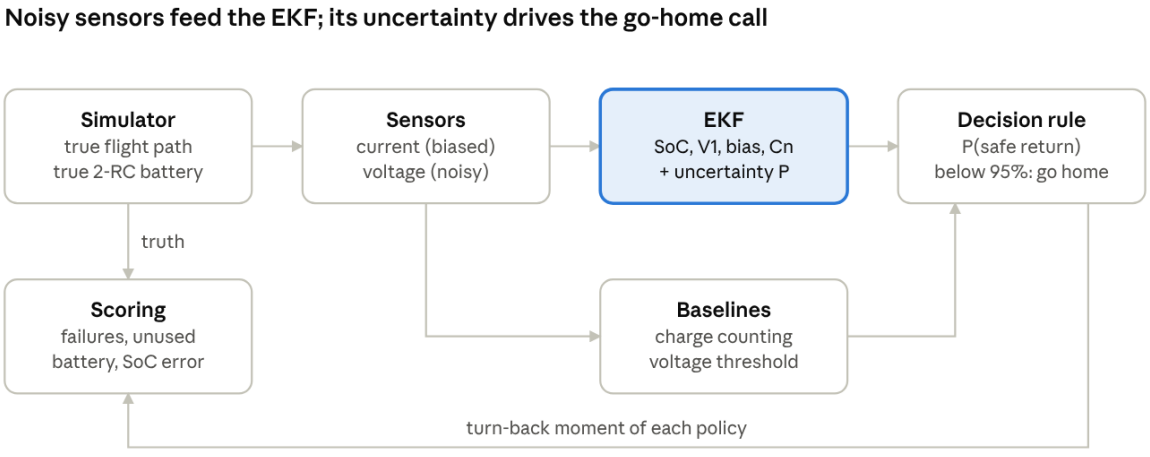



Figure 3.1: System overview. The EKF and the baselines see only the noisy sensor readings; the simulator's truth is used only for scoring. (The EKF block lists four states; the final filter also learns $R_0$, Section 3.4.4.)

## 3.2 Battery models: truth versus filter

A filter tested against its own model always looks good, and a real filter never has the exact model. The simulated battery is therefore deliberately different from the filter's model (Table 3.1).

With the current held constant during a step, an RC pair (2.2) updates exactly as

$$V_{i,k} = a_i\, V_{i,k-1} + R_i\,(1 - a_i)\, I_k, \qquad a_i = e^{-\Delta t/(R_i C_i)}. \tag{3.1}$$

The true battery has a second, slow RC pair ($R_2$, $C_2$) that the filter does not know about, and an OCV curve shifted by $\delta_{\mathrm{OCV}}$:

$$\mathrm{SoC}_k = \mathrm{SoC}_{k-1} - \frac{I_k\,\Delta t}{3600\,C_n}, \qquad V_{1,k},\, V_{2,k} \text{ from (3.1)}, \tag{3.2}$$

$$V_k = \mathrm{OCV}(\mathrm{SoC}_k) + \delta_{\mathrm{OCV}} - R_0\, I_k - V_{1,k} - V_{2,k}. \tag{3.3}$$

The battery is dead if $V_k < 3.0\,\mathrm{V}$ while current flows.

Both models use the same representative lithium-ion OCV table (Figure 2.1), smoothed with shape-preserving cubic interpolation (`pchip`) so its slope does not jump. The EKF needs the slope, found by a central difference over ±0.5% SoC (one-sided at the ends of the table):

$$\frac{d\,\mathrm{OCV}}{d\,\mathrm{SoC}}(s) \approx \frac{\mathrm{OCV}(s^+) - \mathrm{OCV}(s^-)}{s^+ - s^-}, \qquad s^{\pm} = s \pm 0.005. \tag{3.4}$$

Table 3.1: Battery parameters per cell (`config/battery_params.m`).

| Parameter | Filter | Truth | Note |
|---|---|---|---|
| Circuit | 1 RC pair | 2 RC pairs | |
| $C_n$ | 5.0 Ah | 5.0 Ah | 4.25 Ah in E4 |
| $R_0$ | 8 mΩ | 8.8 mΩ | +10% |
| $R_1$, $C_1$ | 6 mΩ, 5000 F | 5.4 mΩ, 5500 F | $\tau_1 \approx 30$ s |
| $R_2$, $C_2$ | – | 1.5 mΩ, 266 667 F | $\tau_2 = 400$ s; 4 mΩ in E6 |
| $\delta_{\mathrm{OCV}}$ | 0 | −5 mV | |

The slow pair is the most visible difference: with $\tau_2 = 400\,\mathrm{s}$ its voltage keeps building through most of a 15-minute flight, and the filter has no state for it. A first setting, $R_2 = 4\,\mathrm{m\Omega}$, was used for the simulator test (Section 5.1); at 15 A it adds about 63 mV of unmodelled drop, which on the flat part of the curve looks like up to 13% SoC error. That is harsher than a reasonably identified model, so the default was reduced to 1.5 mΩ (about 23 mV) and the strong setting kept as stress test E6.

## 3.3 Sensors and mission

The sensors read

$$I_{m,k} = I_k + b + w_{I,k}, \qquad V_{m,k} = V_k + v_k, \tag{3.5}$$

with a current bias $b$ (0.5 A by default), current noise $\sigma_I = 0.2\,\mathrm{A}$ and voltage noise $\sigma_V = 10\,\mathrm{mV}$.

The drone takes off (30 s at 30 A), cruises at 10 m/s to each drop-off (15 A $\times$ $(1 + w)$, where $w$ is a wind factor), hovers 30 s at 20 A to deliver, flies home and lands (40 s at 18 A). A slowly varying random load is added while flying:

$$\varepsilon_k = 0.9\,\varepsilon_{k-1} + 0.3\,n_k, \quad n_k \sim \mathcal{N}(0,1), \qquad I_k = \max(I_{\mathrm{demand},k} + \varepsilon_k,\, 0), \tag{3.6}$$

which wanders by about $\pm 0.7$ A. The *short* route has three drop-offs and ends with charge to spare; the *long* route has six drop-offs reaching 9 km and is too far for one battery, so the return-to-home rule must act.

## 3.4 The Extended Kalman Filter

### 3.4.1 Model

The state, input and measurement are

$$x_k = \begin{bmatrix} \mathrm{SoC}_k & V_{1,k} & b_k & C_{n,k} & R_{0,k} \end{bmatrix}^\top, \qquad u_k = I_{m,k}, \qquad z_k = V_{m,k}. \tag{3.7}$$

The filter's best guess of the true current is $I_{m,k} - b$. The process model is charge counting and the RC update with this corrected current, while the bias, capacity and resistance are treated as constants that may wander slowly (this is how a filter learns an unknown constant):

$$\mathrm{SoC}_k = \mathrm{SoC}_{k-1} - \frac{(I_{m,k} - b_{k-1})\,\Delta t}{3600\,C_{n,k-1}}, \tag{3.8}$$

$$V_{1,k} = a\,V_{1,k-1} + R_1(1-a)(I_{m,k} - b_{k-1}), \tag{3.9}$$

$$b_k = b_{k-1} + w_b, \qquad C_{n,k} = C_{n,k-1} + w_C, \qquad R_{0,k} = R_{0,k-1} + w_R. \tag{3.10}$$

The measurement model is the circuit equation (2.3):

$$V_{m,k} = \mathrm{OCV}(\mathrm{SoC}_k) - V_{1,k} - R_{0,k}\,(I_{m,k} - b_k) + v_k. \tag{3.11}$$

### 3.4.2 Jacobians

Differentiating (3.8)–(3.11) with respect to the state gives

$$F_k = \begin{bmatrix} 1 & 0 & \frac{\Delta t}{3600\,\hat{C}_n} & \frac{(I_m - \hat{b})\,\Delta t}{3600\,\hat{C}_n^2} & 0 \\ 0 & a & -R_1(1-a) & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \qquad H_k^\top = \begin{bmatrix} d\,\mathrm{OCV}/d\,\mathrm{SoC} \\ -1 \\ \hat{R}_0 \\ 0 \\ -(I_m - \hat{b}) \end{bmatrix}. \tag{3.12}$$

The OCV slope makes the voltage correct the SoC strongly near empty and weakly in the flat middle. The resistance entry grows with the current, so the resistance shows up most when the current changes. The capacity entry of $H$ is zero: capacity is learned only slowly, through how the SoC drifts.

### 3.4.3 Noise

The current-sensor noise enters the prediction through

$$G_k = \frac{\partial f}{\partial I_m} = \begin{bmatrix} -\frac{\Delta t}{3600\,\hat{C}_n} & R_1(1-a) & 0 & 0 & 0 \end{bmatrix}^{\top}.$$

Model errors without a single physical source are covered by small diagonal terms:

$$Q_k = G_k\,\sigma_I^2\,G_k^{\top} + \operatorname{diag}(q_{\mathrm{SoC}},\, q_{V_1},\, q_b,\, q_{C_n},\, q_{R_0})\,, \qquad R = \sigma_V^2 + (R_0\,\sigma_I)^2 + \sigma_{\mathrm{model}}^2. \quad (3.13)$$

In $R$, the second term is the current noise seen through the resistance, and $\sigma_{\mathrm{model}} = 10\,\mathrm{mV}$ is an allowance for the filter's imperfect model. The tuning values are in Table 3.2.

Table 3.2: Filter tuning (`estimators/ekf_init.m`).

| | $q_{\mathrm{SoC}}$ | $q_{V_1}$ ($\mathrm{V}^2$) | $q_b$ ($\mathrm{A}^2$) | $q_{C_n}$ ($\mathrm{Ah}^2$) | $q_{R_0}$ ($\Omega^2$) | $R$ ($\mathrm{V}^2$) |
|---|---|---|---|---|---|---|
| Per step | $10^{-8}$ | $10^{-6}$ | $10^{-6}$ | $10^{-7}$ | $10^{-9}$ | $\approx 2.0 \times 10^{-4}$ |

The starting SoC is read from the resting voltage before take-off (the average of 10 readings in E3–E8 and the Monte Carlo study) with ±3% uncertainty; the other states start at $V_1 = 0$, $b = 0$, $C_n = 5\,\mathrm{Ah}$ and $R_0 = 8\,\mathrm{m}\Omega$:

$$\hat{x}_0 = \begin{bmatrix} \widehat{\mathrm{SoC}}_0 & 0 & 0 & 5.0 & 0.008 \end{bmatrix}^{\top}, \quad P_0 = \operatorname{diag}\!\left(0.03^2,\, 0.01^2,\, 1^2,\, 0.5^2,\, 0.004^2\right). \quad (3.14)$$

The covariance is updated in a numerically stable (Joseph) form that keeps it symmetric and positive [3], and the SoC, capacity and resistance are clamped to physical ranges:

$$P_k = (\mathbb{I} - K_k H_k)\,P_k^{-}\,(\mathbb{I} - K_k H_k)^{\top} + K_k R K_k^{\top}. \quad (3.15)$$

Whether the filter's uncertainty is honest is checked in three ways: the true error should stay inside $\pm 3\sigma$ about 99.7% of the time; the normalised innovation squared (NIS, $\tilde{y}^2/S$) should average about 1; and the normalised SoC error squared (NEES, $e_{\mathrm{SoC}}^2/P_{11}$, possible only in simulation) should average about 1.

### 3.4.4 Filter stages

Three versions of the filter are compared (Table 3.3). A state that a stage does not learn is frozen at its nominal value with zero uncertainty, which is the same as leaving it out.

Table 3.3: Filter stages.

| Stage | Learned states | Role |
|---|---|---|
| A | SoC, $V_1$ | Basic EKF |
| B | SoC, $V_1$, $R_0$ | Most accurate on a healthy battery |
| C | all five | Used for the return-to-home decision |

### 3.4.5 Why the resistance became a state

The original plan was a four-state filter without $R_0$. Early tests in a Python copy of the simulator showed that stage A read the SoC about 2.5% too low and was overconfident: its $\pm 3\sigma$ band held the truth only about 16% of the time. Switching on one model mismatch at a time (Table 3.4) found the cause: a wrong resistance. The true battery sags more than the filter expects, and the filter can only explain the extra drop as a lower SoC; at 15 A, a 10% resistance error is 12 mV, which on the flat part of the curve looks like 2.4% less charge.

Table 3.4: Early test (Python copy, E1): SoC error of stage A with one mismatch at a time.

| Mismatch switched on | SoC RMSE |
|---|---|
| None (perfect model) | 0.12% |
| $R_0$ 10% higher | 1.85% |
| Slow RC pair | 1.62% |
| OCV 5 mV lower | 0.73% |
| Current bias 0.5 A | 0.43% |

Telling the filter its sensor was noisier (a larger $R$) did not help (2.49%), because the error is systematic, not random. Learning $R_0$ did: a drop that appears only under load must be resistance, not lower SoC. This gave stage B. The allowance $\sigma_{\text{model}} = 10$ mV was then chosen because it kept stage B's band honest (98% coverage over 20 seeds, against 90% without it).

Learning the bias and capacity as well (stage C) cannot separate them within one flight. Small errors in $b$ and $R_0$ change the voltage by

$$\delta V \approx \hat{R}_0\,\delta b - (I_m - \hat{b})\,\delta R_0, \tag{3.16}$$

which cancel at a steady current, and the capacity shows only through slow SoC drift. What the voltage does pin down is the drain rate

$$r = \frac{I_m - b}{C_n}, \tag{3.17}$$

so stage C's SoC stays accurate even when its $b$ and $C_n$ are individually wrong. On an aged battery, stage B blames the lower voltage on its resistance and overestimates the SoC, which is the dangerous direction, while stage C stays honest (Section 5.2.10). Stage C therefore drives the return-to-home decision.

## 3.5 Risk-aware return-to-home rule

Every second, the drone estimates the charge needed to fly home in a straight line at $v_{\text{cruise}} = 10$ m/s and land:

$$t_{\text{cruise},k} = \frac{d_{\text{home},k}}{v_{\text{cruise}}}, \qquad \Delta\text{SoC}_{\text{home},k} = \frac{\left(\hat{I}_{\text{cr},k} - \hat{b}_k\right) t_{\text{cruise},k} + I_{\text{land}}\, t_{\text{land}}}{3600\,\hat{C}_{n,k}}. \tag{3.18}$$

The cruise term uses the filter's own drain rate (3.17), the quantity the voltage makes reliable. $\hat{I}_{\mathrm{cr}}$ is the cruise current as measured, learned in flight with a 50-second moving average:

$$\hat{I}_{\mathrm{cr},k} = 0.98\,\hat{I}_{\mathrm{cr},k-1} + 0.02\,I_{m,k}, \qquad \hat{I}_{\mathrm{cr},0} = 15\,\mathrm{A}, \tag{3.19}$$

and the landing uses its nominal current ($I_{\mathrm{land}} = 18\,\mathrm{A}$, $t_{\mathrm{land}} = 40\,\mathrm{s}$).

The safety margin after the trip and a reserve of $\mathrm{SoC}_{\mathrm{res}} = 10\%$ is converted into a probability:

$$m_k = \widehat{\mathrm{SoC}}_k - \Delta\mathrm{SoC}_{\mathrm{home},k} - \mathrm{SoC}_{\mathrm{res}}, \tag{3.20}$$

$$\sigma_{\mathrm{total},k} = \sqrt{P_{k,11} + \left(0.1\,\Delta\mathrm{SoC}_{\mathrm{home},k}\right)^2}, \tag{3.21}$$

$$P_{\mathrm{safe},k} = \Phi\left(\frac{m_k}{\sigma_{\mathrm{total},k}}\right). \tag{3.22}$$

The first uncertainty term comes from the EKF; the second allows 10% of the trip cost for wind and load surprises. $\Phi$ is the standard normal distribution function. The drone turns back when $P_{\mathrm{safe}} < 95\%$, that is, when the margin falls below 1.645 standard deviations. When the filter is confident the buffer is small and the drone flies longer; when it is unsure the buffer grows automatically. If the uncertainty is honest, no more than 5% of missions should break the reserve, which the Monte Carlo study tests directly.

Four policies are compared on the same missions (Table 3.5). They decide only while the drone is cruising or hovering; once triggered, the drone flies straight home.

Table 3.5: Return-to-home policies.

| Policy | Turn back when |
|---|---|
| A. Voltage threshold | measured voltage stays below 3.5 V for 5 s |
| B. Charge counting | charge-counted SoC minus trip cost and reserve $< 0$ (rated capacity, no bias correction) |
| C. EKF estimate only | $m_k < 0$ |
| D. EKF risk-aware | $P_{\mathrm{safe},k} < 95\%$ |

Each mission is scored on whether the battery **died**, whether the **reserve was broken** (died or landed below 10%), the **unused battery** at landing above the reserve, and the number of **deliveries**.

## 3.6 Experiments

Eight experiments on the short route test the filter (Table 3.6); single runs use one fixed random seed and E8 repeats five scenarios over 20 seeds. The four policies are then compared on 500 random long-route missions (Table 3.7), each policy flying exactly the same conditions and sensor noise, and policy D's threshold (80–99.9%) and policy A's voltage (3.40–3.60 V) are swept on 200 of them. A single demonstration mission (aged battery, sensor reading 0.5 A low, 10% headwind) shows how each policy decides.

Table 3.6: Estimation experiments.

| # | Experiment | Set-up |
|---|---|---|
| E1 | Nominal flight | True start 95%, read from the resting voltage |
| E2 | Wrong start | True 85%, estimators assume 100% (±20%) |
| E3 | Sensor bias | Bias 0.8 A; stages B and C |
| E4 | Aged battery | True capacity 4.25 Ah, start 100%; stages B and C |
| E5 | Tuning | Stage B with $Q$ or $R$ scaled by 0.01 or 100 |
| E6 | Strong mismatch | Slow pair $R_2 = 4\,\mathrm{m\Omega}$; stages A, B, C |
| E7 | Flat OCV region | $\sigma_{\mathrm{SoC}}$ against true SoC, stages B and C |
| E8 | Stage comparison | 20 seeds × E1, E2, E3, E4, E6 × stages A, B, C |

Table 3.7: Random conditions for each Monte Carlo mission, drawn uniformly.

| Factor | Range |
|---|---|
| Starting SoC | 60–100% |
| Current-sensor bias | −1 to +1 A |
| Wind (cruise current) | −20 to +20% |
| Battery capacity | 80–100% of 5 Ah |
| True $R_0$, $R_1$, $C_1$, $R_2$ | each ±15% |

# Chapter 4

# Implementation

This chapter shows how the method of Chapter 3 is implemented in MATLAB. The simulator, the EKF, the charge-counting baseline, the return-to-home decision layer and the Monte Carlo study are all complete, and each is described with excerpts from the code. The full code is listed in Appendix A.

## 4.1 Tools and code structure

The project uses base MATLAB only; no toolbox is required. (The Control System Toolbox's built-in `extendedKalmanFilter` may be used later as an optional cross-check.) Every run fixes the random-number seed with `rng`, so it produces exactly the same random numbers each time and the results can be reproduced. Runs that are compared with each other use the same seed, so they see exactly the same sensor noise.

The code is split into short files that each do one job, so that each can be understood and tested on its own. Table 4.1 lists the files. All battery quantities in the code are per cell, as in Chapter 3.

The original plan had a single `simulate_truth.m` for the true battery and its sensors. In the implementation this job is split into the battery files (`battery_true_init.m`, `battery_true_step.m`), the sensor file (`read_sensors.m`) and the drone files (`drone_init.m`, `drone_step.m`). Keeping the drone separate from the battery lets a decision policy change the drone's mode without touching the battery code. `run_flight.m` joins all the pieces into one complete flight (Section 4.10).

## 4.2 Parameters

All numbers live in one function, `battery_params.m`, which returns a structure with three parts: `p.filt` (what the filter believes), `p.true` (the simulated battery) and `p.sens` (sensor imperfections). Listing 4.1 shows all three parts. The true values are written as multiples of the filter's values, so the size of each mismatch is visible in the code (Table 3.1). The model-error allowance $\sigma_{\text{model}}$ of (3.13) is stored with the filter's beliefs

Table 4.1: Code files in `drone_battery_ekf/`.

| File | Status | Purpose |
|---|---|---|
| `config/battery_params.m` | Done | Filter, truth and sensor parameters |
| `config/ocv_lookup.m` | Done | OCV and its slope at any SoC |
| `config/soc_from_ocv.m` | Done | Inverse lookup: resting voltage to SoC |
| `sim/battery_true_init.m` | Done | True battery at rest before take-off |
| `sim/battery_true_step.m` | Done | One step of the true 2-RC battery |
| `sim/read_sensors.m` | Done | Biased, noisy current and noisy voltage |
| `sim/make_mission.m` | Done | Routes, phase durations and currents |
| `sim/drone_init.m` | Done | Drone at home, ready for take-off |
| `sim/drone_step.m` | Done | Moves the drone one step; returns current demand |
| `sim/run_flight.m` | Done | One flight with truth, sensors, EKF and charge counting |
| `sim/flight_metrics.m` | Done | Error and consistency measures of one flight |
| `estimators/ekf_init.m` | Done | Initial state, covariance, stage and tuning |
| `estimators/ekf_step.m` | Done | One predict and update step |
| `estimators/coulomb_count.m` | Done | Charge-counting baseline |
| `main_day2_simulator.m` | Done | Simulator test flight and figures |
| `main_day3_ekf.m` | Done | Experiments E1 and E2 with stages A and B |
| `main_day4_experiments.m` | Done | Experiments E3–E8 |
| `learning/day1_ekf_by_hand.m` | Done | One-state EKF exercise (Section 2.3) |
| `decision/trip_soc.m` | Done | SoC needed to get home, (3.18) |
| `decision/p_safe_return.m` | Done | Margin to probability, (3.20)–(3.22) |
| `decision/rth_policy.m` | Done | The four turn-back policies A–D |
| `sim/draw_conditions.m` | Done | Random conditions of one Monte Carlo mission |
| `sim/run_mission_rth.m` | Done | One long mission with EKF and an RTH policy |
| `main_day5_rth.m` | Done | Demo mission, Monte Carlo study, threshold sweep |

(line 21), and the slow RC pair (lines 31–32) is at its mild default; experiment E6 changes it to the stress-test value.

```
15 % ---------- What the filter believes (nominal) ----------
16 p.filt.Cn         = 5.0;       % capacity (Ah)
17 p.filt.R0         = 0.008;     % ohmic resistance (ohm)
18 p.filt.R1         = 0.006;     % polarisation resistance (ohm)
19 p.filt.C1         = 5000;      % polarisation capacitance (F) -> tau = 30 s
20 p.filt.ocv_offset = 0;         % shift of the OCV curve (V)
21 p.filt.sig_model  = 0.010;     % allowance for model error in the voltage (V).
22                                %   The filter's model is never exact; this
23                                %   tells it not to trust its voltage
24                                %   prediction to better than about 10 mV.
25
26 % ---------- The "real" battery (truth) ----------
27 p.true.Cn         = 5.0;               % change this to test an aged battery
28 p.true.R0         = p.filt.R0 * 1.10;  % 10% higher than the filter thinks
29 p.true.R1         = p.filt.R1 * 0.90;  % 10% lower
30 p.true.C1         = p.filt.C1 * 1.10;  % 10% higher
31 p.true.R2         = 0.0015;            % second, slow RC pair the filter does not
32 p.true.C2         = 266667;            %   know about (tau = R2*C2 = 400 s).
33                                       %   Mild on purpose; experiment E6 uses
34                                       %   R2 = 0.004 as a stress test.
35 p.true.ocv_offset = -0.005;            % real OCV curve sits 5 mV lower
36 p.true.V_cutoff   = 3.0;               % below this (per cell) = battery dead
37
38 % ---------- Sensors ----------
39 p.sens.bias = 0.5;              % current sensor reads 0.5 A too high (A)
40 p.sens.sigI = 0.2;              % current sensor noise (A, 1 sigma)
41 p.sens.sigV = 0.01;             % voltage sensor noise per cell (V, 1 sigma)
```

Listing 4.1: Filter, truth and sensor parameters (`config/battery_params.m`).

## 4.3 OCV lookup and initial SoC

`ocv_lookup.m` (Listing 4.2) returns the OCV and, when asked for a second output, its slope. It implements the table of Figure 2.1 with `pchip` interpolation and the finite-difference slope (3.4). The `pchip` fit is built only once, on the first call, and kept in a `persistent` variable (lines 16–21); later calls only evaluate it with `ppval`. This gives exactly the same values as calling `interp1` with `'pchip'` every time, but is much faster, which matters for the 300 flights of experiment E8 and the Monte Carlo study. The SoC is clamped to $[0, 1]$ first (line 24), so the function never extrapolates beyond the table. An optional `offset` argument shifts the curve; the simulator uses it to give the true battery its $-5$ mV offset, while the filter uses the nominal curve.

```
16 persistent pp
17 if isempty(pp)
18     soc_tab = [0 5 10 20 30 40 50 60 70 80 90 100] / 100;
19     ocv_tab = [3.30 3.45 3.55 3.63 3.68 3.73 3.78 3.85 3.93 4.00 4.08 4.18];
20     pp = pchip(soc_tab, ocv_tab);
21 end
22 if nargin < 2, offset = 0; end
23
24 s   = min(max(soc, 0), 1);                          % stay inside the table
```

```
25  ocv = ppval(pp, s) + offset;
26
27  if nargout > 1                                        % slope by finite difference
28      h  = 0.005;
29      hi = min(s + h, 1);
30      lo = max(s - h, 0);
31      slope = (ppval(pp, hi) - ppval(pp, lo)) ./ (hi - lo);
32  end
```

Listing 4.2: OCV value and slope (`config/ocv_lookup.m`).

Before take-off the battery is at rest, so its voltage equals the OCV. `soc_from_ocv.m` (Listing 4.3) uses this to give the filter its starting guess (Section 3.4.3). It evaluates the OCV curve on a fine grid of 1001 SoC values and interpolates in the reverse direction, from voltage to SoC. This works because the OCV curve rises monotonically, so each voltage belongs to exactly one SoC.

```
6  if nargin < 2, offset = 0; end
7  s   = linspace(0, 1, 1001);
8  soc = interp1(ocv_lookup(s, offset), s, V, 'linear', 'extrap');
9  soc = min(max(soc, 0), 1);
```

Listing 4.3: Starting SoC from the resting voltage (`config/soc_from_ocv.m`).

## 4.4 The true battery

`battery_true_step.m` (Listing 4.4) advances the true battery by one step. Its lines map one-to-one onto the equations of Section 3.2: lines 11–12 compute the decay factors $a_1$ and $a_2$ of (3.1); line 14 is the SoC update (3.2); lines 15–16 are the two RC updates (3.2)–(3.2); and line 18 is the terminal voltage (3.3), computed at the end of the step with the updated states. The state starts from `battery_true_init.m`, which sets the chosen initial SoC and $V_1 = V_2 = 0$.

```
11  a1 = exp(-dt / (pt.R1 * pt.C1));
12  a2 = exp(-dt / (pt.R2 * pt.C2));
13
14  b.soc = b.soc - I * dt / (3600 * pt.Cn);
15  b.V1  = a1 * b.V1 + pt.R1 * (1 - a1) * I;
16  b.V2  = a2 * b.V2 + pt.R2 * (1 - a2) * I;
17
18  V = ocv_lookup(b.soc, pt.ocv_offset) - pt.R0 * I - b.V1 - b.V2;
```

Listing 4.4: One step of the true 2-RC battery (`sim/battery_true_step.m`).

## 4.5 Sensors

`read_sensors.m` (Listing 4.5) implements the sensor model (3.5)–(3.5). MATLAB's `randn` draws a new standard normal sample at each call, so the noise is independent from step to step, while the bias stays constant.

```
Im = I + ps.bias + ps.sigI * randn;
Vm = V + ps.sigV * randn;
```

Listing 4.5: Sensor model (`sim/read_sensors.m`).

## 4.6 Drone and mission

`make_mission.m` returns the route and the phase durations and currents of Section 3.3. `drone_step.m` is a small state machine: the field `d.mode` holds the current phase, and each call returns the current that phase demands and moves to the next phase when the current one is finished. Listing 4.6 shows the first three phases. A decision policy makes the drone turn back simply by setting `d.mode = 'return'`; the `return` case then flies it home and lands it.

```
switch d.mode
    case 'takeoff'
        I = m.I_takeoff;
        if d.t_in_mode >= m.T_takeoff
            d = set_mode(d, 'cruise');
        end

    case 'cruise'
        I = m.I_cruise * (1 + m.wind);
        [d, arrived] = fly_towards(d, m.wp(d.wp_index, :), m.v_cruise, dt);
        if arrived
            d = set_mode(d, 'hover');
        end

    case 'hover'
        I = m.I_hover;
        if d.t_in_mode >= m.T_hover
            d.wp_index = d.wp_index + 1;
            if d.wp_index > size(m.wp, 1)
                d = set_mode(d, 'return');      % all deliveries done
            else
                d = set_mode(d, 'cruise');
            end
        end
```

Listing 4.6: Flight-phase state machine, first three phases (`sim/drone_step.m`).

Movement is handled by the local function `fly_towards` (Listing 4.7). Each step the drone moves $v_{\text{cruise}}\Delta t = 10\,\text{m}$ in a straight line towards its target. If the target is closer than one step, the drone is placed on it, so it never overshoots. The distance actually flown is accumulated in `d.dist_flown`, which is recorded with each mission.

```
function [d, arrived] = fly_towards(d, target, v, dt)
gap  = target - d.pos;
dist = norm(gap);
step = v * dt;
if dist <= step                     % close enough: land on the target
    d.pos = target;
    d.dist_flown = d.dist_flown + dist;
```

```
    arrived = true;
else
    d.pos = d.pos + step * gap / dist;
    d.dist_flown = d.dist_flown + step;
    arrived = false;
end
```

Listing 4.7: Straight-line movement towards a target (`sim/drone_step.m`).

## 4.7 The simulator test loop

`main_day2_simulator.m` flies one mission with no filter, to test the simulator. Listing 4.8 shows its main loop. Each second it (1) asks the drone for its current demand, (2) adds the slow random load of (3.6), (3) steps the true battery, (4) reads the sensors and (5) stops if the battery voltage falls below the cut-off while current flows, or when the drone has landed. The script then prints a summary and saves the figures used in Section 5.1.

```
wobble = 0;                              % small random load changes (gusts, manoeuvres)
dead = false;
for k = 1:N
    [drone, I_demand] = drone_step(drone, m, dt);
    if I_demand > 0
        wobble = 0.9*wobble + 0.3*randn;     % slowly varying, about +/-0.7 A
        I = max(I_demand + wobble, 0);
    else
        I = 0;
    end

    [batt, V] = battery_true_step(batt, I, p.true, dt);
    [Im, Vm]  = read_sensors(I, V, p.sens);

    rec.t(k) = k*dt;   rec.I(k) = I;   rec.Im(k) = Im;
    rec.V(k) = V;      rec.Vm(k) = Vm; rec.ocv(k) = ocv_lookup(batt.soc, p.true.ocv_offset);
    rec.soc(k) = batt.soc;  rec.pos(k,:) = drone.pos;  rec.mode(k) = drone.mode;

    if I > 0 && V < p.true.V_cutoff
        dead = true;  break;                 % battery died in the air
    end
    if strcmp(drone.mode, 'landed'), break; end
end
```

Listing 4.8: Main loop of the simulator test (`main_day2_simulator.m`).

The charge-counting check reported in Section 5.1 is computed after the flight from the logged measured current, as $\mathrm{SoC}_0 - \sum_k I_{m,k}\Delta t/(3600 C_n)$, which is (2.4) applied to the biased, noisy sensor reading.

## 4.8 EKF implementation

The filter is stored in a MATLAB structure `f` that holds the estimate `f.x`, the covariance `f.P` and the settings. Two functions operate on it: `ekf_init.m` creates it, and `ekf_step.m` runs one predict–update step. The state is always the five-element vector $[\mathrm{SoC}, V_1, b, C_n, R_0]$ of (3.7), whatever the stage.

### 4.8.1 Initialisation and stages

Listing 4.9 shows the core of `ekf_init.m`. Lines 26–28 build the measurement-noise variance (3.13); the voltage noise is called `Rv` in the code so that it is never confused with the resistors. The current-noise term uses the nominal $R_0$, so `Rv` is a constant. Lines 31–32 set the initial estimate (3.14) and the initial standard deviations of $P_0$ in (3.14), and line 36 the diagonal process-noise terms of Table 3.2. Lines 38–45 implement the stages of Table 3.3: the vector `learn` marks which states a stage learns, and lines 44–45 set the initial standard deviation and the process noise of every other state to zero, which freezes it at its nominal value (Section 3.4.4). Line 49 builds $P_0$ from the squared standard deviations.

```
24 % Noise settings
25 f.sigI = p.sens.sigI;                                % current sensor noise (A)
26 f.Rv   = p.sens.sigV^2 ...                           % voltage sensor noise
27        + (p.filt.R0 * p.sens.sigI)^2 ...             % current noise seen through
       R0
28        + p.filt.sig_model^2;                         % allowance for model error (V
       ^2)
29
30 % Starting guess and uncertainty (1 sigma) for [SoC, V1, b, Cn, R0]
31 x0  = [soc0; 0; 0; p.filt.Cn; p.filt.R0];
32 sd0 = [sig_soc0, 0.01, 1.0, 0.5, 0.004];             % SoC, 10 mV, 1 A, 0.5 Ah, 4
       mOhm
33
34 % Process noise per step: how much each state may drift in ways the
35 % model does not explain (variances)
36 q = [1e-8, 1e-6, 1e-6, 1e-7, 1e-9];
37
38 switch f.stage
39     case 'A', learn = [1 1 0 0 0];
40     case 'B', learn = [1 1 0 0 1];
41     case 'C', learn = [1 1 1 1 1];
42     otherwise, error('Stage must be ''A'', ''B'' or ''C''.');
43 end
44 sd0(~learn) = 0;
45 q(~learn)   = 0;
46
47 f.learn = logical(learn);
48 f.x = x0;
49 f.P = diag(sd0.^2);
50 f.q = q;
```

Listing 4.9: Noise settings, initial values and stages (`estimators/ekf_init.m`).

### 4.8.2 Predict

Listing 4.10 shows the prediction half of `ekf_step.m`. Line 12 forms the bias-corrected current $I_m - \hat{b}$ and line 13 the decay factor $a$. Lines 16–20 are the process model (3.8)–(3.10) without noise. Lines 22–26 build the Jacobian $F_k$ of (3.12): the identity matrix plus its four non-trivial entries. Line 28 is the noise-input vector $G_k$ of (3.13), line 29 the process noise $Q_k$ of (3.13), and line 30 the covariance prediction (2.7).

```
11 soc = f.x(1);  v1 = f.x(2);  b = f.x(3);  Cn = f.x(4);  R0 = f.x(5);
12 I = Im - b;                                    % bias-corrected current
13 a = exp(-dt / (f.R1 * f.C1));                  % how much of V1 survives one step
14
15 % ---------- 1. Predict: move the estimate with the model ----------
16 x_pred = [soc - I*dt/(3600*Cn);                % charge counting
17           a*v1 + f.R1*(1 - a)*I;               % slow voltage relaxes toward R1*I
18           b;                                   % bias, capacity and resistance
19           Cn;                                  %   stay the same (apart from
20           R0];                                 %   slow random wander)
21
22 F = eye(5);                                     % Jacobian of the process model
23 F(1,3) = dt/(3600*Cn);                         %   dSoC/db
24 F(1,4) = I*dt/(3600*Cn^2);                     %   dSoC/dCn
25 F(2,2) = a;                                    %   dV1/dV1
26 F(2,3) = -f.R1*(1 - a);                        %   dV1/db
27
28 G = [-dt/(3600*Cn); f.R1*(1 - a); 0; 0; 0];   % how current-sensor noise enters
29 Q = G * f.sigI^2 * G' + diag(f.q);
30 P_pred = F * f.P * F' + Q;                     % uncertainty grows
```

Listing 4.10: EKF prediction (`estimators/ekf_step.m`).

### 4.8.3 Compare and update

Listing 4.11 shows the second half. Line 33 reads the OCV and its slope at the predicted SoC. Lines 34–35 compute the expected voltage $h(\hat{x}_k^-, u_k)$ of (3.11), using the predicted bias and resistance, and line 36 the innovation (2.8). Line 38 is the Jacobian $H_k$ of (3.12) and line 39 the scalar $S_k$ of (2.2). Because $S_k$ is a single number, line 42 computes the gain with a division instead of a matrix inverse, and line 43 updates the state (2.9). Lines 44–46 are the Joseph-form covariance update (3.15) followed by symmetrisation. Lines 48–50 apply the safeguards of Section 3.4.3. The function also returns the innovation, $S_k$, the gain and the NIS (line 54) for the consistency checks.

For a frozen state the rows and columns of $P$ are zero, so its entry of `K` is zero and line 43 never moves it. The same code therefore serves all three stages without any `if` statements.

```
32 % ---------- 2. Compare: expected vs measured voltage ----------
33 [ocv, docv] = ocv_lookup(x_pred(1), f.ocv_offset);
34 I_pred = Im - x_pred(3);
35 V_pred = ocv - x_pred(2) - x_pred(5)*I_pred;
36 y = Vm - V_pred;                                % innovation (the surprise)
37
38 H = [docv, -1, x_pred(5), 0, -I_pred];         % Jacobian of the measurement model
```

```
39 S = H * P_pred * H' + f.Rv;                    % expected size of the surprise (one
       number)
40
41 % ---------- 3. Update: move by K x surprise ----------
42 K = P_pred * H' / S;                            % Kalman gain (5x1)
43 x = x_pred + K * y;
44 IKH = eye(5) - K*H;
45 P = IKH * P_pred * IKH' + K * f.Rv * K';       % Joseph form: stays symmetric and
       positive
46 P = (P + P') / 2;
47
48 x(1) = min(max(x(1), 0), 1);                    % SoC stays in [0, 1]
49 x(4) = max(x(4), 0.5);                          % capacity stays positive
50 x(5) = max(x(5), 1e-4);                         % resistance stays positive
```

Listing 4.11: EKF compare and update (`estimators/ekf_step.m`).

## 4.9 Charge-counting baseline

`coulomb_count.m` (Listing 4.12) is Equation (2.4) applied to the measured current. It is called with the nominal capacity and starts from the same initial guess as the EKF, so the two estimators differ only in whether they use the voltage.

```
4 soc = soc - Im * dt / (3600 * Cn);
```

Listing 4.12: Charge-counting baseline (`estimators/coulomb_count.m`).

## 4.10 One complete flight

`run_flight.m` flies one mission with the true battery, the sensors, the EKF and charge counting together, and records everything each second. The experiment is described by a small options structure: the true starting SoC, the estimators' starting guess, the initial SoC uncertainty, the EKF stage and the random seed.

Listing 4.13 shows what happens before take-off. Lines 23–25 read three optional settings, with defaults that reproduce the first E1 and E2 runs (Section 5.2.1) exactly. The battery is at rest, so lines 29–34 read the voltage sensor with zero current `n_rest` times and average the readings. Each reading is the OCV plus independent noise, so averaging 10 readings cuts the noise by $\sqrt{10} \approx 3$ (Section 3.4.3); the default of one reading keeps those runs reproducible. If the experiment does not prescribe a starting guess (`NaN`), line 37 converts the averaged resting voltage into an SoC with `soc_from_ocv`, using the filter's nominal OCV curve. Lines 41–43 serve the tuning experiment E5: they multiply the process noise by `q_scale` and the measurement noise by `r_scale`. Because $Q_k = G_k \sigma_I^2 G_k^\top + \mathrm{diag}(q)$ by (3.13), scaling the diagonal terms `f.q` by `q_scale` and $\sigma_I$ by $\sqrt{\texttt{q_scale}}$ scales the whole of $Q_k$ by exactly `q_scale`. The EKF and charge counting then start from the same guess (lines 40 and 44).

```
23 n_rest  = get_opt(opts, 'n_rest', 1);
24 q_scale = get_opt(opts, 'q_scale', 1);
```

```
r_scale = get_opt(opts, 'r_scale', 1);

% ---- Before takeoff: battery at rest, so voltage = OCV ----
batt = battery_true_init(opts.soc_true0);
V_rest = ocv_lookup(opts.soc_true0, p.true.ocv_offset);
Vm0 = 0;
for i = 1:n_rest                                % average a few readings to beat the
    noise
    [~, Vm_i] = read_sensors(0, V_rest, p.sens);
    Vm0 = Vm0 + Vm_i / n_rest;
end
soc_guess0 = opts.soc_guess0;
if isnan(soc_guess0)
    soc_guess0 = soc_from_ocv(Vm0, p.filt.ocv_offset);
end

f      = ekf_init(p, opts.stage, soc_guess0, opts.sig_soc0);
f.q    = f.q * q_scale;                         % tuning experiments (E5)
f.sigI = f.sigI * sqrt(q_scale);
f.Rv   = f.Rv * r_scale;
soc_cc = soc_guess0;                            % charge counting starts from the
    same guess
drone  = drone_init(m);
```

Listing 4.13: Before take-off: options, averaged resting voltage and tuning scales (`sim/run_flight.m`).

The flight loop is the simulator test loop of Listing 4.8, with the two estimators added after the sensors (Listing 4.14). The estimators receive only the measured current `Im` and voltage `Vm`, never the true values.

```
    [batt, V] = battery_true_step(batt, I, p.true, dt);   % truth
    [Im, Vm]  = read_sensors(I, V, p.sens);               % what the drone sees
    [f, info] = ekf_step(f, Im, Vm, dt);                  % EKF
    soc_cc    = coulomb_count(soc_cc, Im, p.filt.Cn, dt); % baseline
```

Listing 4.14: Each second: truth, sensors, EKF and baseline (`sim/run_flight.m`).

`main_day3_ekf.m` uses `run_flight` to run experiments E1 and E2 with stages A and B (Listing 4.15). Both stages use the same seed in each experiment, so they see exactly the same noise and any difference between them comes from the filter alone. The script then prints a table of SoC errors and consistency measures and saves the figures for Sections 5.2.1 and 5.2.2.

```
% E1: battery at 95%, starting guess read from the resting voltage
E1 = struct('soc_true0', 0.95, 'soc_guess0', NaN,  'sig_soc0', 0.03, 'seed', 2);
% E2: battery really at 85%, but everyone assumes it is full
E2 = struct('soc_true0', 0.85, 'soc_guess0', 1.00, 'sig_soc0', 0.20, 'seed', 2);

E1.stage = 'A';  r1A = run_flight(p, m, E1);
E1.stage = 'B';  r1B = run_flight(p, m, E1);
E2.stage = 'A';  r2A = run_flight(p, m, E2);
E2.stage = 'B';  r2B = run_flight(p, m, E2);
```

Listing 4.15: Experiment definitions for E1 and E2 (`main_day3_ekf.m`).

## 4.11 Performance measures

`flight_metrics.m` (Listing 4.16) turns one recorded flight into the numbers reported in Chapter 5, for either the EKF or charge counting. Line 16 forms the SoC error (estimate minus truth). Lines 18–22 compute the RMSE, the error at landing, the largest absolute error and the convergence time (the last time the error was 2% or more). For the EKF, lines 27–29 add the three consistency measures of Section 3.4.3: the share of time the truth was inside $\pm 3\sqrt{P_{11}}$, the mean NIS, and the mean NEES on the SoC, $e^2/P_{11}$, which should average about 1. Charge counting has no uncertainty, so these are set to `NaN` for it.

```
15 if strcmp(which, 'cc'), est = r.soc_cc; else, est = r.x(1,:); end
16 e = est - r.soc;
17
18 M.rmse   = sqrt(mean(e.^2));
19 M.final  = e(end);
20 M.maxabs = max(abs(e));
21 bad = find(abs(e) >= 0.02, 1, 'last');
22 if isempty(bad), M.conv = 0; else, M.conv = r.t(bad); end
23
24 if strcmp(which, 'cc')
25     M.inside = NaN;  M.nis = NaN;  M.nees = NaN;
26 else
27     M.inside = mean(abs(e) <= 3*sqrt(r.Pd(1,:)));
28     M.nis    = mean(r.nis);
29     M.nees   = mean(e.^2 ./ r.Pd(1,:));
30 end
```

Listing 4.16: Performance measures of one flight (`sim/flight_metrics.m`).

## 4.12 Experiments E3 to E8

`main_day4_experiments.m` runs experiments E3–E8 of Table 3.6. Each scenario is a pair of a parameter structure and an options structure, built from the nominal ones by changing only what the scenario tests (Listing 4.17). All start from the normal-flight options on lines 19–20, which include averaging 10 resting readings. E3 changes the sensor bias (line 29), E4 the true capacity and starting SoC (line 32), and E6 the slow RC pair (line 35).

```
18 % Normal flight settings; every scenario starts from these
19 base = struct('soc_true0', 0.95, 'soc_guess0', NaN, 'sig_soc0', 0.03, ...
20               'seed', 2, 'n_rest', 10);   % n_rest: average 10 resting readings
       before takeoff
21
22 %% Scenarios
23 sc = {};
24 sc{end+1} = scenario('E1', 'Normal flight',          p0, base);
25
26 o = base;  o.soc_true0 = 0.85;  o.soc_guess0 = 1.00;  o.sig_soc0 = 0.20;
27 sc{end+1} = scenario('E2', 'Wrong start (85%, assumed full)', p0, o);
28
29 p = p0;  p.sens.bias = 0.8;
```

```
30 sc{end+1} = scenario('E3', 'Sensor bias 0.8 A',      p, base);
31
32 p = p0;  p.true.Cn = 0.85 * p0.true.Cn;  o = base;  o.soc_true0 = 1.00;
33 sc{end+1} = scenario('E4', 'Aged battery (85% capacity)', p, o);
34
35 p = p0;  p.true.R2 = 0.004;  p.true.C2 = 1e5;        % slow RC 2.7x bigger (tau
       400 s)
36 sc{end+1} = scenario('E6', 'Strong model mismatch',  p, base);
```

Listing 4.17: Scenario definitions (`main_day4_experiments.m`).

E3, E4, E6 and E7 are single runs with seed 2. E5 reruns stage B on the E1 scenario with the five `[q_scale, r_scale]` pairs `[1 1; 0.01 1; 100 1; 1 0.01; 1 100]`. E8 (Listing 4.18) flies every scenario with seeds 1 to 20 and every stage: $5 \times 20 \times 3 = 300$ flights. Inside each scenario and seed, the three stages see exactly the same noise, because `run_flight` resets the random generator with the seed. Charge counting is recorded once per scenario and seed (line 142), from the stage-A run; it does not depend on the stage. The script prints the mean results, saves the figures, and writes the summary table to `results/day4/e8_summary.csv`.

```
137 for i = 1:nS
138     for j = 1:nSeed
139         for k = 1:numel(stages)
140             o = sc{i}.opts;  o.seed = seeds(j);  o.stage = stages{k};
141             r = run_flight(sc{i}.p, m, o);
142             if k == 1, R = store(R, i, 1, j, flight_metrics(r, 'cc')); end
143             R = store(R, i, k+1, j, flight_metrics(r, 'ekf'));
144         end
145     end
146     fprintf('  %s done (%.0f s)\n', sc{i}.code, toc);
147 end
```

Listing 4.18: Stage comparison over 20 seeds (`main_day4_experiments.m`).

## 4.13 Decision rule and policies

The decision layer has three short functions in `decision/`. `trip_soc.m` (Listing 4.19) is Equation (3.18): line 15 is the cruise time (3.18), line 16 the charge for cruising home (with the bias removed from the measured cruise current) and for landing (with the nominal landing current as is), and line 17 divides by the capacity. The same function serves the EKF policies, called with $\hat{b}$ and $\hat{C}_n$, and policy B, called with $b = 0$ and the rated 5 Ah.

```
15 t_cruise = dist_home / m.v_cruise;                    % s
16 charge   = (I_cruise - b) * t_cruise + m.I_land * m.T_land;   % A*s
17 dsoc     = charge / (3600 * Cn);
```

Listing 4.19: SoC needed to get home (`decision/trip_soc.m`).

`p_safe_return.m` (Listing 4.20) takes the stage-C filter and computes the margin (3.20) (line 17), the two standard deviations and their combination (3.21) (lines 19–21), and the probability (3.22) with `erfc` (line 23).

```
soc = f.x(1);  b = f.x(3);  Cn = f.x(4);
dsoc   = trip_soc(dist_home, I_cruise, b, Cn, m);
margin = soc - dsoc - rp.reserve;

sig_soc   = sqrt(f.P(1,1));          % from the EKF: how unsure is the SoC
sig_trip  = rp.trip_frac * dsoc;     % wind and load surprises on the way home
sig_total = sqrt(sig_soc^2 + sig_trip^2);

ps = 0.5 * erfc(-margin / (sqrt(2) * sig_total));   % normal CDF, base MATLAB
```

Listing 4.20: Probability of a safe return (`decision/p_safe_return.m`).

`rth_policy.m` (Listing 4.21) implements the four rules of Table 3.5. It receives a small structure `s` with what the drone knows this second. Policy A needs a memory: `low_count` counts consecutive seconds below the voltage threshold and is reset by any reading above it (line 14), so a single noisy dip cannot trigger a return.

```
low_count = s.low_count;
switch policy
    case 'A'   % voltage stays below the threshold for V_hold seconds
        if s.Vm < rp.V_thr, low_count = low_count + 1; else, low_count = 0; end
        go_home = low_count >= rp.V_hold;
    case 'B'   % charge counting says the margin is gone
        go_home = s.soc_cc - s.dsoc_cc - rp.reserve < 0;
    case 'C'   % EKF estimate says the margin is gone
        go_home = s.margin < 0;
    case 'D'   % EKF is no longer sure enough of getting home with the reserve
        go_home = s.ps < rp.p_thr;
    otherwise
        error('Unknown policy "%s".', policy);
end
```

Listing 4.21: The four turn-back policies (`decision/rth_policy.m`).

## 4.14 One mission with a return-to-home policy

`run_mission_rth.m` flies the long route under given conditions and lets one policy decide. Listing 4.22 shows the start. Lines 28–35 read the starting SoC from 10 averaged resting readings. Line 37 starts the stage-C filter from that guess with $\sigma_{\mathrm{SoC},0} = 3\%$, and line 38 starts charge counting from the same guess, so policy B is not handicapped by a wrong start. Line 40 initialises the learned cruise current at the nominal 15 A.

```
% ---- Before takeoff: average 10 resting readings for the starting guess ----
batt = battery_true_init(c.soc0);
V_rest = ocv_lookup(c.soc0, p.true.ocv_offset);
Vm0 = 0;
for i = 1:10
    [~, Vm_i] = read_sensors(0, V_rest, p.sens);
    Vm0 = Vm0 + Vm_i / 10;
end
soc_guess0 = soc_from_ocv(Vm0, p.filt.ocv_offset);

f      = ekf_init(p, 'C', soc_guess0, 0.03);     % stage C drives the decision
```

```
38 soc_cc = soc_guess0;                                % charge counting starts from the
       same guess
39 drone  = drone_init(m);
40 I_cr   = m.I_cruise;                                % learned cruise current (as
       measured)
```

Listing 4.22: Start of a return-to-home mission (`sim/run_mission_rth.m`).

Listing 4.23 shows each second of the flight. After the truth, sensors and both estimators (lines 59–62), line 64 updates the learned cruise current (3.19), only while cruising. Line 69 computes $P_{\text{safe}}$ every second, so it can be recorded for plots whatever the policy. Lines 70–79 make the decision, only while the drone is cruising or hovering. Policy B's trip cost is computed on line 73 with no bias correction and the rated capacity. When the policy fires, lines 76–77 switch the drone to the return phase and record the turn-back time; `drone_step` then flies it home and lands it (Section 4.6). The mission counts as died if, while current flows, the voltage falls below 3.0 V or the true SoC reaches zero.

```
59     [batt, V] = battery_true_step(batt, I, p.true, dt);
60     [Im, Vm]  = read_sensors(I, V, p.sens);
61     f         = ekf_step(f, Im, Vm, dt);
62     soc_cc    = coulomb_count(soc_cc, Im, p.filt.Cn, dt);
63     if strcmp(drone.mode, 'cruise')
64         I_cr = 0.98*I_cr + 0.02*Im;                % slowly learn the real cruise
       current
65     end
66
67     % ---- What the drone knows, and the decision ----
68     dist_home = norm(drone.pos - m.home);
69     [ps, margin, dsoc] = p_safe_return(f, dist_home, I_cr, m, rp);
70     if any(strcmp(drone.mode, {'cruise', 'hover'}))       % decide only while
       working
71         s.Vm = Vm;  s.low_count = low_count;  s.ps = ps;  s.margin = margin;
72         s.soc_cc  = soc_cc;
73         s.dsoc_cc = trip_soc(dist_home, I_cr, 0, p.filt.Cn, m);   % no bias, label
        capacity
74         [go_home, low_count] = rth_policy(policy, s, rp);
75         if go_home
76             drone.mode = 'return';  drone.t_in_mode = 0;
77             out.t_rth = k;
78         end
79     end
```

Listing 4.23: Each second: estimators, cruise-current learning and decision (`sim/run_mission_rth.m`).

## 4.15 Monte Carlo study and threshold sweep

`draw_conditions.m` (Listing 4.24) draws the random conditions of one mission, uniformly within the ranges of Table 3.7.

```
4 u = @(a, b) a + (b - a) * rand;
5
6 c.soc0  = u(0.60, 1.00);              % starting charge
```

```
c.bias  = u(-1.0, 1.0);              % current-sensor bias (A)
c.wind  = u(-0.20, 0.20);            % cruise needs -20%..+20% current
c.Cn    = p.true.Cn * u(0.80, 1.00); % battery age: 80-100% of rated capacity
c.R0    = p.true.R0 * u(0.85, 1.15); % the real battery's parameters vary too
c.R1    = p.true.R1 * u(0.85, 1.15);
c.C1    = p.true.C1 * u(0.85, 1.15);
c.R2    = p.true.R2 * u(0.85, 1.15);
```

Listing 4.24: Random conditions of one mission (`sim/draw_conditions.m`).

`main_day5_rth.m` has three parts. Part 1 flies the demonstration mission with each policy (seed 7). Part 2 is the Monte Carlo study (Listing 4.25): line 109 fixes the seed for drawing the conditions, line 111 draws all 500 conditions once, and line 119 flies every mission with every policy using the same conditions and the same noise seed, $10\,000 + i$. For the policy-D flights, line 121 also stores the estimators' accuracy (stage-C and charge-counting SoC RMSE, and the share of time the truth was inside the stage-C $\pm 3\sigma$ band), which `run_mission_rth` computes at the end of each flight. Part 3 is the threshold sweep, which reruns policies D and A on the first 200 missions with the thresholds of Section 3.6. The settings used by all parts are on lines 28–32 of the script ($\mathrm{SoC}_{\mathrm{res}} = 10\%$, $f_{\mathrm{trip}} = 0.10$, $p^* = 0.95$, $V_{\mathrm{thr}} = 3.5\,\mathrm{V}$ held for 5 s). A local function computes the scores of Section 3.5; the script saves four figures and two CSV tables in `results/day5/`.

```
109 rng(2026);
110 cond = cell(N_MC, 1);
111 for i = 1:N_MC, cond{i} = draw_conditions(p); end
112
113 died = false(N_MC, 4);  land = nan(N_MC, 4);  drops = nan(N_MC, 4);
114 acc = nan(N_MC, 3);                                % EKF RMSE, CC RMSE, EKF
        inside +/-3 sigma (policy D flights)
115 fprintf('\n==== Part 2: Monte Carlo, %d missions x 4 policies ====\n', N_MC);
116 tic;
117 for i = 1:N_MC
118     for j = 1:4
119         o = run_mission_rth(p, m, cond{i}, policies{j}, rp, 10000 + i);   % same
        noise for every policy
120         died(i,j) = o.died;  land(i,j) = o.soc_land;  drops(i,j) = o.drops;
121         if j == 4, acc(i,:) = [o.rmse_ekf, o.rmse_cc, o.inside]; end
122     end
123     if mod(i, 50) == 0, fprintf('  %d / %d missions (%.0f s)\n', i, N_MC, toc);
        end
124 end
125 S = summarise(died, land, drops, rp.reserve);
```

Listing 4.25: Monte Carlo loop (`main_day5_rth.m`).

# Chapter 5

# Results

This chapter first checks that the simulator behaves as intended (Section 5.1). It then presents the estimation experiments E1–E8 (Section 5.2), the decisions of the four policies on one flight (Section 5.3), the Monte Carlo comparison (Section 5.4) and the risk-threshold sweep (Section 5.5).

## 5.1 Simulator verification

Before any filter is added, the simulator must produce believable data: the voltage should sag under load, change in the right direction at each phase change, and stay above the cut-off on a mission that is meant to be safe. This section reports one test flight made with `main_day2_simulator.m` (Section 4.7).

### 5.1.1 Set-up

The drone flew the *short* mission (three drop-offs, Section 3.3) with no wind. The true battery started at rest at 95% SoC and used the true parameters of Table 3.1, except that the slow RC pair was at its first setting, $R_2 = 4\,\mathrm{m\Omega}$ and $C_2 = 100\,000\,\mathrm{F}$. This setting was later kept only for the E6 stress test (Section 3.2). The current sensor had a bias of $+0.5\,\mathrm{A}$ and noise of $0.2\,\mathrm{A}$; the voltage sensor had noise of $10\,\mathrm{mV}$. The random seed was fixed (`rng(2)`). No estimator was running; charge counting was applied to the logged measured current after the flight. Table 5.1 summarises the flight.

### 5.1.2 OCV curve

Figure 5.1 shows the OCV curve that the filter believes and the true battery's curve, $5\,\mathrm{mV}$ lower. The two are almost indistinguishable by eye, yet $5\,\mathrm{mV}$ corresponds to about 1% SoC in the flat middle of the curve, so even this small offset is a real test for the filter. The right-hand axis shows the slope that becomes the first entry of $H$ in (3.12). It is several times larger near empty than in the middle, so the voltage corrects the SoC

Table 5.1: Simulator test flight: summary (short mission, one run).

| Quantity | Value |
|---|---|
| Outcome | Landed safely |
| Flight time | 14.2 min |
| Distance flown | 6.89 km |
| True SoC, start → landing | 95% → 18.5% |
| Lowest cell voltage | 3.297 V (cut-off 3.0 V) |
| Largest voltage sag (OCV − terminal voltage) | 367 mV per cell, at take-off (30 A) |
| Charge-counting SoC error at landing | −2.3% |

much more strongly at low charge.

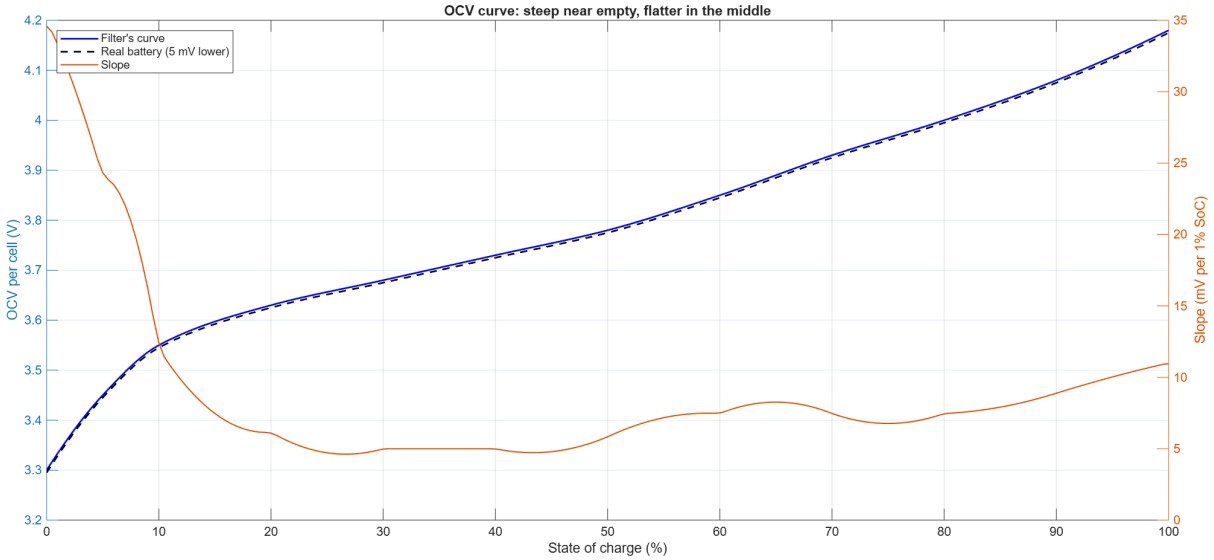



Figure 5.1: OCV curve of the filter (solid) and of the true battery (dashed, 5 mV lower), with the slope $d\,\mathrm{OCV}/d\,\mathrm{SoC}$ on the right axis. Notice that the slope, which sets how much the voltage can tell the EKF about the SoC, is high below about 10% SoC and low between about 20% and 50%.

### 5.1.3 Mission and current

Figure 5.2 shows the flight path and the current. Two simple checks confirm that the drone model works. First, the straight legs home → drop 1 → drop 2 → drop 3 → home add up to 6.89 km, the same as the logged distance flown, so the drone neither overshoots its targets nor cuts corners. Second, 6.89 km at 10 m/s takes about 689 s; adding 30 s of take-off, three 30 s hovers and 40 s of landing gives about 849 s, or 14.2 min, which matches the logged flight time.

The current profile shows each phase clearly: 30 A at take-off, about 15 A in cruise, three 20 A blocks while hovering at the drop-offs, and 18 A while landing. The slow random load of (3.6) makes the current wander slowly, by about 0.7 A (one standard deviation). The measured current (grey) sits slightly above the true current (blue), because of the

+0.5 A bias, and is noisier.

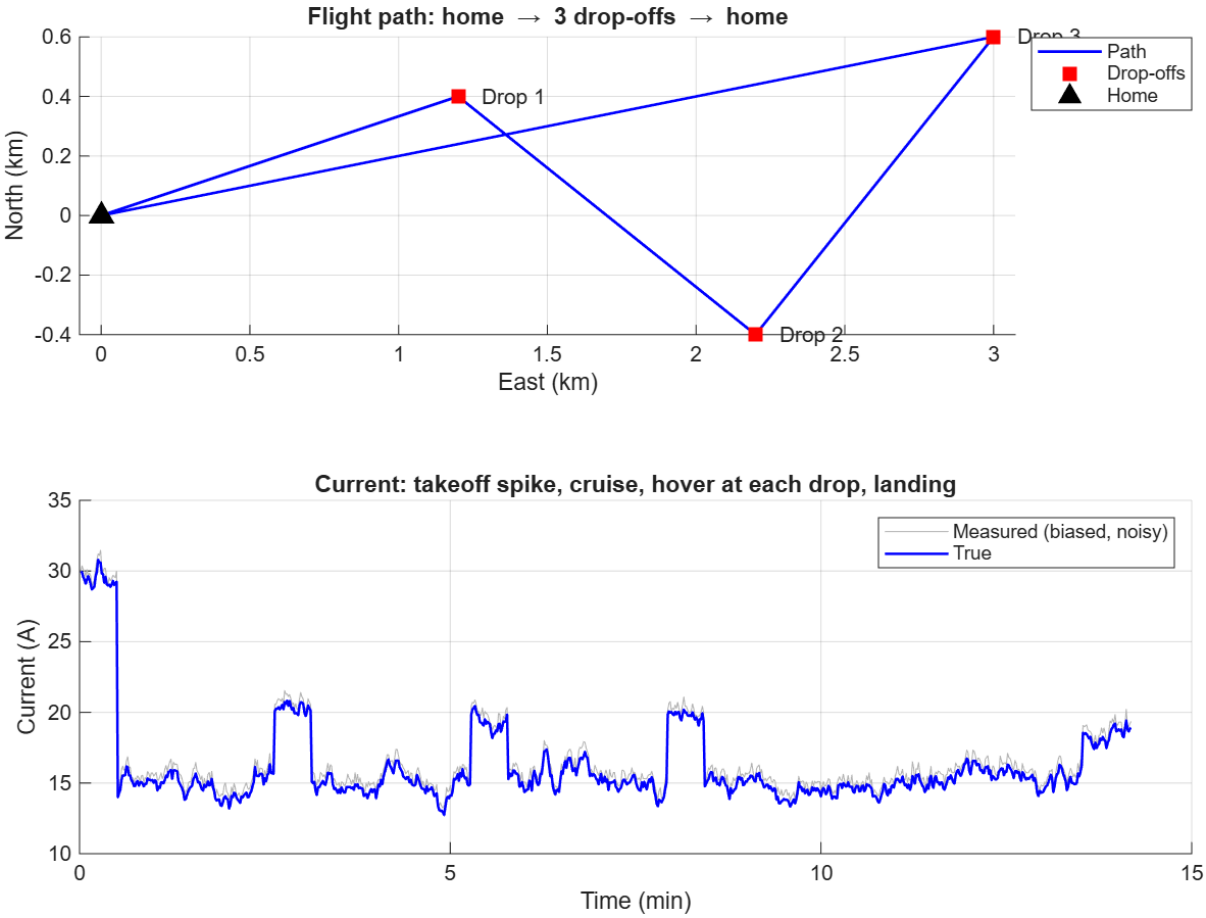



Figure 5.2: Simulator test flight. Top: flight path from home through three drop-offs and back. Bottom: true current (blue) and measured current (grey). Notice the take-off spike, the three hover blocks and the landing phase, and that the measured current runs slightly above the truth because of the sensor bias.

### 5.1.4 Voltage and state of charge

Figure 5.3 shows the voltages and the true SoC. The true SoC falls from 95% to 18.5%, steadily in cruise and slightly faster during take-off and hover, when the current is higher.

**Voltage sag.** The gap between the OCV (dashed) and the terminal voltage (blue) is the sag. It is largest at take-off, 367 mV per cell at 30 A. Of this, the ohmic part alone (2.1) is $R_0 I = 8.8\,\text{m}\Omega \times 30\,\text{A} = 264\,\text{mV}$; the rest is polarisation that builds up in the RC pairs during the 30 s climb. A simple voltage gauge would therefore read the battery as much emptier than it is during take-off.

**Steps at phase changes.** When take-off ends and the current falls from 30 A to 15 A, the voltage steps *up*; at each drop-off, when the current rises to 20 A for hovering, it steps *down*. The instant part of each step is $R_0 \Delta I$, about 132 mV when take-off ends and 44 mV at the start of each hover, and the rounded part that follows is the RC voltage settling. This is the behaviour the circuit model of Figure 2.2 predicts.

**Slowly widening gap.** Over the flight, the gap between the OCV and the terminal voltage slowly widens, even in cruise at a nearly constant current. This is the slow RC pair ($\tau_2 = 400\,\mathrm{s}$), which exists only in the true battery and keeps charging up for most of the flight. It is the deliberate model mismatch of Section 3.2: the EKF, with its single 30 s RC pair, cannot represent this slow drift and has to absorb it in its other states. This observation, together with a rough estimate of the settled extra drop, led to a milder default slow pair for the main experiments, with the original setting kept as the E6 stress test.

**Safety.** The lowest cell voltage, 3.297 V near the end of the flight, stays about 0.3 V above the 3.0 V cut-off, and the drone landed with 18.5% SoC. The short mission is therefore safe, as intended, and leaves the long mission to test the RTH rule.

The figure also hints at the weakness of policy A (Table 3.5). The measured voltage first stayed below 3.6 V (the threshold first planned for policy A) for 5 consecutive seconds at 5.4 min, during the hover at the second drop-off, when the true SoC was still 64.9%. A voltage-threshold rule at that level would have turned back with most of the battery unused. This is one reason the final policy A uses 3.5 V and is swept over a range of thresholds (Section 3.5).

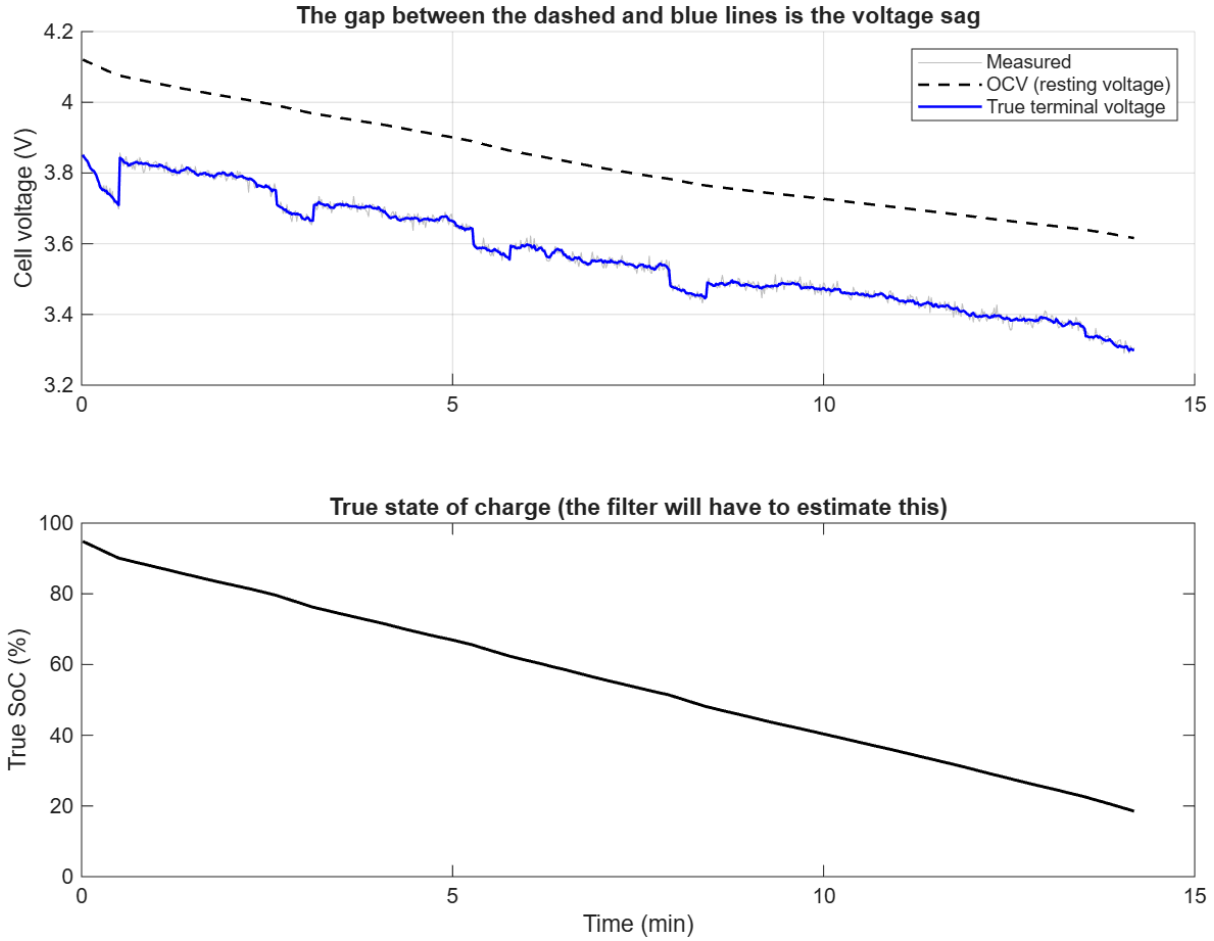



Figure 5.3: Simulator test flight. Top: OCV (dashed), true terminal voltage (blue) and measured voltage (grey), per cell. Bottom: true SoC. Notice the large sag at take-off, the step up when take-off ends, the steps down during each hover, and the gap that slowly widens through the flight because of the slow RC pair that only the true battery has.

### 5.1.5 Charge counting with a biased sensor

Charge counting (2.4), applied to the measured current, ended the flight 2.3% below the true SoC. Equation (2.5) predicts this: a 0.5 A bias over 14.2 min removes about 0.118 Ah, or about 2.4% of 5 Ah. The sign is negative because the sensor reads too high, so the counter believes more charge has been used than really has. The small difference from 2.4% is due to the sensor noise. This drift grows linearly with time and would be larger on a longer flight or with a larger bias, and it is one of the errors the EKF is designed to remove.

### 5.1.6 Summary of the simulator check

The simulator behaves as intended. The voltage sags realistically under load, it steps in the correct direction at every phase change, the drone's distance and flight time agree with the route, the charge-counting drift matches theory, and the deliberate mismatch between the true battery and the filter's model is visible in the data. This check is based on a single run with one random seed; it verifies behaviour, not statistics.

## 5.2 Estimation experiments

Experiments E1 and E2 were run with `main_day3_ekf.m` (Section 4.10) on the short mission, with the default true battery of Table 3.1 (slow pair $R_2 = 1.5\,\mathrm{m\Omega}$), the sensor errors of Section 3.3 (current bias +0.5 A) and random seed 2. Three estimators were compared on exactly the same sensor readings: charge counting, EKF stage A (learns SoC and $V_1$) and EKF stage B (also learns $R_0$). Both EKF stages used the model-error allowance $\sigma_{\mathrm{model}} = 10\,\mathrm{mV}$. Table 5.2 summarises the results. The convergence time is the last time the SoC error was 2% or more; "never" means the error was still at least 2% at landing.

Table 5.2: SoC estimation in E1 and E2 (MATLAB, short mission, seed 2). Errors are estimate minus truth, in percentage points of SoC. Final: error at landing. Max: largest absolute error. In $\pm 3\sigma$: share of time the truth was inside the EKF's band. NIS: mean NIS. $< 2\%$ after: time after which the error stayed below 2%.

| | Estimator | RMSE | Final | Max | In $\pm 3\sigma$ | NIS | $< 2\%$ after |
|---|---|---|---|---|---|---|---|
| E1 | Charge counting | 4.36% | −5.50% | 5.50% | – | – | – |
| | EKF stage A | 2.51% | −3.89% | 3.90% | 17% | 0.54 | never |
| | EKF stage B | 0.97% | −1.53% | 3.76% | 100% | 0.51 | 30 s |
| E2 | Charge counting | 13.85% | +12.64% | 15.00% | – | – | – |
| | EKF stage A | 2.49% | −2.49% | 3.62% | 18% | 0.60 | never |
| | EKF stage B | 1.32% | −0.38% | 7.52% | 100% | 0.51 | 42 s |

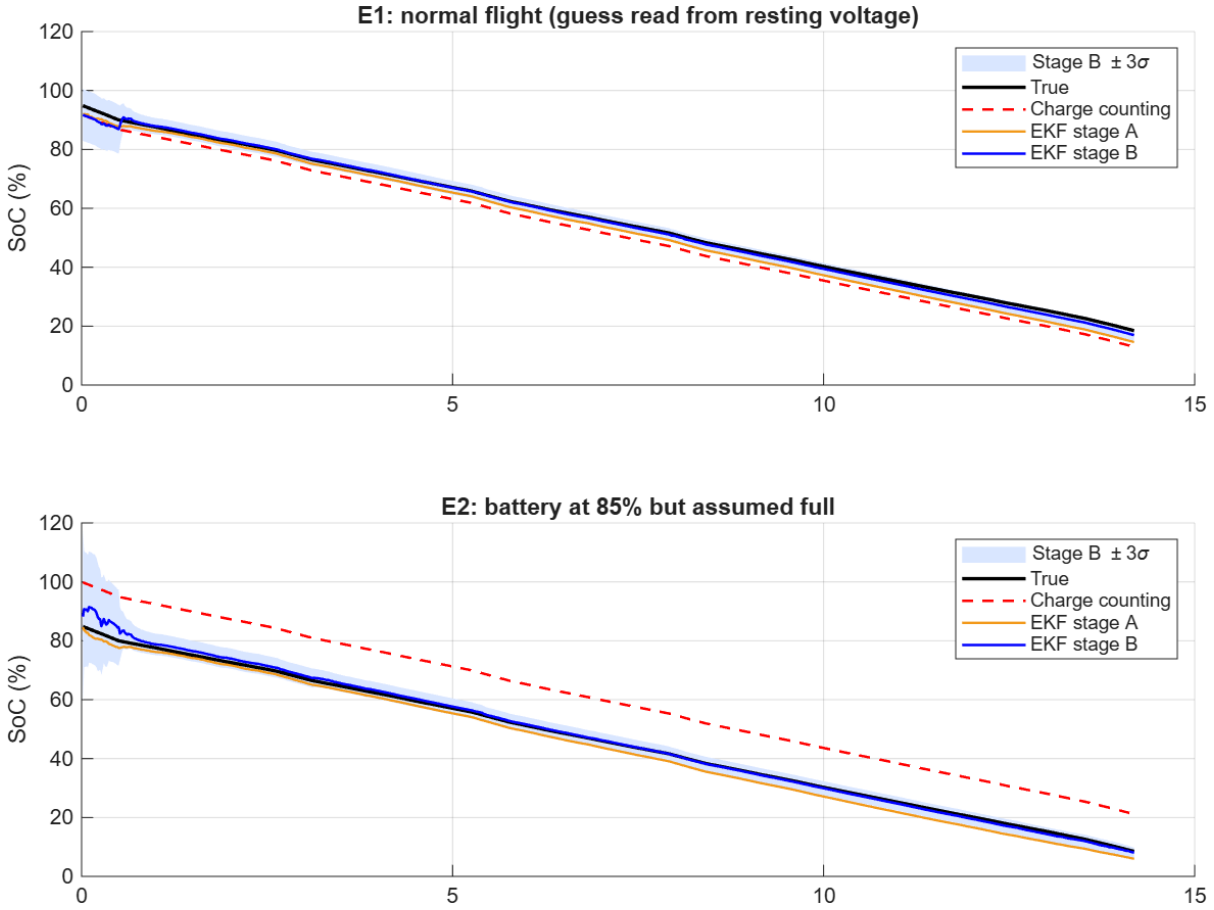



Figure 5.4: SoC estimates in E1 (top) and E2 (bottom): truth (black), charge counting (red dashed), EKF stage A (orange) and stage B (blue) with its $\pm 3\sigma$ band. Notice that stage B stays on the true SoC in both flights, while charge counting keeps its starting error, and in E2 believes the battery holds about 21% at landing when it really holds about 9%.

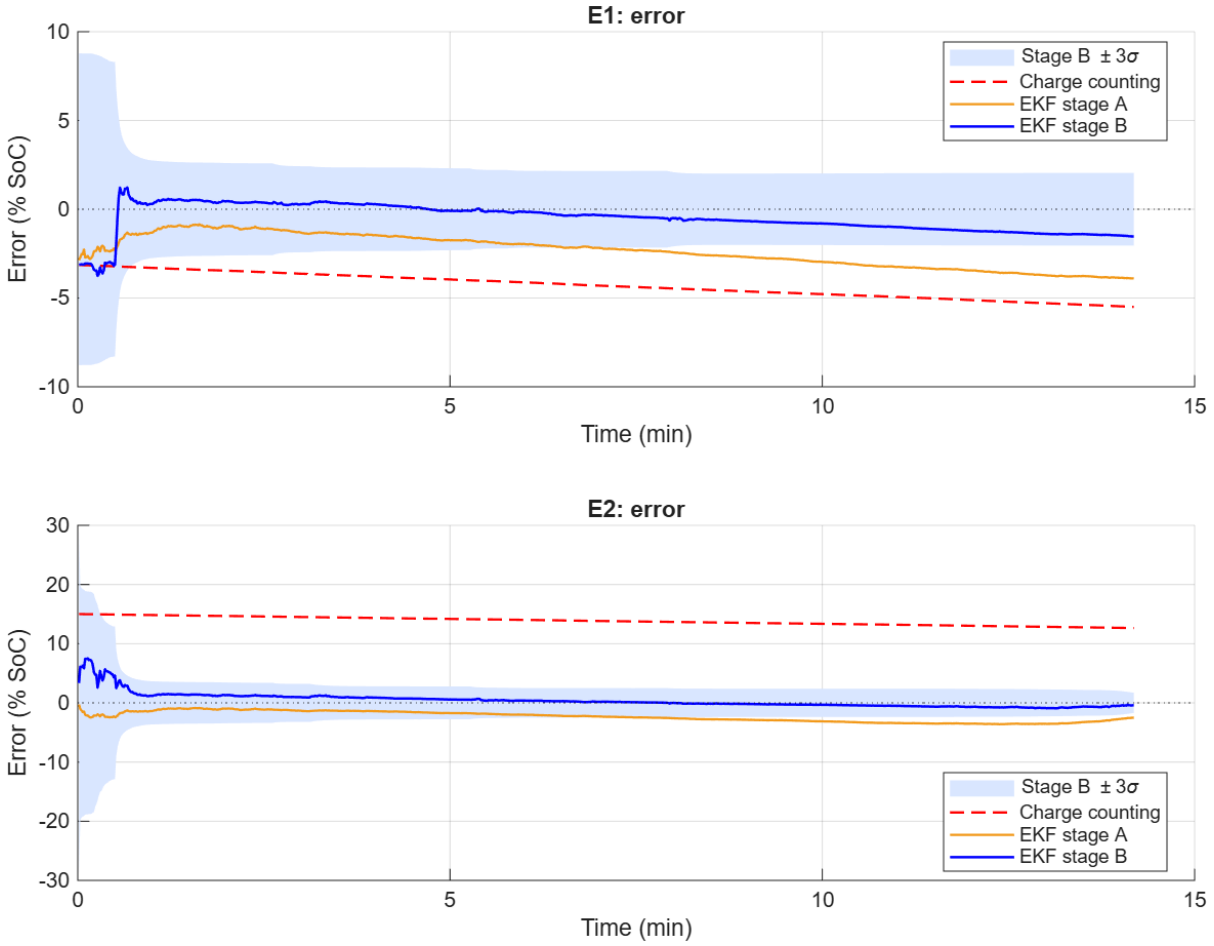



Figure 5.5: SoC error (estimate minus truth) in E1 (top) and E2 (bottom), with the stage-B $\pm 3\sigma$ band. Notice the jump in the stage-B error and the collapse of its band at about 0.5 min, when take-off ends; after that, the stage-B error stays inside its band, while stage A drifts steadily below it and charge counting drifts linearly.

### 5.2.1 E1: Nominal flight

**Starting guess.** The true battery started at 95%. The SoC read from the resting voltage before take-off was about 92%, roughly 3% low. Near full charge the OCV curve rises by about 10 mV per 1% SoC (Figure 2.1, 90–100%), so the −5 mV offset of the true curve accounts for about 0.5% of this, and the rest comes from the noise on the single voltage reading (10 mV, about 1% SoC per standard deviation). All three estimators started from this guess.

**Charge counting** never corrects the start error and adds the drift caused by the sensor bias. Its error grows linearly from about −3% to −5.50% at landing (Figure 5.5). The growth matches (2.5): a +0.5 A bias over about 14 minutes adds about −2.4%.

**Stage A** (no $R_0$ state) first moves towards the truth, but then drifts steadily low. It ends at −3.89% and spends most of the flight outside the stage-B band. Its own $\pm 3\sigma$ band contained the truth only 17% of the time: the filter is both biased and overconfident. This is the behaviour the design-stage investigation predicted (Section 3.4.5): the true $R_0$ is 10% higher than the filter assumes, and stage A can only explain the extra sag as a lower SoC.

Stage A's mean NIS of 0.54 looks healthy, and this is itself a lesson. Stage A explains the measured voltage well, but with the wrong SoC, so its innovations stay small. The innovation-based checks only see whether the voltage is explained, not whether the hidden state is right. Only a comparison with the truth (NEES, or the $\pm 3\sigma$ check above) reveals the bias, and that is possible only in simulation.

**Stage B** ends at −1.53% with an RMSE of 0.97%, less than a quarter of the charge-counting RMSE and less than half that of stage A. Its $\pm 3\sigma$ band contained the truth 100% of the time.

**The take-off current step.** The most instructive moment of the flight is at about 0.5 min, when take-off ends and the current steps from 30 A to 15 A. At that moment three things happen together (Figures 5.5 and 5.6):

- the stage-B SoC error jumps from about −3% to about +1%;
- its $\pm 3\sigma$ band collapses from about ±8.5% to about ±2.5%;
- the $R_0$ estimate jumps from about 7.7–8.0 mΩ to about 8.9 mΩ.

During take-off the current is almost constant at 30 A. A voltage that is lower than expected can then be explained equally well by a lower SoC or by a higher $R_0$, so the two cannot be separated, and the filter keeps both uncertain. The measurement Jacobian (3.12) shows why the current step resolves this. The SoC entry, $d\,\mathrm{OCV}/d\,\mathrm{SoC}$, does not depend on the current, but the $R_0$ entry, $-(I_m - \hat{b})$, halves when the current halves. Only when the current changes does the voltage carry separate information about the two states. The current step provides this *excitation*: the instant voltage jump it causes has the size $R_0\,\Delta I$, which pins down $R_0$, and with $R_0$ known the remaining voltage level pins down the SoC. This is the observability argument of Section 3.4.5 seen in a real run, and it is why the stage-B error falls below 2% exactly 30 s into the flight, when take-off ends.

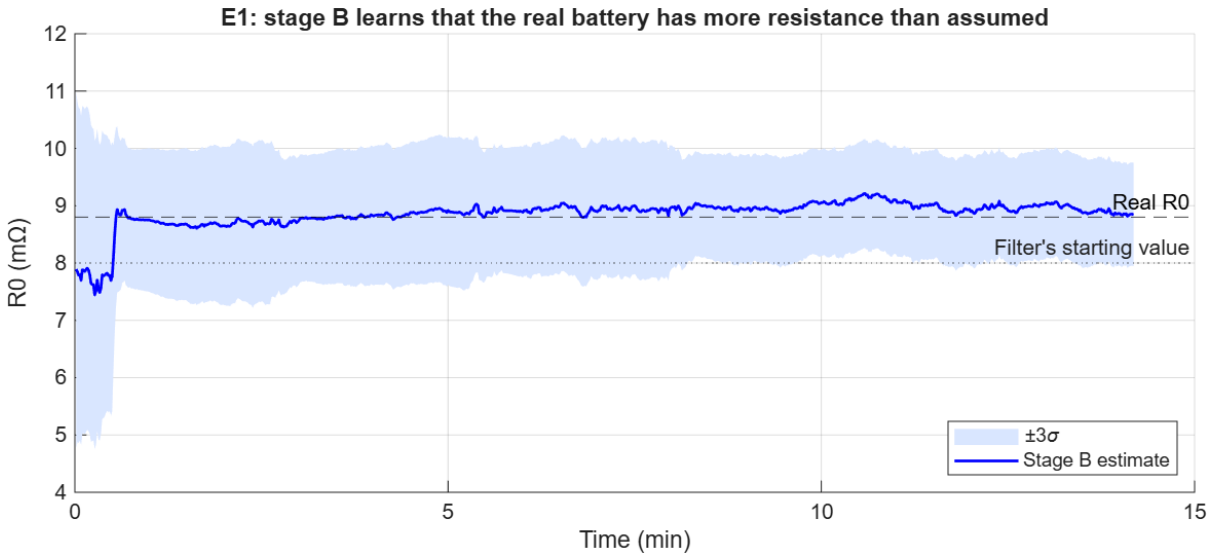



Figure 5.6: Stage-B estimate of $R_0$ in E1, with its $\pm 3\sigma$ band. Notice that the estimate barely moves during the constant-current take-off, then jumps to the true value (8.8 mΩ) when the current steps down at about 0.5 min, and that the band always contains the true value.

**Resistance estimate.** After the step the $R_0$ estimate settles between about 8.8 and 9.1 mΩ and ends the flight at 8.85 mΩ, against a true value of 8.8 mΩ and a starting value of 8.0 mΩ. Its $\pm 3\sigma$ band, about $\pm 1$ mΩ wide, always contains the true value.

**Is the filter honest?** Figure 5.7 compares the measured voltage with the voltage stage B expected before each update. The expected voltage follows every sag and step. The innovations look like white noise around zero and stay well inside $\pm 3\sqrt{S_k}$ after the first few seconds; the band is wide only at the very start because of the large initial uncertainty. The mean NIS of 0.51 is below 1, so the filter is slightly more cautious than necessary. This is the expected effect of the 10 mV model-error allowance, chosen on purpose to keep the bands honest (Section 3.4.5).

**Remaining error.** Stage B still drifts slowly low during the flight and ends at $-1.53\%$. The most likely cause is the slow RC pair ($R_2 = 1.5$ mΩ, $\tau_2 = 400$ s), which only the true battery has and which builds up over the flight (Section 3.2). The filter has no state for it, so part of the extra drop is read as a lower SoC. The error stays inside the $\pm 3\sigma$ band, so the filter's stated uncertainty remains honest; this is discussed further in Section 6.4.

### 5.2.2 E2: Wrong initial SoC

In E2 the true battery started at 85%, but all three estimators assumed a full battery (100%). The EKF was told that this guess was uncertain ($\sigma_{\mathrm{SoC},0} = 20\%$).

**Charge counting** keeps the +15% start error for the whole flight, reduced slowly by the bias drift, and ends +12.64% high. The consequence is serious. At landing the true SoC is only about 9% (8.6%), below the 10% reserve, while charge counting believes the battery still holds about 21%. A drone relying on charge counting would land nearly empty while believing it had a fifth of its battery left. On a longer route it would have

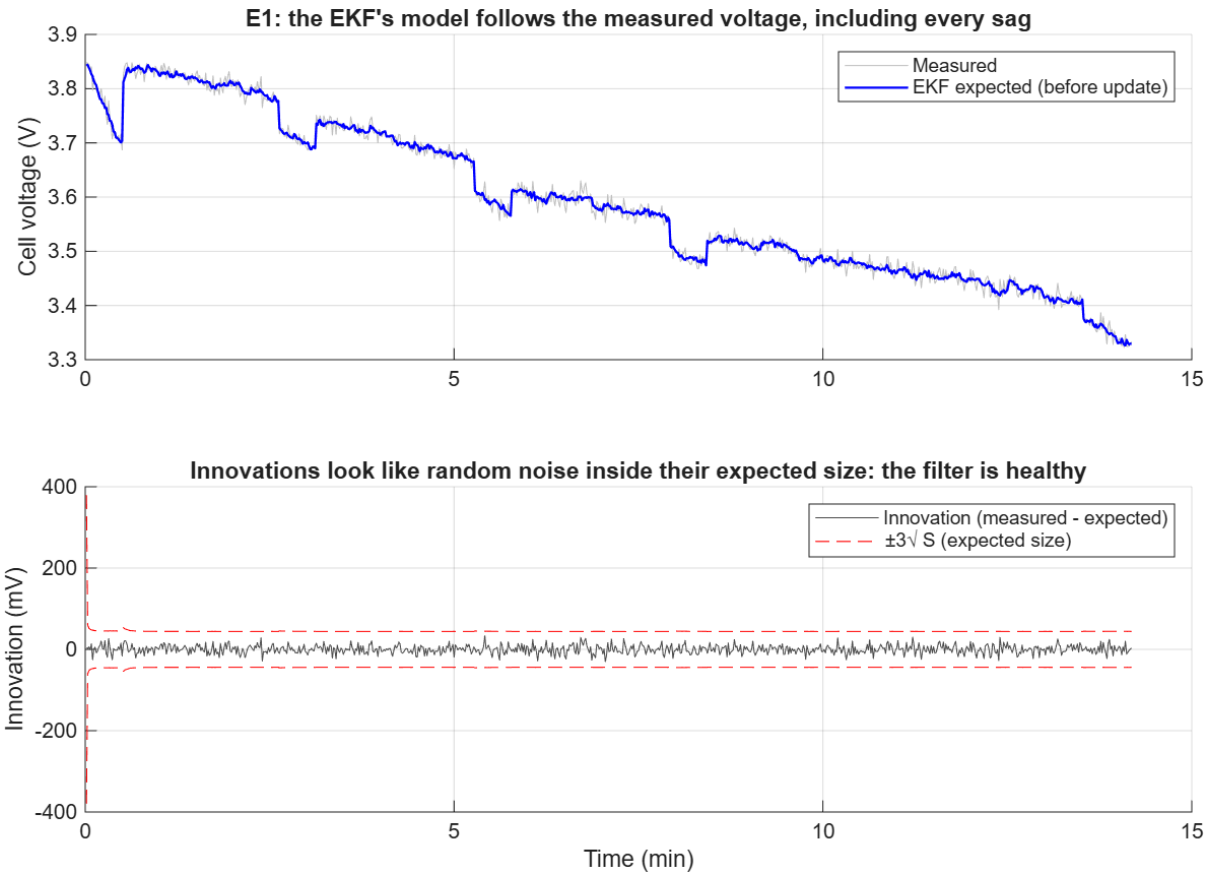



Figure 5.7: E1, stage B. Top: measured cell voltage (grey) and the voltage the EKF expected before each update (blue). Bottom: innovations with their expected size $\pm 3\sqrt{S}$. Notice that the model follows every sag and step, and that the innovations look like zero-mean noise well inside the band after the first seconds.

flown on past the point of no return. This is exactly the failure the risk-aware RTH rule is designed to prevent.

**Stage B** starts at 100% with a wide band. For the first 30 s, during take-off, it sits about 5–7% too high (maximum error 7.52%). Then, at the take-off current step, it converges: within about 1% of the truth by about 1 min, and with the error staying below 2% after 42 s. For the rest of the flight it stays close to the truth and ends at −0.38%, inside its band 100% of the time. The slow start has the same cause as in E1. While the current is constant, a voltage that is lower than a full battery would give can be explained partly by a higher $R_0$ instead of a lower SoC, so stage B cannot fully correct the SoC until the current step separates the two.

**Stage A** has no $R_0$ state, so it has no such ambiguity and corrects the start error within the first seconds. But it then drifts low for the same reason as in E1, ends at −2.49%, and contains the truth in its band only 18% of the time. Its "never" in Table 5.2 reflects this late drift, not a failure to recover from the wrong start.

### 5.2.3 Comparison with the design-stage checks

The MATLAB results show the same pattern as the design-stage checks in the Python port (Section 3.4.5). For stage B, the SoC RMSE was 0.97% in MATLAB against 1.04% in the port for E1, and 1.32% against 1.00% for E2. The two implementations use different random-number generators, so the same seed gives different noise, and single-run numbers differ slightly. The largest difference is for charge counting in E1 (4.36% in MATLAB against 1.60% in the port). This is because the MATLAB resting-voltage guess happened to be about 3% low, and charge counting never corrects a start error. The 20-flight averages of E8 (Section 5.2.10) remove this single-run luck.

### 5.2.4 Common settings for E3 to E8

Experiments E3–E8 were run with `main_day4_experiments.m` (Section 4.12) on the short mission. The starting SoC was read from 10 averaged resting readings, and single runs used seed 2. Because of the averaging, the random numbers differ from the first E1 and E2 runs, so the E1 and E2 scenarios inside E8 do not reproduce Table 5.2 exactly. Besides the errors, the tables give the mean NEES on the SoC ($e^2/P_{11}$, Section 3.4.3), which should be about 1 for an honest filter; much larger means overconfident.

### 5.2.5 E3: Bias observability (stage C)

In E3 the current sensor read 0.8 A too high instead of 0.5 A. Table 5.3 and Figure 5.8 show the results.

**Charge counting** drifts low at a steady rate and ends at −4.83%. This matches (2.5): 0.8 A for about 14 minutes is about 0.19 Ah, or 3.8% of 5 Ah, on top of a start error of about −1%.

Table 5.3: Single runs of E3 (bias 0.8 A) and E4 (aged battery), seed 2. NEES is the mean NEES on the SoC (about 1 for an honest filter).

| | Estimator | RMSE | Final error | Inside $\pm 3\sigma$ | Mean NIS | Mean NEES |
|---|---|---|---|---|---|---|
| E3 | Charge counting | 3.13% | −4.83% | – | – | – |
| | EKF stage B | 1.28% | −2.50% | 88% | 0.51 | 3.10 |
| | EKF stage C | 1.58% | −1.88% | 100% | 0.51 | 0.64 |
| E4 | Charge counting | 5.72% | +10.20% | – | – | – |
| | EKF stage B | 4.64% | +8.47% | 28% | 0.56 | 44.81 |
| | EKF stage C | 0.80% | +0.17% | 100% | 0.51 | 0.22 |

**Both EKF stages** do much better. Stage B, which does not model the bias, is slightly more accurate (RMSE 1.28%), because the voltage keeps correcting the drift the bias causes. Its band, however, contained the truth only 88% of the time, and its mean NEES of 3.10 shows it is overconfident: it does not know about the extra error. Stage C ended closer to the truth (−1.88%) and was honest (100% inside, NEES 0.64).

**The bias itself is only partly learned.** Stage C's bias estimate rose to about +0.5 A soon after take-off and stayed there for most of the flight, ending at +0.33 A against a true +0.80 A (Figure 5.8, bottom). Its $\pm 3\sigma$ band stayed wide, about $\pm 1.8$ A. The filter knows that it has not pinned the bias down, which is the honest answer: within one flight the bias is confounded with $R_0$ and $C_n$ (Equation (3.16) and Section 3.4.5).

### 5.2.6 E4: Capacity observability (stage C)

In E4 the true battery had aged to 85% of its rated capacity (4.25 Ah instead of 5.00 Ah) and started full, so that it could still complete the short mission. All estimators assumed 5.00 Ah at the start.

**Charge counting** ended +10.20% too high. At landing the true SoC was 9.9%, below the 10% reserve, while charge counting believed 20.1%. Most of this error comes from the capacity: the charge actually drawn, about 3.8 Ah, is 90% of 4.25 Ah but only about 77% of 5 Ah.

**Stage B failed, in the dangerous direction.** Its error grew almost parallel to that of charge counting (Figure 5.9, top), ending at +8.47%. Its band contained the truth only 28% of the time, and its mean NEES was 44.81, far above 1. Stage B was therefore confidently wrong, and wrong on the optimistic side: a drone using it would believe it had more charge than it had and would turn back too late. This is the failure predicted by the design check (Section 3.4.5). The voltage falls faster than stage B expects, and stage B blames the shortfall on its resistance state instead of lowering its SoC.

**Stage C stayed accurate and honest**: RMSE 0.80%, final error +0.17%, 100% inside its band, mean NEES 0.22. Its capacity estimate fell from 5.0 Ah to about 4.35–4.45 Ah within about 5 minutes and ended at 4.45 Ah, against the true 4.25 Ah (Figure 5.9, bottom). Its bias estimate ended at −0.42 A, against a true +0.50 A: the wrong sign. The two errors compensate each other. With a measured cruise current of about $15 + 0.5$ A,

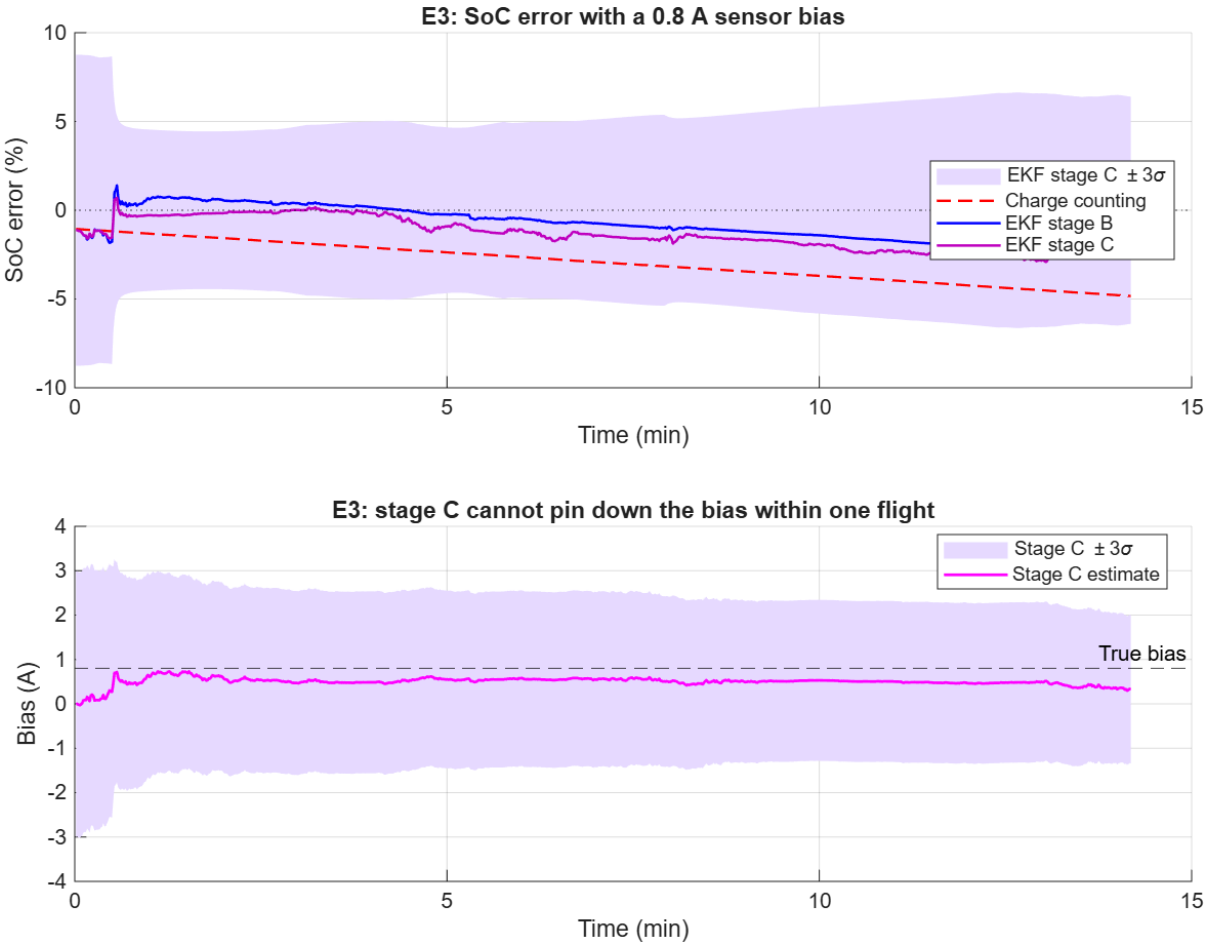



Figure 5.8: E3, sensor bias 0.8 A. Top: SoC error of charge counting, stage B and stage C, with the stage-C $\pm 3\sigma$ band. Bottom: stage-C bias estimate with its band. Notice that charge counting drifts steadily while both EKF stages stay within about 3%, and that the bias estimate settles near 0.5 A, below the true 0.8 A, with a band that stays wide: the bias is only partly learned in one flight.

the filter's discharge rate (3.17) is approximately $(15 + 0.5 + 0.42)/4.45 \approx 3.58$ per hour, against the true $15/4.25 \approx 3.53$ per hour. The rate, which is what the voltage can check, is right to within about 1.5%, although its two parts are individually wrong. This confirms the observability argument of Section 3.4.5.

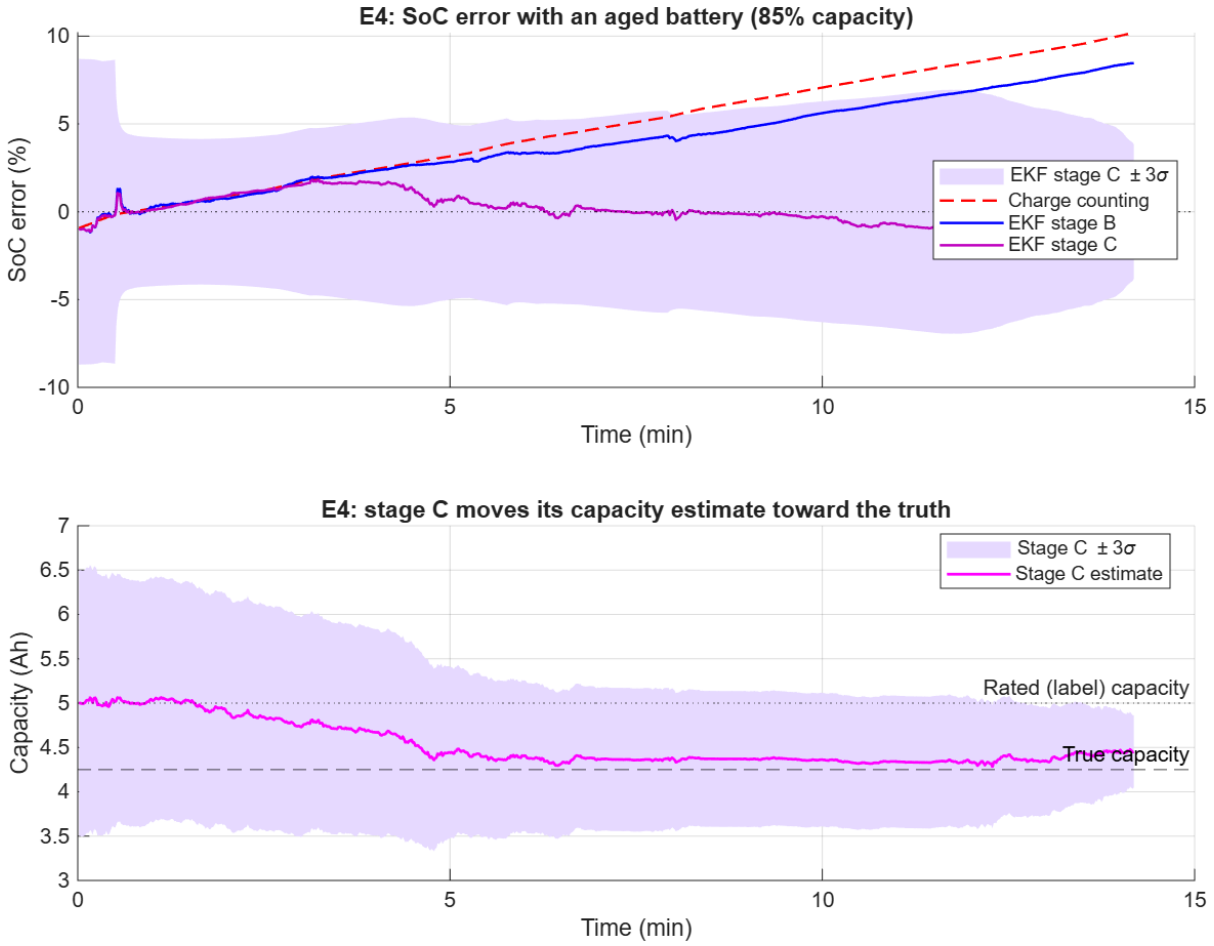



Figure 5.9: E4, battery aged to 85% capacity. Top: SoC error of charge counting, stage B and stage C, with the stage-C $\pm 3\sigma$ band. Bottom: stage-C capacity estimate with its band. Notice that stage B's error grows almost as fast as that of charge counting and leaves the band, both overestimating the SoC, while stage C stays near zero as its capacity estimate falls from 5.0 towards the true 4.25 Ah within about 5 minutes.

### 5.2.7 E5: Tuning Q and R

E5 reran stage B on the E1 scenario with $Q$ or $R$ scaled by 0.01 or 100 (Table 5.4, Figure 5.10).

- $Q\times100$**:** the filter distrusts its model, so it follows the voltage noise. Its error is jumpy and it is less accurate (RMSE 2.35%). The band is honest but wider than needed (NIS 0.32).
- $Q\times0.01$**:** the filter trusts its model more. In this run it was slightly more accurate (RMSE 0.67%), but its band held the truth only 94% of the time: it has less margin for the model errors it does not know about.

Table 5.4: E5: stage B on E1 with different tunings (seed 2).

| Tuning | RMSE | Final error | Inside $\pm 3\sigma$ | Mean NIS | Mean NEES |
|---|---|---|---|---|---|
| $Q\times 1$, $R\times 1$ (chosen) | 1.06% | −1.92% | 100% | 0.51 | 2.12 |
| $Q\times 0.01$ | 0.67% | −1.27% | 94% | 0.54 | 2.41 |
| $Q\times 100$ | 2.35% | −3.92% | 100% | 0.32 | 0.66 |
| $R\times 0.01$ | 2.77% | −4.19% | 30% | 32.34 | 53.55 |
| $R\times 100$ | 1.69% | −2.56% | 100% | 0.01 | 0.52 |

- $R\times 0.01$**:** the filter believes every wiggle of the voltage. During the first 30 s, before $R_0$ has been learned at the take-off current step, its SoC error swung down to about −13% (Figure 5.10). It was badly overconfident: 30% inside its band, mean NEES 53.55. Its mean NIS of 32.34, far above 1, flags the mistuning clearly.
- $R\times 100$**:** the filter barely uses the voltage. It is smooth but slower to correct (RMSE 1.69%), and its band is far too wide (NIS 0.01, NEES 0.52). With the RTH rule, such a band would make the drone turn back needlessly early.

The chosen tuning was kept. $Q\times 0.01$ was slightly more accurate in this single run, but it gave up honesty, and the RTH rule relies on $\sigma_{\mathrm{SoC}}$. E5 also shows what NIS can and cannot detect. It flags a wrongly set $R$ clearly ($32 \gg 1$), because that makes the innovations larger than the filter expects. It cannot flag a biased model: in E1 stage A had a normal-looking NIS but a biased SoC (Section 5.2.1).

### 5.2.8 E6: Model-mismatch stress test

In E6 the true battery's slow RC pair was set to the original, strong value ($R_2 = 4\,\mathrm{m\Omega}$, $C_2 = 10^5\,\mathrm{F}$, $\tau_2 = 400\,\mathrm{s}$), about 2.7 times larger in resistance than the default. Table 5.5 and Figure 5.11 show the results.

Table 5.5: E6: strong model mismatch ($R_2 = 4\,\mathrm{m\Omega}$), seed 2.

| Estimator | RMSE | Final error | Inside $\pm 3\sigma$ | Mean NIS | Mean NEES |
|---|---|---|---|---|---|
| Charge counting | 2.33% | −3.41% | – | – | – |
| EKF stage A | 5.11% | −7.91% | 2% | 0.71 | 406 |
| EKF stage B | 2.48% | −3.75% | 44% | 0.53 | 11.8 |
| EKF stage C | 4.60% | −4.99% | 75% | 0.52 | 5.56 |

All three filters degraded. The large unmodelled slow drop pulls the voltage down over the flight, and each filter reads part of it as a lower SoC. Stage A ended at −7.91% with a NEES of 406. Stage C drifted to about −8% around 12 minutes, leaving its own band, and partly recovered to −4.99% at landing. Stage B was about as accurate as charge counting, and stages A and C were worse. Charge counting is not affected at all, because the slow RC pair changes only the voltage, not the charge drawn. None of the filters was consistent: the best band, stage C's, held the truth 75% of the time. This is the honest limit of the method: when the filter's model is too wrong, the voltage feedback misleads the EKF, and it becomes biased and overconfident. Section 6.4 discusses remedies.

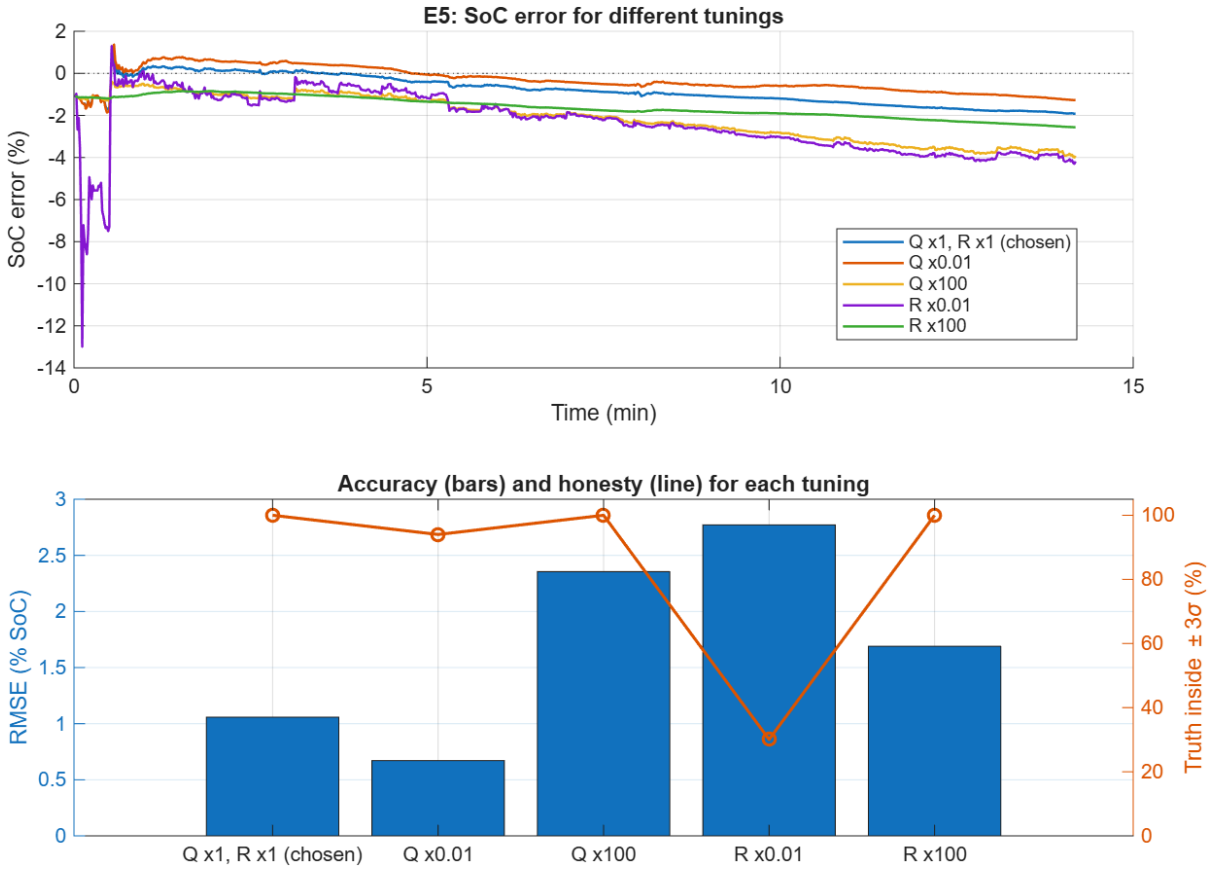



Figure 5.10: E5, stage B with five tunings. Top: SoC error over the flight. Bottom: RMSE (bars) and share of time the truth was inside the band (line). Notice the $-13\%$ excursion of $R{\times}0.01$ during take-off and its low band coverage, and that $Q{\times}0.01$ is the most accurate but the only other tuning below 100% coverage.

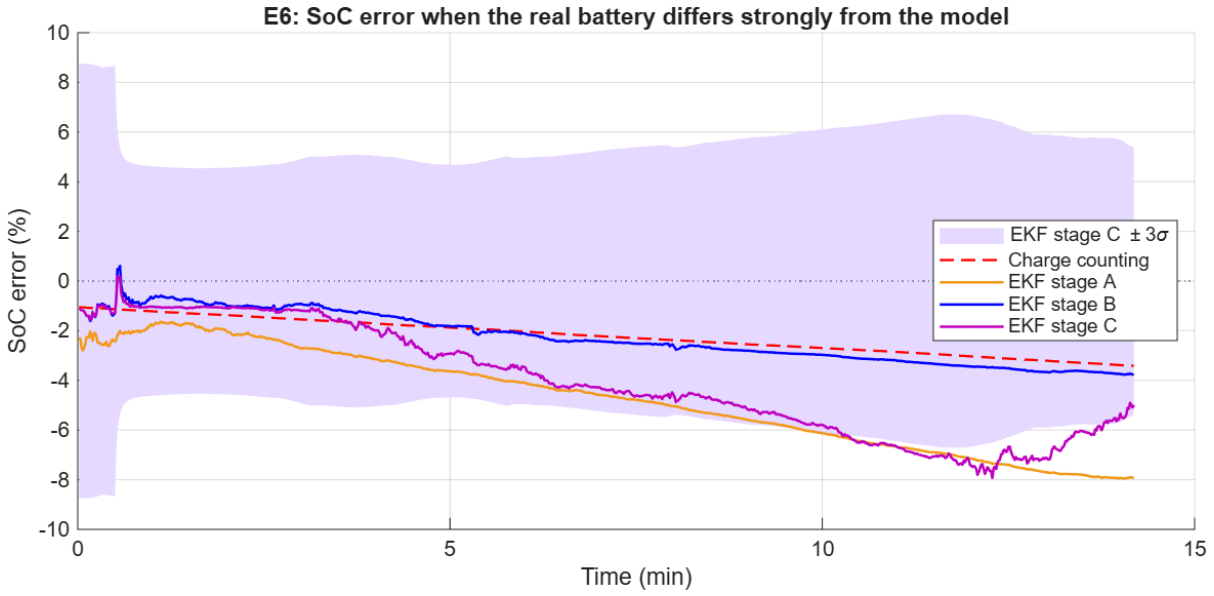



Figure 5.11: E6, strong model mismatch. SoC error of charge counting and stages A, B and C, with the stage-C $\pm 3\sigma$ band. Notice that all EKF stages drift low, that stage C leaves its band around 12 minutes, and that charge counting, which ignores the voltage, is as good as the best filter here.

### 5.2.9 E7: Flat region of the OCV curve

Figure 5.12 plots $\sigma_{\mathrm{SoC}}$ against the true SoC for stages B and C during the E1 flight, after the first minute (the flight runs from about 88% to 18%), together with the OCV slope.

**Stage C** behaves as theory predicts (Section 2.2). Its uncertainty rises from about 1.5% to about 2.2% as the OCV slope falls from about 8 to about 5 mV per 1% SoC. The figure contains a sharper test of the cause. Around 65–70% SoC the OCV slope has a local bump (to about 8.3 mV per 1%), and exactly there stage C's $\sigma_{\mathrm{SoC}}$ dips briefly, from about 1.67% to about 1.55%. The uncertainty tracks the slope, not merely the time since take-off: where the voltage says more about the SoC, the filter becomes more certain.

**Stage B** behaves differently: its uncertainty falls steadily from about 0.95% to about 0.70%. Stage B's only parameter is $R_0$, and the information it collects about $R_0$ at every current change keeps reducing the SoC uncertainty, which masks the flat-region effect. Both behaviours are reported because they show that the stated uncertainty depends on the whole filter structure, not on the OCV curve alone. For the RTH rule, stage C's behaviour is the desirable one: it becomes more cautious exactly where the voltage is a weaker clue.

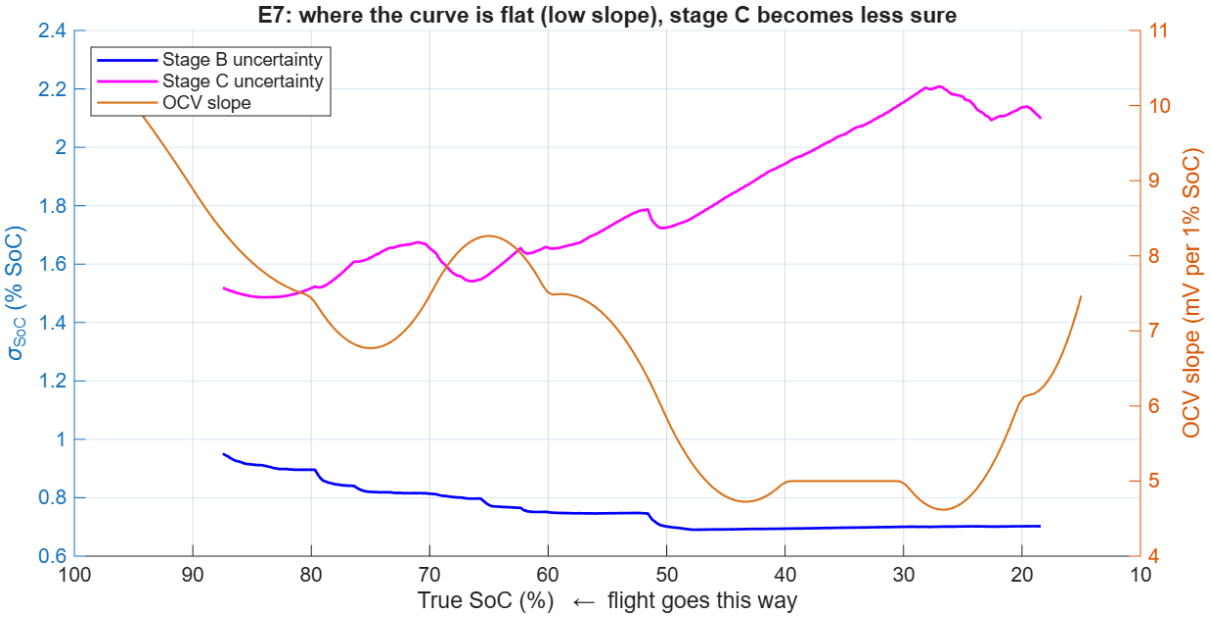



Figure 5.12: E7: SoC uncertainty $\sigma_{\mathrm{SoC}}$ of stages B and C against the true SoC (the flight runs from left to right), with the OCV slope on the right axis. Notice that stage C's uncertainty grows as the slope falls, and dips briefly around 65–70% SoC exactly where the slope has a local bump; stage B's uncertainty falls steadily instead.

### 5.2.10 E8: Comparison of filter stages

E8 flew each of five scenarios with 20 seeds and all three stages, 300 flights in total, in about 20 s of MATLAB time. Table 5.6 gives the mean RMSE and the band coverage, Table 5.7 the mean final error and the mean NEES, and Figure 5.13 shows the RMSE and coverage graphically.

Table 5.6: E8: mean SoC RMSE over 20 flights per scenario, with the share of time the truth was inside the filter's $\pm 3\sigma$ band in brackets.

| Scenario | Charge counting | Stage A | Stage B | Stage C |
|---|---|---|---|---|
| E1 Nominal | 1.83% | 2.46% (16%) | 1.03% (100%) | 1.41% (100%) |
| E2 Wrong start | 13.84% | 2.47% (17%) | 1.64% (100%) | 1.80% (100%) |
| E3 Bias 0.8 A | 2.64% | 2.31% (22%) | 1.27% (91%) | 1.24% (100%) |
| E4 Aged battery | 6.17% | 1.39% (46%) | 4.72% (27%) | 1.03% (100%) |
| E6 Strong mismatch | 1.83% | 5.10% (6%) | 2.37% (50%) | 4.16% (83%) |

Table 5.7: E8: mean final SoC error at landing, and mean NEES on the SoC in brackets (about 1 for an honest filter), over 20 flights per scenario (from `e8_summary.csv`).

| Scenario | Charge counting | Stage A | Stage B | Stage C |
|---|---|---|---|---|
| E1 Nominal | −2.87% | −3.79% (91.5) | −1.82% (1.95) | −1.87% (0.56) |
| E2 Wrong start | +12.65% | −2.39% (92.9) | −0.58% (1.32) | +0.08% (0.46) |
| E3 Bias 0.8 A | −4.29% | −3.78% (83.0) | −2.38% (2.91) | −1.67% (0.43) |
| E4 Aged battery | +10.69% | +1.91% (30.2) | +8.34% (45.6) | +0.16% (0.40) |
| E6 Strong mismatch | −2.87% | −7.82% (403.8) | −3.69% (10.8) | −4.69% (4.69) |

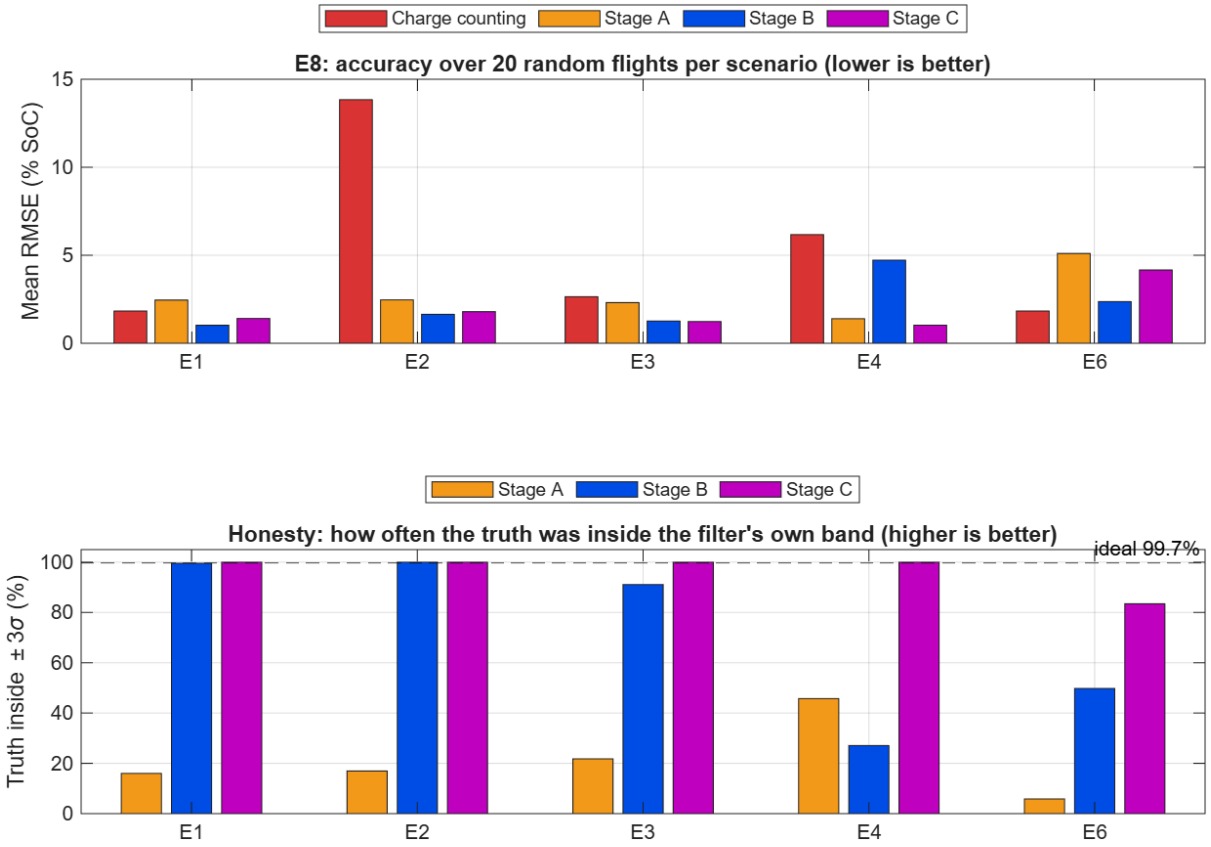



Figure 5.13: E8, 20 flights per scenario. Top: mean SoC RMSE of charge counting and stages A, B and C. Bottom: share of time the truth was inside each filter's $\pm 3\sigma$ band, with the ideal 99.7% marked. Notice that stage C is the only filter that reaches about 100% coverage in every scenario except the strong mismatch E6, and that stage B's accuracy and coverage both collapse on the aged battery (E4).

The 20-flight averages confirm the single runs and the early Python tests (Section 3.4.5).

- **Stage C is the only consistent filter in E1–E4.** Its band held the truth 100% of the time in all four scenarios, with mean RMSE between 1.03% and 1.80%. Its mean NEES (0.40–0.56) is below 1, so it is slightly cautious, in line with its mean NIS of about 0.49.
- **Stage B is the most accurate on a healthy battery** (1.03% in E1, 1.64% in E2), but it fails on the aged battery: 4.72% RMSE, 27% coverage, a mean final error of +8.34% (optimistic) and a mean NEES of 45.6. It is also slightly overconfident with the larger bias (91% coverage in E3).
- **Stage A is never consistent** (6–46% coverage, NEES 30–404), although its mean NIS stayed around 0.5–1.1. In E4 its accuracy looks good (1.39%), but this is probably a coincidence of two errors cancelling: its resistance error pulls the SoC low (as in E1, final −3.79%), while the aged capacity pushes it high.
- **Charge counting** is badly affected by a wrong start (13.84%) and an aged battery (6.17%, optimistic). In E6 it gives exactly the same result as in E1, because the slow RC pair affects only the voltage.
- **Under strong mismatch (E6) no filter is consistent** (at most 83% coverage), and the most accurate filter there (stage B, 2.37%) does not beat charge counting (1.83%).

**Conclusion for the decision filter.** Stage C is the only filter whose uncertainty stayed honest in all of E1–E4, with an RMSE of 1.0–1.8%. Stage B is the most accurate on a healthy battery, but on an aged battery it becomes confidently optimistic, which is the dangerous failure for an RTH decision. The MATLAB results therefore confirm the design choice of Section 3.4.5: stage C provides the SoC estimate and $\sigma_{\mathrm{SoC}}$ for the RTH rule. The strong-mismatch case E6 remains a stated limitation (Section 6.8).

## 5.3 Return-to-home decisions on a single flight

The demonstration mission (Section 3.6) was flown on the long route with each of the four policies under the same difficult conditions: start at 95%, a battery aged to 85% of its capacity, a current sensor reading 0.5 A low ($b = -0.5$ A), a 10% headwind, and noise seed 7. Table 5.8 summarises the outcomes, and Figures 5.14 and 5.15 show how they came about.

Table 5.8: Demonstration mission: outcome under each policy.

| Policy | Turn-back | Deliveries | True SoC at landing | Outcome |
|---|---|---|---|---|
| A. Voltage threshold (3.5 V) | 6.4 min | 1 | 11.7% | Safe |
| B. Charge counting | 8.2 min | 2 | 0.0% | Battery died |
| C. EKF estimate only | 6.4 min | 1 | 11.9% | Safe |
| D. EKF risk-aware (95%) | 5.9 min | 1 | 17.8% | Safe |

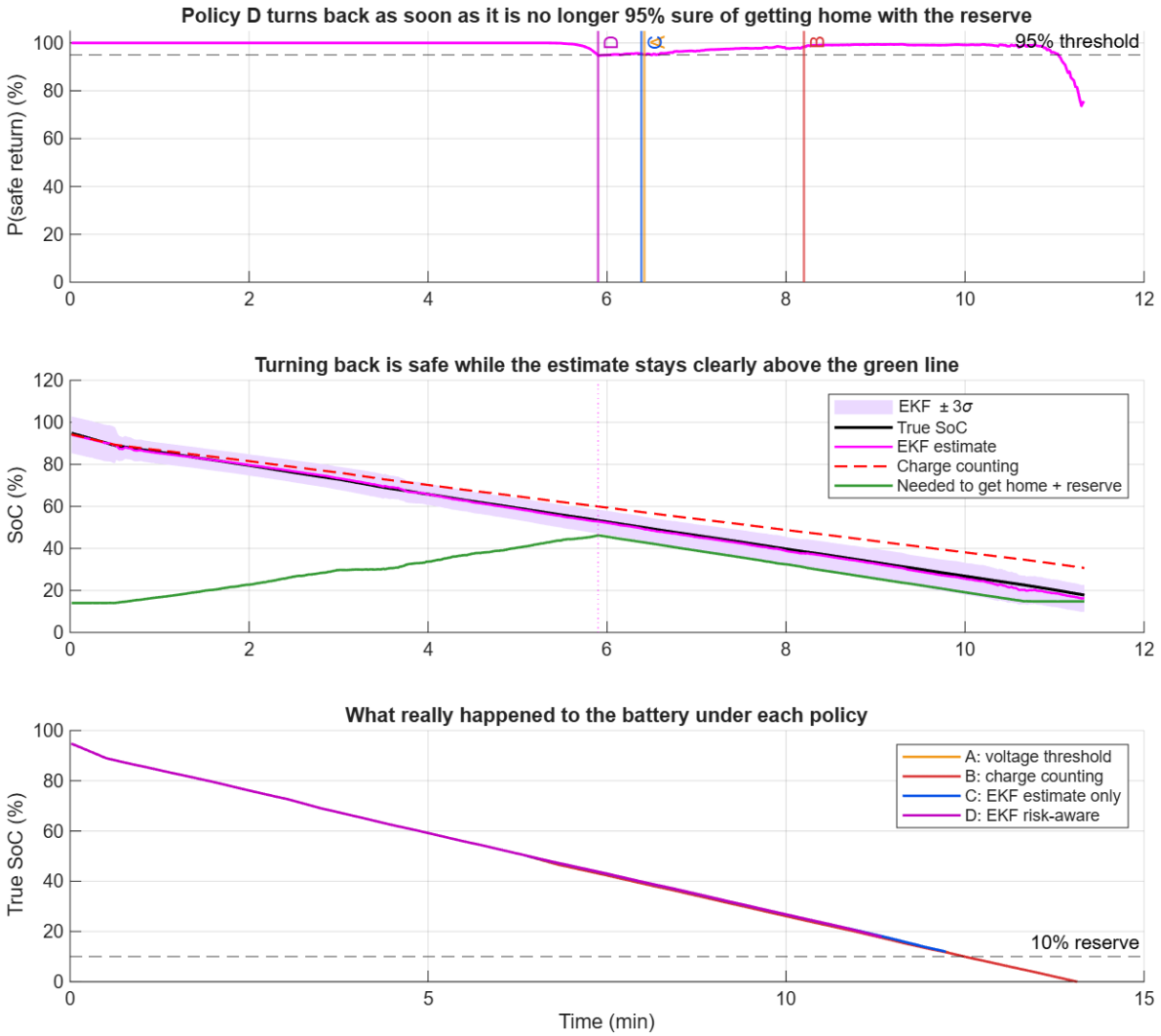



Figure 5.14: Demonstration mission. Top: $P_{\text{safe}}$ during policy D's flight, with the turn-back time of each policy marked. Middle: true SoC, stage-C estimate with its $\pm 3\sigma$ band, charge counting, and the SoC needed to get home plus the reserve (green). Bottom: true SoC under each policy. Notice that D turns back when the lower edge of the EKF band approaches the green line, that charge counting runs well above the truth, and that only B's battery falls through the reserve to zero.

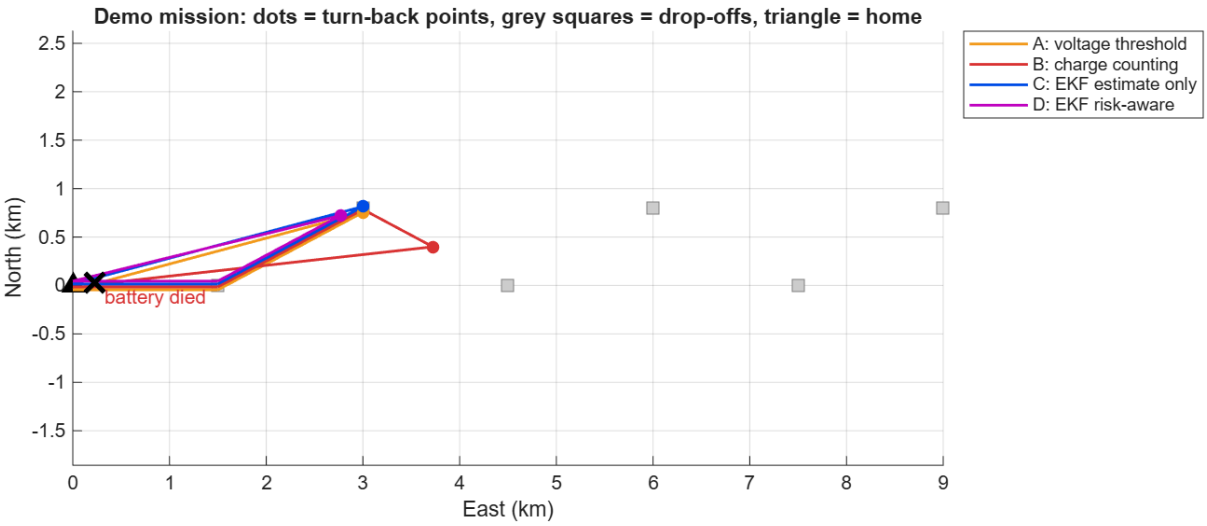



Figure 5.15: Demonstration mission: flight paths and turn-back points (dots) of the four policies; grey squares are drop-offs and the triangle is home. Notice that A, C and D turn back near drop-off 2, while B carries on towards drop-off 3 and its battery dies just short of home.

**The estimators.** In the middle panel of Figure 5.14, the stage-C estimate stays close to the true SoC, inside its band, for the whole flight. Charge counting runs about 7–12% above the truth, and the gap grows. Both conditions of this mission make it optimistic: the aged battery empties faster than the rated 5 Ah suggests, and the low-reading sensor hides part of the current drawn. The SoC needed to get home plus the reserve (green) rises as the drone flies away from home and peaks at about 46% when policy D turns back.

**The decisions.**

- **D** kept $P_{\text{safe}}$ near 100% until about 5.8 min. It then fell to 95%, and D turned back at 5.9 min, on the way to drop-off 2, when the estimate minus its uncertainty met the green line. It landed with 17.8%, that is 7.8 points above the reserve.
- **C** and **A** both turned back 0.5 min later, at 6.4 min, while hovering at drop-off 2 (Figure 5.15). Because a drop counts only once its hover is finished, this drop does not count, and all three safe policies completed one delivery. C, which aims exactly at the reserve, landed at 11.9%; A landed at 11.7%. Both kept the reserve, but only just.
- **B** believed the optimistic charge counter. It completed a second delivery, turned back at 8.2 min on its way to drop-off 3 (about 3.7 km east), and its battery died about 200 m short of home. The reserve that the counter promised did not exist.

At the very end of policy D's flight, $P_{\text{safe}}$ drops to about 75% (Figure 5.14, top). This is an artefact: during the final landing the formula (3.18) still charges the full 40 s landing cost, although part of the landing is already done. Decisions are made only in cruise and hover, so it has no effect.

## 5.4 Monte Carlo study

The four policies flew the same 500 random missions (Table 3.7), with the same conditions and the same sensor noise; the study took about 95 s of MATLAB time. Table 5.9 and Figure 5.16 give the results. Deliveries count a drop only once its hover is finished.

Table 5.9: Monte Carlo results over 500 missions. Reserve broken: died, or landed below the 10% reserve. Unused battery: mean true SoC at landing minus the reserve, over the missions that landed (negative means below the reserve on average).

| Policy | Died | Reserve broken | Unused battery | Deliveries | Died, aged only |
|---|---|---|---|---|---|
| A. Voltage (3.5 V) | 9.2% | 32.2% | 8.0% | 0.85 | 10.0% |
| B. Charge counting | 30.4% | 90.2% | −4.1% | 1.56 | 58.2% |
| C. EKF estimate | 0.0% | 38.0% | 0.6% | 1.32 | 0.0% |
| D. EKF risk-aware (95%) | 0.0% | 2.8% | 6.1% | 1.16 | 0.0% |

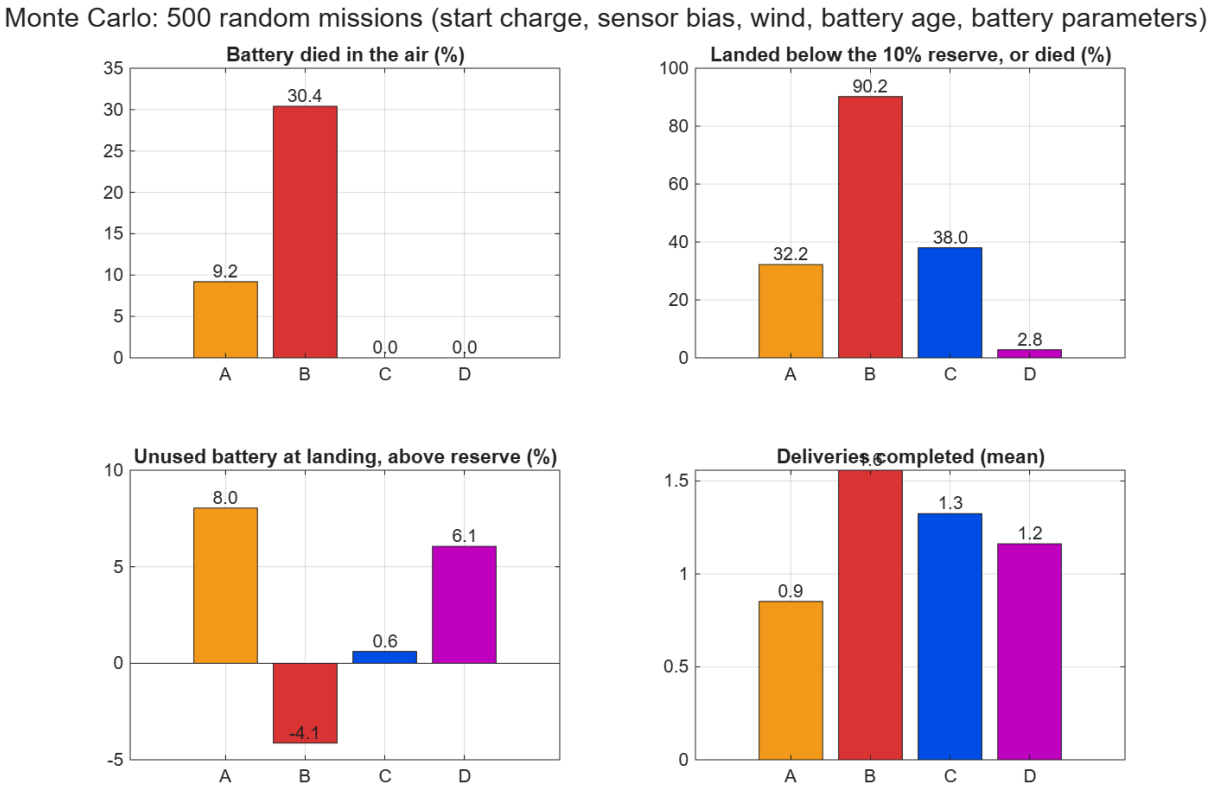



Figure 5.16: Monte Carlo study, 500 missions. Share of missions in which the battery died (top left) or the reserve was broken (top right), mean unused battery above the reserve (bottom left), and mean deliveries (bottom right), for each policy. Notice that only D combines zero deaths with a low breach rate, and that B's extra deliveries come with a 30% death rate.

The aged-battery column covers the 239 missions in which the true capacity was below 90% of rated.

**D, the risk-aware rule, is the only policy that is both safe and calibrated.** It had no deaths and broke the reserve in 2.8% of missions, inside its promise of at

most 5% (Section 3.5). The breach rate is lower than the promise, so the rule is slightly conservative. This matches the finding of E8 that stage C's uncertainty is slightly larger than necessary (mean NIS about 0.5, mean NEES below 1, Section 5.2.10). The price is 6.1% unused battery and 1.16 deliveries per mission.

**C uses the same filter but ignores its uncertainty.** It also had no deaths, because the stage-C estimate is accurate, but it broke the reserve in 38% of missions, and its landings sat on average only 0.6 points above the reserve. C turns back when the estimated margin reaches zero, so it aims exactly at the reserve: whenever the estimate or the trip cost is slightly optimistic, the drone lands slightly below it. With unbiased errors this would happen in about half of the missions. The difference between C and D is therefore not the estimator but what is done with its uncertainty.

**B, charge counting, is dangerous.** The battery died in 30.4% of all missions and in 58.2% of the aged-battery missions, and the reserve was broken in 90.2%. On average it landed below the reserve (−4.1%). Charge counting with the rated capacity is optimistic for an aged battery and for a sensor that reads low, and nothing in it notices. B also "completed" the most deliveries (1.56), but only because it flew on recklessly.

**A, the voltage threshold, is neither safe nor efficient at 3.5 V.** It died in 9.2% of missions and broke the reserve in 32.2%, while leaving on average 8.0% of the battery unused, more than D, and completing only 0.85 deliveries. A voltage threshold ignores how far from home the drone is: the same voltage can be plenty close to home and too little far away. Section 5.5 shows that changing the threshold only trades one problem for the other.

**Estimator accuracy over the Monte Carlo missions.** Over the 500 policy-D flights, which are longer than the flights of E1–E8 and include random battery parameters, the stage-C EKF had a mean SoC RMSE of 1.31%, against 3.35% for charge counting, and the true SoC was inside its $\pm 3\sigma$ band 100% of the time. This agrees with E8 (Table 5.6) and confirms that the uncertainty passed to the decision rule was honest.

## 5.5 Risk-threshold sweep

Policies D and A were rerun on the first 200 Monte Carlo missions with a range of thresholds (Table 5.10, Figure 5.17).

**D keeps its promise at every threshold.** No mission died at any threshold, and the breach rate was always within the promise $1 - p^*$: 9.5% against at most 20%, 4.0% against 10%, 2.0% against 5%, and none at 99% and 99.9%. At the three lower thresholds the breach rate was about half the promise, which again shows a slightly conservative uncertainty. With 200 missions the smallest non-zero breach rate that can be observed is 0.5%, so the 99.9% promise ($\leq 0.1\%$) is not contradicted, but it cannot be confirmed either.

**No voltage threshold works.** Low thresholds kill the drone (96% died at 3.40 V, 56.5% at 3.45 V). To reach D's level of safety, A needs 3.55 V (3.0% broken), but then it wastes 17.9% of the battery, about three times D's 6.1% at 95%, and completes only 0.56 deliveries, about half of D's 1.18. In Figure 5.17, for every voltage threshold that keeps

Table 5.10: Threshold sweep over the first 200 missions. For policy D the promised maximum breach rate, $1 - p^*$, is given in brackets.

| Policy | Threshold | Died | Reserve broken | Unused battery | Deliveries |
|---|---|---|---|---|---|
| D | $p^* = 80\%$ | 0% | 9.5% ($\leq 20\%$) | 3.6% | 1.27 |
| | $p^* = 90\%$ | 0% | 4.0% ($\leq 10\%$) | 5.0% | 1.22 |
| | $p^* = 95\%$ | 0% | 2.0% ($\leq 5\%$) | 6.1% | 1.18 |
| | $p^* = 99\%$ | 0% | 0.0% ($\leq 1\%$) | 8.2% | 1.15 |
| | $p^* = 99.9\%$ | 0% | 0.0% ($\leq 0.1\%$) | 10.4% | 1.11 |
| A | $V_{\text{thr}} = 3.40\,\text{V}$ | 96.0% | 100.0% | −7.9% | 1.88 |
| | $V_{\text{thr}} = 3.45\,\text{V}$ | 56.5% | 84.0% | −1.0% | 1.41 |
| | $V_{\text{thr}} = 3.50\,\text{V}$ | 11.0% | 35.0% | 7.5% | 0.89 |
| | $V_{\text{thr}} = 3.55\,\text{V}$ | 0.5% | 3.0% | 17.9% | 0.56 |
| | $V_{\text{thr}} = 3.60\,\text{V}$ | 0% | 0% | 28.7% | 0.36 |

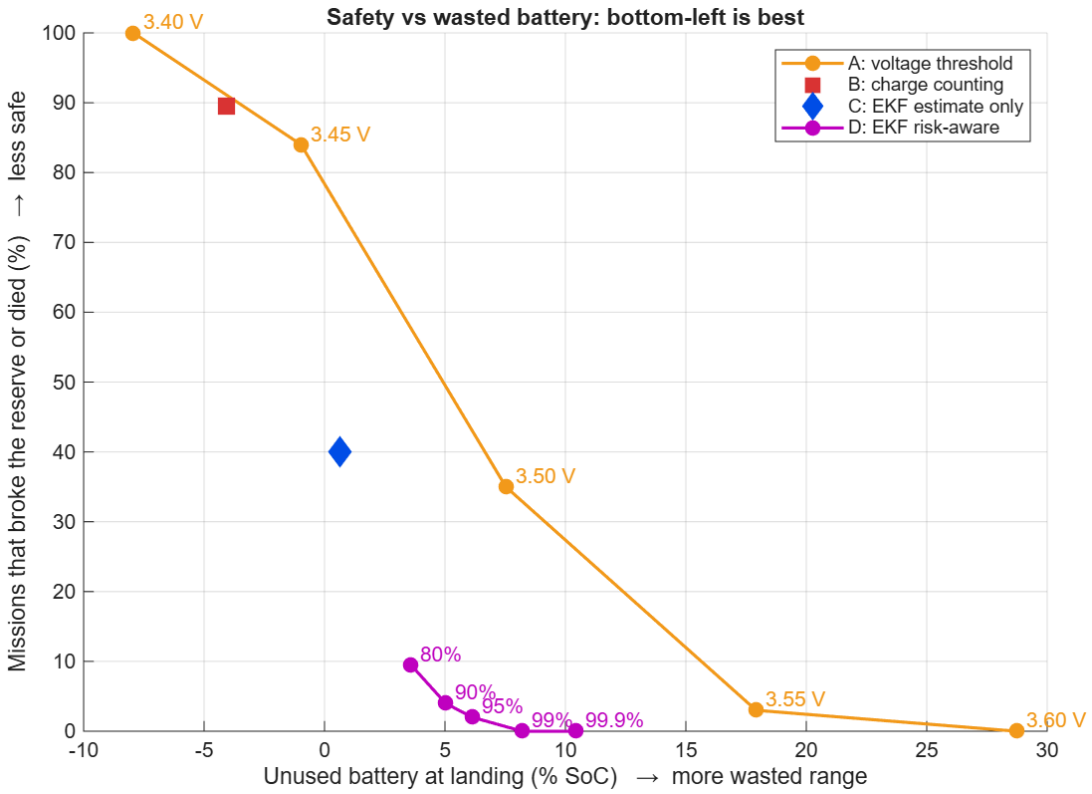



Figure 5.17: Safety against wasted battery over 200 missions: share of missions that broke the reserve or died, against the mean unused battery, for each threshold of policies A and D, with B and C as single points. Bottom-left is best. Notice that D's curve lies far below A's at every level of unused battery, and that B and C sit far above it.

the breach rate below that of policy C, there is a D threshold that is at least as safe and less wasteful. The only A points with less unused battery than D are those that land below the reserve on average. B and C, which have no threshold to tune, sit far above D's curve.

**The price of safety.** Moving D from 90% to 99% raised the unused battery from 5.0% to 8.2% and cut the deliveries from 1.22 to 1.15; moving from 99% to 99.9% cost a further 2.2 points of battery and 0.04 deliveries. Each extra "nine" therefore costs a few percent of the battery and a few hundredths of a delivery per mission. The curve lets an operator choose the point that suits the risk they can accept; 95% is a reasonable default, with a breach rate of about 2–3% and about 6% unused battery.

# Chapter 6

# Discussion

Chapter 5 showed three main things. An EKF that learns the cell resistance tracks the SoC far better than charge counting. The five-state filter is the only one whose uncertainty stays honest across healthy, biased and aged batteries. And a decision rule that uses that uncertainty keeps its safety promise while a rule that ignores it does not. This chapter explains why these results occur, what limits them, and what would improve them.

## 6.1 Why the EKF works for this problem

The EKF works here because the voltage gives it an independent check on the SoC at every second, and the Kalman gain decides how much to trust that check (Section 2.2). Charge counting has no such check. In E1 its error grew steadily from its start error through the sensor bias, to $-5.50\%$ at landing, while stage B stayed within 1.53% (RMSE 0.97% against 4.36%). In E2 charge counting kept almost all of its 15-point start error (RMSE 13.85%), while stage B removed it within 42 s (RMSE 1.32%). The covariance did what it should: it started wide when the start was uncertain, let the gain be large, and shrank as the voltage supplied information.

The tangent-line approximation was adequate. The OCV curve is smooth, and even in E2, with a 20% initial uncertainty and a 15-point start error, the filter converged without any sign of divergence. The design-stage ablation (Table 3.4) gives a sharper test: with a perfect model and no bias, the two-state filter reached an SoC RMSE of 0.12%. Linearisation errors were therefore small; the errors that remained in every later experiment came from model mismatch, not from the EKF's approximation. Finally, the noise in the simulation is Gaussian and white by construction, which matches the EKF's assumptions; real sensors may be less kind (Section 6.8).

## 6.2 Gain, OCV slope and uncertainty

The OCV slope sets how much the voltage can say about the SoC, because it is the first entry of $H$ (3.12). Where the curve is steep, a small SoC error gives a large voltage error,

the gain is large and the SoC uncertainty stays small; where it is flat, the opposite holds. E7 showed this directly for stage C: its $\sigma_{\mathrm{SoC}}$ rose from about 1.5% to about 2.2% as the slope fell from about 8 to about 5 mV per 1% SoC, and it dipped briefly exactly where the slope has a local bump near 65–70% SoC. Because the uncertainty follows the slope rather than simply the time since take-off, the effect is caused by the information in the voltage.

This matters for the decision rule. The middle of the SoC range, where the curve is flat, is where the turn-back decisions of the long missions are made, and there stage C reports a larger $\sigma_{\mathrm{SoC}}$. Policy D's safety buffer, $1.645\,\sigma_{\mathrm{total}}$, grows accordingly. The drone becomes more cautious exactly where the voltage is a weaker clue.

Stage B behaved differently: its uncertainty fell steadily from about 0.95% to about 0.70% through the flight. Every change in current gives it more information about $R_0$, and better knowledge of $R_0$ reduces the SoC uncertainty, which masks the flat-region effect. The stated uncertainty therefore depends on the structure of the whole filter, not on the OCV curve alone. The same slope effect also explains a detail of the experiments: near full charge the curve rises by about 10 mV per 1% SoC, so a single noisy resting reading (10 mV noise) gives a starting SoC uncertain by about 1%. The start about 3% low in the first E1 run was an unlucky case of this, and averaging 10 readings reduced it.

## 6.3 Observability of resistance, bias and capacity

Whether a state can be learned depends on whether the measurements contain information that separates it from the others. The resistance $R_0$ is a good example of a state that needs *excitation*. During the constant-current take-off, a voltage that is lower than expected can be explained equally well by a lower SoC or by a higher $R_0$. The SoC entry of $H$, $d\,\mathrm{OCV}/d\,\mathrm{SoC}$, does not depend on the current, while the $R_0$ entry, $-(I_m - \hat{b})$, does. When the current stepped from 30 A to 15 A at about 0.5 min in E1, the two became separable: within seconds the stage-B SoC error moved from about $-3\%$ to about $+1\%$, its $\pm 3\sigma$ band shrank from about $\pm 8.5\%$ to about $\pm 2.5\%$, and $R_0$ moved from about 8 mΩ to about 8.9 mΩ, close to the true 8.8 mΩ. The same mechanism delayed stage B's recovery in E2, where it stayed 5–7% too high until the step. A delivery drone gets this excitation naturally from take-off, hover and cruise; a flight at perfectly constant current would not.

The bias and the capacity are harder. Equation (3.16) shows that, at a steady current, a bias error and a resistance error change the voltage in the same way, and the capacity does not appear in the voltage at all. Both affect the SoC only through the discharge rate $(I_m - b)/C_n$ (3.17), so within one flight only that rate is observable. The experiments show both sides of this. Stage C did not pin down the bias in E3 (it ended at $+0.33$ A against $+0.80$ A, with a band of about $\pm 1.8$ A), and in E4 it ended with a wrong-signed bias ($-0.42$ A against $+0.50$ A) and a capacity of 4.45 Ah against 4.25 Ah. Yet in E4 its rate was within about 1.5% of the truth and its final SoC error was only $+0.17\%$.

The voltage feedback also explains why an unlearned bias did little harm. In the design-stage ablation a 0.5 A bias alone gave stage A an RMSE of only 0.43% (Table 3.4), and in E3 stage B, which has no bias state, reached 1.28% against 3.13% for charge

counting. The drift that a bias causes is slow, and the voltage corrects it continuously. An unlearned capacity error is different, because it can hide behind $R_0$, as E4 showed for stage B (Section 6.6).

## 6.4 Effect of model mismatch

A Kalman filter assumes that its errors are white noise. Model mismatch breaks this assumption, because it produces errors that have the same sign for minutes at a time. The design-stage investigation showed the consequence clearly: making the filter trust the voltage less by inflating $R$ did not reduce the error of the two-state filter (RMSE 2.49%), because the systematic resistance error still pointed it to the wrong SoC, only more slowly. Learning the dominant error, $R_0$, did reduce it. The 10 mV model-error allowance then serves a different purpose: once the large error has been removed by learning $R_0$, it widens the bands enough to cover the smaller errors that remain. The cost is a mean NIS of about 0.5, a slightly cautious filter; the gain is that stage C's band contained the truth 100% of the time in E1–E4.

Model mismatch is also hard to detect from the filter's own signals. Stage A had a mean NIS of about 0.5 in every healthy scenario, which looks healthy, while its mean NEES was between 83 and 93 in E1–E3 and reached 404 in E6 (Table 5.7). Stage A explained the voltage well, but with the wrong SoC. Innovation-based checks test whether the measurements are explained, not whether the hidden state is right. On a real drone, where the true SoC is unknown, this kind of failure could not be seen from the innovations alone, which is a strong argument for testing a filter in simulation against a deliberately different truth model, as this project does.

The remaining mismatch in the default true battery is the slow RC pair. It caused the gap between OCV and terminal voltage to widen slowly in the simulator verification (Section 5.1), and it is the most likely cause of stage B's slow drift to $-1.53\%$ in E1, which stayed inside its band. E6 showed the limit when this mismatch is made large ($R_2 = 4\,\mathrm{m\Omega}$). Every filter then drifted low; stage A ended at $-7.91\%$, stage C left its band around 12 minutes, and no filter's band held the truth more than 83% of the time over 20 flights. Charge counting, which ignores the voltage and is not affected by the slow RC pair, was as accurate as the best filter. When the model is too wrong, the voltage feedback misleads the EKF. The honest remedies are a better model (a second RC state in the filter), a larger model-error allowance, or adaptive noise estimation that raises $R$ when the innovations show a trend.

## 6.5 Tuning

E5 showed that the noise settings trade accuracy against honesty, and that only one kind of mistuning is visible from the filter's own signals. With $R\times0.01$ the filter believed every wiggle of the voltage, swung to about $-13\%$ during take-off before $R_0$ was learned, and was badly overconfident (30% coverage). Its mean NIS of 32.34 made the mistake obvious. With $R\times100$ the band became far too wide (NIS 0.01), which would make the

RTH rule turn back needlessly early. With $Q\times100$ the estimate became jumpy (RMSE 2.35%). The most instructive case was $Q\times0.01$: it was the most accurate in its single run (RMSE 0.67% against 1.06%), but its band held the truth only 94% of the time, because a filter that trusts its model more has less room for the model errors it does not know about. The chosen tuning was kept, accepting slightly lower accuracy for a band that stayed honest, because the decision rule turns $\sigma_{\mathrm{SoC}}$ directly into a safety margin. Together with the stage-A result of Section 6.4, E5 shows what NIS can and cannot do: it detects a wrongly set $R$, but not a biased model.

## 6.6 Choosing the filter for decisions

The usual way to choose between filters is accuracy: the one with the lowest error wins. By that measure stage B is the best filter in this project. On a healthy battery its mean SoC RMSE over 20 flights was 1.03%, against 1.41% for stage C (Table 5.6). The aged-battery experiment E4 shows why accuracy alone is the wrong test for a filter that feeds a safety decision. When the capacity is lower than assumed, the voltage falls faster than stage B expects. During steady cruise a lower voltage can be explained either by a lower SoC or by a higher resistance, and stage B explains it with its $R_0$ state. Its SoC estimate therefore stays too high: it ended on average 8.34% above the truth, its band contained the truth only 27% of the time, and its mean NEES was 45.6. It was not only wrong but confidently wrong, and on the optimistic side, which is the side that makes a drone turn back too late.

Stage C has more unknowns to share the same information, so it is a little less accurate on a healthy battery. In exchange it was the only filter whose band contained the truth 100% of the time in all four scenarios E1–E4. It achieves this although its individual parameter estimates are often wrong. In E4 it ended with a capacity of 4.45 Ah (true 4.25 Ah) and a bias of $-0.42$ A (true $+0.50$ A), yet its final SoC error was $+0.17$%. The reason is the observability structure of the model (Section 3.4.5): the bias and the capacity affect the SoC only through the discharge rate $(I_m - b)/C_n$, and the voltage checks only that rate. Stage C got the rate right to within about 1.5% and was honest about not knowing the two parts separately; in E3 its bias estimate kept a band of about $\pm1.8$ A.

The Monte Carlo study confirmed the choice in the setting that matters. Over 500 long missions with random battery ages, biases and parameters, stage C had a mean RMSE of 1.31% and a band coverage of 100%, and the decision rule built on it had no deaths. The lesson generalises beyond batteries: *the most accurate filter is not always the safest one to make decisions with.* A filter that feeds a decision should be judged by the honesty of its uncertainty across all the conditions it will meet, not only by its accuracy in the nominal case. The design also points to how stage C could become more accurate without losing honesty: calibrating the current-sensor bias at rest before take-off, when the true current is known to be zero, and estimating the capacity between flights from the charge drawn and the SoC read from the resting voltage before and after. Both would make $b$ and $C_n$ individually observable.

## 6.7 Value of uncertainty in the return-to-home decision

Policies C and D are the clearest experiment in the project, because they share everything except the use of the uncertainty. They use the same stage-C filter, the same trip cost and the same reserve. The only difference is that C turns back when the estimated margin reaches zero, while D turns back when the probability of keeping the reserve falls below 95%. Over 500 missions neither had a single death, which shows that the estimator itself was good. But C broke the reserve in 38% of missions, and D in 2.8%.

The reason is simple. C aims at the reserve. Its estimate and its trip cost are close to the truth, but never exact, so whenever they err slightly on the optimistic side the drone lands slightly below the reserve. If the errors were symmetric, this would happen in about half of the missions; 38% is of that order. D aims above the reserve by $1.645\,\sigma_{\text{total}}$, a buffer that is large when the filter is unsure and small when it is confident. In the demonstration mission this buffer cost half a minute of flight and gave 7.8 points of SoC margin at landing instead of C's 1.9.

D's breach rates also show that the probabilities mean what they say. At every threshold in the sweep the share of missions that broke the reserve stayed within the promised $1 - p^*$, and at 80–95% it was about half of the promise (Table 5.10). This is called *calibration*. It is a test of the whole chain: the EKF's covariance, the trip cost, the trip allowance and the Gaussian assumption. The slight conservatism agrees with what E8 showed about stage C (mean NIS about 0.5, mean NEES below 1), which comes from the deliberate 10 mV model-error allowance. In a safety system, a promise that is kept with some room to spare is preferable to one that is broken. An honest uncertainty is what turns a filter into something a decision rule can rely on, and it is why the stage-C choice of Section 6.6 matters.

The two baselines show what happens without a reliable estimate. Charge counting (B) has no uncertainty and no way of noticing that it is wrong. With an aged battery or a sensor that reads low it is optimistic, and it died in 30.4% of all missions and in 58.2% of those with an aged battery. Its higher delivery count (1.56) is the result of recklessness, not of better use of the battery: on average it landed below the reserve. The voltage threshold (A) fails differently. The voltage at which a drone must turn back depends on how far it is from home and on the state of the battery, so no fixed value can be right for every mission. In the sweep, the thresholds that keep A safe waste a great deal of battery: at 3.55 V it broke the reserve about as rarely as D at 95% (3.0% against 2.0%), but left 17.9% of the battery unused, about three times D's 6.1%, and completed about half as many deliveries (0.56 against 1.18). Across the whole sweep, D's trade-off curve lies below A's: for every voltage threshold that keeps the breach rate below that of C, a D threshold is at least as safe and less wasteful.

Finally, the threshold sweep turns the choice of $p^*$ into an explicit trade-off. Moving from 90% to 99% raised the unused battery from 5.0% to 8.2% and reduced the deliveries from 1.22 to 1.15 per mission; moving on to 99.9% cost a further 2.2 points of battery. An operator can read off the price of each extra "nine" and choose. The 95% used here is a reasonable default: no deaths, a breach rate of about 2–3%, and about 6% of the battery left unused.

## 6.8 Limitations

The results should be read with the following limitations in mind.

- **Simulation only.** All results come from simulation, where the true SoC is known and every error can be measured. The sensor noise is Gaussian and white by construction. Real flight logs have no true SoC, so the consistency checks that revealed stage A's bias (NEES and band coverage) would not be available.
- **A representative OCV curve.** The OCV table has the typical shape of an NMC cell but was not measured on a specific cell. A real application would need the curve of its own cells, measured at rest; errors in it act like the 5 mV offset tested here, but could be larger.
- **No temperature.** A cold battery has a higher resistance and less usable capacity. Learning $R_0$ would follow part of this, but the capacity change would face the same observability limits as ageing.
- **No hysteresis.** Lithium-ion cells show a small difference in resting voltage after charge and after discharge, which is ignored in both models.
- **Weakly observable bias and capacity.** Within one flight only their combined effect on the discharge rate is learned (E3, E4).
- **Strong model mismatch.** With a large unmodelled slow RC pair (E6) no filter was consistent, and the EKF was no better than charge counting.
- **Gaussian assumption in the decision rule.** $P_{\text{safe}}$ assumes Gaussian errors. Heavy-tailed events, such as a sudden gust or a motor fault, are not covered.
- **Simple trip model.** The cost of getting home assumes a straight path and a cruise current that stays at its learned value. Obstacles, no-fly zones or a change in wind on the way home are covered only by the 10% trip allowance, and the uncertainty of the drain rate itself is not added to $\sigma_{\text{total}}$.
- **Landing-phase artefact.** During the final landing the trip cost still includes the full 40 s landing, so $P_{\text{safe}}$ drops there (Section 5.3). Decisions are not made in that phase, so the results are not affected, but the displayed probability is misleading.
- **One mission geometry.** All return-to-home missions used the same long route, with the same pattern of take-off, cruise and hover. Other routes would give different current changes, and so different excitation for learning $R_0$.
- **Simplifications in the filter.** The correlation between the current noise in the process and in the measurement is ignored, and the current-noise term of $R$ uses the nominal $R_0$.

## 6.9 When a UKF or particle filter would be better

An unscented Kalman filter avoids the single tangent line by propagating a small set of sample points through the nonlinear functions, and a particle filter represents the uncertainty with many samples and can handle non-Gaussian errors [3]. Dibbern et al. found a UKF more accurate than an EKF for a multirotor UAV battery [10]. In this project, however, the evidence points to model mismatch, not linearisation, as the main source of error: with a perfect model the EKF error was 0.12%, and the errors in E1–E6 came from resistance, slow-RC and capacity differences. A UKF uses the same model and would not remove those errors. It would become more attractive with sharper nonlinearity, for example deep into the steep region near empty, with cell chemistries whose OCV curve has a very flat plateau, or with a much larger initial uncertainty. A particle filter would be justified mainly if the noise were strongly non-Gaussian or the uncertainty multimodal. A direct comparison on the same simulator would settle the question and is left as future work.

## 6.10 Future work

Several extensions follow directly from the results.

- **Bias calibration at rest.** Before take-off the true current is known to be zero, so the current sensor's bias can be measured directly. This would remove one of the confounded unknowns of stage C.
- **Capacity estimation between flights.** Comparing the charge drawn in a flight (ampere-hours) with the change in SoC read from the resting voltage before and after gives the capacity directly, without the single-flight observability problem.
- **A two-RC filter model** to remove the slow-RC mismatch that limited the filters in E1 and E6.
- **Adaptive noise estimation**, raising $R$ or $Q$ when the innovations show a trend, to keep the bands honest under a strong mismatch.
- **A UKF and particle-filter comparison** on the same simulator and missions.
- **Validation on real flight logs**, for example the CMU in-flight energy data of a delivery quadcopter [11] and the eVTOL battery dataset [12, 13], using consistency checks that do not need the true SoC.
- **A richer decision model**: temperature, varied routes, and the uncertainty of the drain rate added to $\sigma_{\text{total}}$.

# Chapter 7

# Conclusion

This project set out to let a delivery drone decide for itself when to fly home, using only a noisy, biased current sensor and a voltage sensor whose readings sag under load. It combined an Extended Kalman Filter, which estimates the battery's state of charge together with its uncertainty, with a return-to-home rule that turns that uncertainty into a safety margin. Both were tested in a MATLAB simulator whose battery deliberately differs from the filter's model.

## 7.1 Answers to the objectives

**A realistic simulator.** The simulator produced the expected battery behaviour: a voltage sag of 367 mV per cell at take-off, steps in the right direction at every change of flight phase, and a charge-counting drift with a biased sensor ($-2.3\%$ in 14.2 min) that matched the theoretical value of about 2.4%. Its true battery has a second, slow RC pair and parameters offset by 10%, so the filter was never tested against its own model.

**An EKF that estimates the SoC and its uncertainty.** A design-stage investigation found that a wrong internal resistance was the main source of SoC error, so the filter learns $R_0$ during flight. On the nominal flight (E1) this filter (stage B) reached an SoC RMSE of 0.97%, against 4.36% for charge counting and 2.51% for the filter without $R_0$, with the truth inside its $\pm 3\sigma$ band 100% of the time; its resistance estimate ended at 8.85 mΩ against a true 8.8 mΩ. From a wrong start (E2: true 85%, assumed 100%) it was within 2% after 42 s (RMSE 1.32%), while charge counting stayed wrong all flight (RMSE 13.85%). Adding the sensor bias and the capacity as states (stage C) could not separate those two within one flight, because only the discharge rate $(I_m - b)/C_n$ is observable, but it got that rate right. Over 20 flights per scenario, stage C was the only filter whose band held the truth 100% of the time in all of the nominal, wrong-start, biased-sensor and aged-battery scenarios, with mean RMSE between 1.03% and 1.80%. Stage B, the most accurate on a healthy battery, overestimated the SoC of an aged battery by 8.34% on average with only 27% coverage. Stage C was therefore chosen to drive the decision: the most accurate filter is not always the safest one to make decisions with.

**A risk-aware return-to-home rule.** The rule turns back when the probability of

landing with the 10% reserve falls below 95%, using the filter's SoC uncertainty and a trip cost based on the observable drain rate. In the demonstration mission it turned back half a minute earlier than the estimate-only rule and landed with 17.8% instead of 11.9%, while charge counting led the drone to a battery failure 200 m from home.

**Evaluation against baselines.** In 500 random missions with random start charge, sensor bias, wind, battery age and battery parameters, the risk-aware rule (D) had no battery failures and broke the reserve in 2.8% of missions, inside its promise of at most 5%, leaving 6.1% of the battery unused on average. The same filter used without its uncertainty (C) also had no failures, but broke the reserve in 38.0% of missions. Charge counting (B) let the battery die in 30.4% of missions, and in 58.2% of those with an aged battery. The voltage threshold (A) died in 9.2% and broke the reserve in 32.2%. Over these missions the stage-C filter had an SoC RMSE of 1.31% against 3.35% for charge counting, with 100% band coverage. Across a sweep of thresholds from 80% to 99.9%, the share of missions that broke the reserve stayed within the promised $1 - p^*$ every time, so the probabilities were calibrated and slightly conservative. To match the risk-aware rule's safety, the voltage threshold had to rise to 3.55 V, where it left 17.9% of the battery unused and completed about half as many deliveries.

Together, these results show that an EKF is effective for this application when its model includes the dominant error source, that its uncertainty is as valuable as its estimate, and that a decision rule which uses an honest uncertainty can keep an explicit safety promise at a modest cost in range.

## 7.2 Limitations and future work

The results come from simulation with a representative OCV curve, no temperature effects and one route, and the method reaches its limit when the true battery differs strongly from the filter's model (Section 6.8). The most useful next steps are to calibrate the current-sensor bias at rest before take-off and estimate the capacity between flights, which would make both individually observable; to add a second RC pair or adaptive noise to the filter for robustness to model mismatch; to compare the EKF with a UKF and a particle filter; and to validate the estimator on real flight logs such as the CMU delivery-drone and eVTOL datasets.

# References


[1] G. L. Plett, "Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs: Part 1. Background," *Journal of Power Sources*, vol. 134, no. 2, pp. 252–261, 2004. [Online]. Available: https://doi.org/10.1016/j.jpowsour.2004.02.031

[2] ——, "Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs: Part 3. State and parameter estimation," *Journal of Power Sources*, vol. 134, no. 2, pp. 277–292, 2004. [Online]. Available: https://doi.org/10.1016/j.jpowsour.2004.02.033

[3] D. Simon, *Optimal State Estimation: Kalman, $H_\infty$, and Nonlinear Approaches*. Hoboken, NJ: Wiley-Interscience, 2006.

[4] G. Welch and G. Bishop, "An introduction to the Kalman filter," Department of Computer Science, University of North Carolina at Chapel Hill, Tech. Rep. TR 95-041, 2004, updated 5 April 2004. [Online]. Available: https://techreports.cs.unc.edu/papers/95-041.pdf

[5] G. L. Plett, "Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs: Part 2. Modeling and identification," *Journal of Power Sources*, vol. 134, no. 2, pp. 262–276, 2004. [Online]. Available: https://doi.org/10.1016/j.jpowsour.2004.02.032

[6] ——, *Battery Management Systems, Volume II: Equivalent-Circuit Methods*. Artech House, 2015.

[7] E. Siampis, "Extended Kalman filter with battery estimation example," MATLAB Central File Exchange, version 1.0.0, 2022, accessed 19 Oct. 2026. [Online]. Available: https://www.mathworks.com/matlabcentral/fileexchange/105855-extended-kalman-filter-with-battery-estimation-example

[8] R. Schacht-Rodríguez, G. Ortiz-Torres, C. D. García-Beltrán, C. Astorga-Zaragoza, J.-C. Ponsart, and D. Theilliol, "SoC estimation using an extended Kalman filter for UAV applications," in *Proc. International Conference on Unmanned Aircraft Systems (ICUAS)*, Miami, FL, USA, 2017, pp. 179–187. [Online]. Available: https://hal.archives-ouvertes.fr/hal-01537730

[9] E. F. Hogge, B. M. Bole, S. L. Vazquez, C. S. Kulkarni, T. H. Strom, B. L. Hill, K. M. Smalling, and C. C. Quach, "Verification of prognostic algorithms to predict remaining flying time for electric unmanned vehicles," *International Journal of Prognostics and Health Management*, vol. 9, no. 1, 2018, accepted manuscript

also available as NASA Technical Reports Server document 20190025952. [Online]. Available: https://papers.phmsociety.org/index.php/ijphm/article/view/2700

[10] H. Dibbern, M. Roßberg, and C. Werner, "Comparative evaluation of investigation methods for estimating the load-dependent state of charge and end of discharge of a multirotor UAV battery," *Journal of Aviation Technology and Engineering*, vol. 12, no. 2, p. Article 4, 2023. [Online]. Available: https://docs.lib.purdue.edu/jate/vol12/iss2/4

[11] T. A. Rodrigues, J. Patrikar, A. Choudhry, J. Feldgoise, V. Arcot, A. Gahlaut, S. Lau, B. Moon, B. Wagner, H. S. Matthews, S. Scherer, and C. Samaras, "In-flight positional and energy use data set of a DJI Matrice 100 quadcopter for small package delivery," *Scientific Data*, vol. 8, p. Article 155, 2021. [Online]. Available: https://doi.org/10.1038/s41597-021-00930-x

[12] A. Bills, V. Viswanathan, S. Sripad, E. Frank, D. Charles, and W. L. Fredericks, "eVTOL battery dataset," Carnegie Mellon University, KiltHub repository, version 3, 2021. [Online]. Available: https://doi.org/10.1184/R1/14226830.v3

[13] A. Bills, S. Sripad, L. Fredericks, M. Guttenberg, D. Charles, E. Frank, and V. Viswanathan, "A battery dataset for electric vertical takeoff and landing aircraft," *Scientific Data*, vol. 10, p. Article 344, 2023. [Online]. Available: https://doi.org/10.1038/s41597-023-02180-5

[14] P. Kollmeyer, M. Naguib, and M. Skells, "LG 18650HG2 li-ion battery data," Mendeley Data, McMaster University, version 2, 2020. [Online]. Available: https://doi.org/10.17632/b5mj79w5w9.2

# Appendix A

# MATLAB Code

This appendix lists the MATLAB code. The folder structure is described in Section 4.1.

## A.1 Configuration

```
function p = battery_params()
% BATTERY_PARAMS  All battery and sensor numbers in one place.
%
%   p.filt : what the EKF BELIEVES (simple 1-RC model, nominal values)
%   p.true : the "real" battery in the simulator (2-RC model, values
%            deliberately a bit different, so the filter is tested honestly)
%   p.sens : sensor imperfections
%
% All battery values are PER CELL. The pack is 6 cells in series (6S),
% so the current is the same for every cell and pack voltage = 6 x cell.

p.Ns = 6;                       % cells in series
p.dt = 1;                       % time step (s)

% ---------- What the filter believes (nominal) ----------
p.filt.Cn         = 5.0;        % capacity (Ah)
p.filt.R0         = 0.008;      % ohmic resistance (ohm)
p.filt.R1         = 0.006;      % polarisation resistance (ohm)
p.filt.C1         = 5000;       % polarisation capacitance (F) -> tau = 30 s
p.filt.ocv_offset = 0;          % shift of the OCV curve (V)
p.filt.sig_model  = 0.010;      % allowance for model error in the voltage (V).
                                %   The filter's model is never exact; this
                                %   tells it not to trust its voltage
                                %   prediction to better than about 10 mV.

% ---------- The "real" battery (truth) ----------
p.true.Cn         = 5.0;                % change this to test an aged battery
p.true.R0         = p.filt.R0 * 1.10;   % 10% higher than the filter thinks
p.true.R1         = p.filt.R1 * 0.90;   % 10% lower
p.true.C1         = p.filt.C1 * 1.10;   % 10% higher
p.true.R2         = 0.0015;             % second, slow RC pair the filter does not
p.true.C2         = 266667;             %   know about (tau = R2*C2 = 400 s).
                                        %   Mild on purpose; experiment E6 uses
                                        %   R2 = 0.004 as a stress test.
```

```
p.true.ocv_offset = -0.005;            % real OCV curve sits 5 mV lower
p.true.V_cutoff   = 3.0;               % below this (per cell) = battery dead

% ---------- Sensors ----------
p.sens.bias = 0.5;             % current sensor reads 0.5 A too high (A)
p.sens.sigI = 0.2;             % current sensor noise (A, 1 sigma)
p.sens.sigV = 0.01;            % voltage sensor noise per cell (V, 1 sigma)
end
```

Listing A.1: `config/battery_params.m`

```
function [ocv, slope] = ocv_lookup(soc, offset)
% OCV_LOOKUP  Open-circuit voltage of one cell and its slope dOCV/dSoC.
%
%   [ocv, slope] = ocv_lookup(soc)          nominal curve
%   [ocv, slope] = ocv_lookup(soc, offset)  curve shifted by offset (V)
%
%   soc   : state of charge as a fraction (0..1), scalar or vector
%   ocv   : resting voltage (V)
%   slope : tangent slope (V per unit SoC) - this is H for the EKF
%
% The table is a representative Li-ion (NMC) curve: steep near empty,
% flatter in the middle. A 'pchip' fit gives a smooth curve with no kinks,
% so the slope does not jump between table points. The fit is built once
% and reused (persistent), which makes long Monte Carlo runs much faster.

persistent pp
if isempty(pp)
    soc_tab = [0 5 10 20 30 40 50 60 70 80 90 100] / 100;
    ocv_tab = [3.30 3.45 3.55 3.63 3.68 3.73 3.78 3.85 3.93 4.00 4.08 4.18];
    pp = pchip(soc_tab, ocv_tab);
end
if nargin < 2, offset = 0; end

s   = min(max(soc, 0), 1);                          % stay inside the table
ocv = ppval(pp, s) + offset;

if nargout > 1                                       % slope by finite difference
    h  = 0.005;
    hi = min(s + h, 1);
    lo = max(s - h, 0);
    slope = (ppval(pp, hi) - ppval(pp, lo)) ./ (hi - lo);
end
end
```

Listing A.2: `config/ocv_lookup.m`

```
function soc = soc_from_ocv(V, offset)
% SOC_FROM_OCV  Reverse lookup: resting voltage -> state of charge.
%   Used before takeoff, when the battery is at rest and V = OCV.
%   This gives the filter a sensible starting guess.

if nargin < 2, offset = 0; end
s   = linspace(0, 1, 1001);
soc = interp1(ocv_lookup(s, offset), s, V, 'linear', 'extrap');
soc = min(max(soc, 0), 1);
end
```

Listing A.3: `config/soc_from_ocv.m`

## A.2 Simulator

```
function b = battery_true_init(soc0)
% BATTERY_TRUE_INIT  The real battery at rest before takeoff.
b.soc = soc0;   % true state of charge (fraction)
b.V1  = 0;      % fast RC voltage (V) - zero after resting
b.V2  = 0;      % slow RC voltage (V)
end
```

Listing A.4: `sim/battery_true_init.m`

```
function [b, V] = battery_true_step(b, I, pt, dt)
% BATTERY_TRUE_STEP  The "real" battery: a 2-RC model, one time step.
%
%   b  : battery state (soc, V1, V2)
%   I  : true current drawn during this step (A, positive = discharge)
%   pt : true parameters (p.true from battery_params)
%   V  : terminal voltage of one cell at the END of the step (V)
%
% This model is deliberately richer than the filter's 1-RC model.

a1 = exp(-dt / (pt.R1 * pt.C1));
a2 = exp(-dt / (pt.R2 * pt.C2));

b.soc = b.soc - I * dt / (3600 * pt.Cn);
b.V1  = a1 * b.V1 + pt.R1 * (1 - a1) * I;
b.V2  = a2 * b.V2 + pt.R2 * (1 - a2) * I;

V = ocv_lookup(b.soc, pt.ocv_offset) - pt.R0 * I - b.V1 - b.V2;
end
```

Listing A.5: `sim/battery_true_step.m`

```
function [Im, Vm] = read_sensors(I, V, ps)
% READ_SENSORS  What the drone's sensors report (imperfect).
%
%   I, V : true current (A) and true cell voltage (V)
%   ps   : sensor settings (p.sens from battery_params)
%   Im   : measured current = true + constant bias + noise
%   Vm   : measured cell voltage = true + noise

Im = I + ps.bias + ps.sigI * randn;
Vm = V + ps.sigV * randn;
end
```

Listing A.6: `sim/read_sensors.m`

```
function m = make_mission(name)
% MAKE_MISSION  Delivery route and the current the drone draws in each phase.
%
%   m = make_mission('short')  3 drops, finishes with charge to spare (Day 2 test)
%   m = make_mission('long')   6 drops, too long for one battery - the drone
%                              MUST decide when to turn back (Day 5)
%
% Positions are in metres, home is at (0, 0).

if nargin < 1, name = 'short'; end

```

```
switch lower(name)
    case 'short'
        m.wp = [1200  400;
                2200 -400;
                3000  600];
    case 'long'
        m.wp = [1500    0;
                3000  800;
                4500    0;
                6000  800;
                7500    0;
                9000  800];
    otherwise
        error('Unknown mission "%s". Use ''short'' or ''long''.', name);
end
m.name = name;
m.home = [0 0];

% Flight phases: how long they last and the current they draw (pack current, A)
m.v_cruise  = 10;      % cruise ground speed (m/s)
m.T_takeoff = 30;      % s
m.T_hover   = 30;      % s, hovering at each drop-off
m.T_land    = 40;      % s
m.I_takeoff = 30;      % A  (climbing: hardest work)
m.I_cruise  = 15;      % A
m.I_hover   = 20;      % A
m.I_land    = 18;      % A
m.wind      = 0;       % e.g. 0.2 = headwind, cruise needs 20% more current
end
```

Listing A.7: `sim/make_mission.m`

```
function d = drone_init(m)
% DRONE_INIT  Drone sitting at home, about to take off.
d.pos       = m.home;     % position (m)
d.mode      = 'takeoff';  % takeoff | cruise | hover | return | landing | landed
d.t_in_mode = 0;          % seconds spent in the current mode
d.wp_index  = 1;          % next drop-off to visit
d.dist_flown = 0;         % metres flown so far (work done)
end
```

Listing A.8: `sim/drone_init.m`

```
function [d, I] = drone_step(d, m, dt)
% DRONE_STEP  Move the drone one time step and return the current it draws.
%
% The drone follows a simple sequence:
%   takeoff -> cruise to drop 1 -> hover -> cruise to drop 2 -> ... -> return ->
    landing -> landed
%
% To make the drone turn back early, the caller sets  d.mode = 'return'.

d.t_in_mode = d.t_in_mode + dt;

switch d.mode
    case 'takeoff'
        I = m.I_takeoff;
        if d.t_in_mode >= m.T_takeoff
```

```
            d = set_mode(d, 'cruise');
        end

    case 'cruise'
        I = m.I_cruise * (1 + m.wind);
        [d, arrived] = fly_towards(d, m.wp(d.wp_index, :), m.v_cruise, dt);
        if arrived
            d = set_mode(d, 'hover');
        end

    case 'hover'
        I = m.I_hover;
        if d.t_in_mode >= m.T_hover
            d.wp_index = d.wp_index + 1;
            if d.wp_index > size(m.wp, 1)
                d = set_mode(d, 'return');      % all deliveries done
            else
                d = set_mode(d, 'cruise');
            end
        end

    case 'return'
        I = m.I_cruise * (1 + m.wind);
        [d, arrived] = fly_towards(d, m.home, m.v_cruise, dt);
        if arrived
            d = set_mode(d, 'landing');
        end

    case 'landing'
        I = m.I_land;
        if d.t_in_mode >= m.T_land
            d = set_mode(d, 'landed');
        end

    case 'landed'
        I = 0;

    otherwise
        error('Unknown drone mode "%s".', d.mode);
end
end

% ---------------------------------------------------------------
function d = set_mode(d, mode)
d.mode = mode;
d.t_in_mode = 0;
end

function [d, arrived] = fly_towards(d, target, v, dt)
gap  = target - d.pos;
dist = norm(gap);
step = v * dt;
if dist <= step                     % close enough: land on the target
    d.pos = target;
    d.dist_flown = d.dist_flown + dist;
    arrived = true;
else
    d.pos = d.pos + step * gap / dist;
```

```
    d.dist_flown = d.dist_flown + step;
    arrived = false;
end
end
```

Listing A.9: `sim/drone_step.m`

```
function rec = run_flight(p, m, opts)
% RUN_FLIGHT  Fly one mission: real battery + sensors + EKF + charge counting.
%
%   rec = run_flight(p, m, opts)
%
%   opts.soc_true0  : true starting SoC (fraction)
%   opts.soc_guess0 : what the estimators assume at takeoff;
%                     NaN = read it from the resting voltage (normal use)
%   opts.sig_soc0   : EKF starting uncertainty on SoC (1 sigma)
%   opts.stage      : EKF stage 'A', 'B' or 'C' (see ekf_init)
%   opts.seed       : random seed (same seed = same noise = fair comparison)
%
%   Optional:
%   opts.n_rest     : number of resting-voltage readings averaged before
%                     takeoff (default 1; 10 cuts the starting error by ~3x)
%   opts.q_scale    : multiply the EKF process noise Q by this (default 1)
%   opts.r_scale    : multiply the EKF measurement noise R by this (default 1)
%
%   rec : everything recorded each second (truth, sensors, estimates)

dt = p.dt;
rng(opts.seed);
n_rest  = get_opt(opts, 'n_rest', 1);
q_scale = get_opt(opts, 'q_scale', 1);
r_scale = get_opt(opts, 'r_scale', 1);

% ---- Before takeoff: battery at rest, so voltage = OCV ----
batt = battery_true_init(opts.soc_true0);
V_rest = ocv_lookup(opts.soc_true0, p.true.ocv_offset);
Vm0 = 0;
for i = 1:n_rest                              % average a few readings to beat the
    noise
    [~, Vm_i] = read_sensors(0, V_rest, p.sens);
    Vm0 = Vm0 + Vm_i / n_rest;
end
soc_guess0 = opts.soc_guess0;
if isnan(soc_guess0)
    soc_guess0 = soc_from_ocv(Vm0, p.filt.ocv_offset);
end

f      = ekf_init(p, opts.stage, soc_guess0, opts.sig_soc0);
f.q    = f.q * q_scale;                       % tuning experiments (E5)
f.sigI = f.sigI * sqrt(q_scale);
f.Rv   = f.Rv * r_scale;
soc_cc = soc_guess0;                          % charge counting starts from the
    same guess
drone  = drone_init(m);

% ---- Storage ----
N  = 3600;
nx = numel(f.x);
```

```
rec.t = nan(1,N);  rec.I = nan(1,N);  rec.Im = nan(1,N);
rec.V = nan(1,N);  rec.Vm = nan(1,N); rec.V_pred = nan(1,N);
rec.soc = nan(1,N); rec.soc_cc = nan(1,N);
rec.x = nan(nx,N); rec.Pd = nan(nx,N);
rec.y = nan(1,N);  rec.S = nan(1,N);  rec.nis = nan(1,N);
rec.pos = nan(N,2); rec.mode = strings(1,N);

% ---- Fly ----
wobble = 0;
rec.dead = false;
for k = 1:N
    [drone, I_demand] = drone_step(drone, m, dt);
    if I_demand > 0
        wobble = 0.9*wobble + 0.3*randn;     % small random load changes
        I = max(I_demand + wobble, 0);
    else
        I = 0;
    end

    [batt, V] = battery_true_step(batt, I, p.true, dt);   % truth
    [Im, Vm]  = read_sensors(I, V, p.sens);               % what the drone sees
    [f, info] = ekf_step(f, Im, Vm, dt);                  % EKF
    soc_cc    = coulomb_count(soc_cc, Im, p.filt.Cn, dt); % baseline

    rec.t(k) = k*dt;  rec.I(k) = I;  rec.Im(k) = Im;
    rec.V(k) = V;     rec.Vm(k) = Vm; rec.V_pred(k) = info.V_pred;
    rec.soc(k) = batt.soc;  rec.soc_cc(k) = soc_cc;
    rec.x(:,k) = f.x;  rec.Pd(:,k) = diag(f.P);
    rec.y(k) = info.y; rec.S(k) = info.S; rec.nis(k) = info.nis;
    rec.pos(k,:) = drone.pos;  rec.mode(k) = drone.mode;

    if I > 0 && V < p.true.V_cutoff
        rec.dead = true;  break;             % battery died in the air
    end
    if strcmp(drone.mode, 'landed'), break; end
end

% ---- Trim unused storage ----
n = k;
names = fieldnames(rec);
for i = 1:numel(names)
    v = rec.(names{i});
    if ~isnumeric(v) && ~isstring(v), continue; end
    if size(v, 2) == N,     rec.(names{i}) = v(:, 1:n);
    elseif size(v, 1) == N, rec.(names{i}) = v(1:n, :);
    end
end
rec.soc_guess0 = soc_guess0;
rec.dist_flown = drone.dist_flown;
rec.opts = opts;
end

% ------------------------------------------------------------------
function v = get_opt(opts, name, default)
if isfield(opts, name), v = opts.(name); else, v = default; end
end
```

Listing A.10: `sim/run_flight.m`

```
function M = flight_metrics(r, which)
% FLIGHT_METRICS  How well an estimator tracked the true SoC in one flight.
%
%   M = flight_metrics(rec, 'ekf')   the EKF estimate
%   M = flight_metrics(rec, 'cc')    the charge-counting estimate
%
%   M.rmse    : root-mean-square SoC error (fraction)
%   M.final   : error at landing (fraction, + = estimate too high)
%   M.maxabs  : largest error
%   M.inside  : fraction of time the truth was inside the EKF's +/-3 sigma
%   M.nis     : mean normalised innovation squared (about 1 = consistent)
%   M.nees    : mean normalised SoC error squared, e^2/P (about 1 = consistent)
%   M.conv    : last time (s) the error was 2% or more (0 = never)

if strcmp(which, 'cc'), est = r.soc_cc; else, est = r.x(1,:); end
e = est - r.soc;

M.rmse   = sqrt(mean(e.^2));
M.final  = e(end);
M.maxabs = max(abs(e));
bad = find(abs(e) >= 0.02, 1, 'last');
if isempty(bad), M.conv = 0; else, M.conv = r.t(bad); end

if strcmp(which, 'cc')
    M.inside = NaN;  M.nis = NaN;  M.nees = NaN;
else
    M.inside = mean(abs(e) <= 3*sqrt(r.Pd(1,:)));
    M.nis    = mean(r.nis);
    M.nees   = mean(e.^2 ./ r.Pd(1,:));
end
end
```

Listing A.11: `sim/flight_metrics.m`

## A.3 Estimators

```
function f = ekf_init(p, stage, soc0, sig_soc0)
% EKF_INIT  Set up the filter: starting guess, starting uncertainty, tuning.
%
%   f = ekf_init(p, stage, soc0, sig_soc0)
%
%   p        : parameters from battery_params (the filter uses p.filt only)
%   stage    : which states the filter is allowed to learn
%              'A' -> SoC, V1                          (basic EKF)
%              'B' -> SoC, V1, R0                      (main filter: also learns
    resistance)
%              'C' -> SoC, V1, R0, bias b, capacity Cn (everything)
%   soc0     : starting SoC guess (fraction)
%   sig_soc0 : how unsure we are about that guess (1 sigma, fraction)
%
% The state is always x = [SoC; V1; b; Cn; R0]. States a stage does not
% learn are frozen at their nominal values (zero uncertainty and zero
% process noise), which is the same as leaving them out of the filter.

```

```
f.stage = upper(stage);

% What the filter believes about the battery
f.R1 = p.filt.R1;   f.C1 = p.filt.C1;
f.ocv_offset = p.filt.ocv_offset;

% Noise settings
f.sigI = p.sens.sigI;                                % current sensor noise (A)
f.Rv   = p.sens.sigV^2 ...                           % voltage sensor noise
       + (p.filt.R0 * p.sens.sigI)^2 ...             % current noise seen through
    R0
       + p.filt.sig_model^2;                         % allowance for model error (V
    ^2)

% Starting guess and uncertainty (1 sigma) for [SoC, V1, b, Cn, R0]
x0  = [soc0; 0; 0; p.filt.Cn; p.filt.R0];
sd0 = [sig_soc0, 0.01, 1.0, 0.5, 0.004];             % SoC, 10 mV, 1 A, 0.5 Ah, 4
    mOhm

% Process noise per step: how much each state may drift in ways the
% model does not explain (variances)
q = [1e-8, 1e-6, 1e-6, 1e-7, 1e-9];

switch f.stage
    case 'A', learn = [1 1 0 0 0];
    case 'B', learn = [1 1 0 0 1];
    case 'C', learn = [1 1 1 1 1];
    otherwise, error('Stage must be ''A'', ''B'' or ''C''.');
end
sd0(~learn) = 0;
q(~learn)   = 0;

f.learn = logical(learn);
f.x = x0;
f.P = diag(sd0.^2);
f.q = q;
end
```

Listing A.12: `estimators/ekf_init.m`

```
function [f, info] = ekf_step(f, Im, Vm, dt)
% EKF_STEP  One predict + update of the battery EKF.
%
%   f    : filter (from ekf_init), holds the estimate f.x and uncertainty f.P
%   Im   : measured current during this step (A)
%   Vm   : measured cell voltage at the end of the step (V)
%   info : innovation, its expected size, gain, ... (for plots and checks)
%
% State x = [SoC; V1; b; Cn; R0].

soc = f.x(1);  v1 = f.x(2);  b = f.x(3);  Cn = f.x(4);  R0 = f.x(5);
I = Im - b;                                          % bias-corrected current
a = exp(-dt / (f.R1 * f.C1));                        % how much of V1 survives one step

% ---------- 1. Predict: move the estimate with the model ----------
x_pred = [soc - I*dt/(3600*Cn);                      % charge counting
          a*v1 + f.R1*(1 - a)*I;                     % slow voltage relaxes toward R1*I
          b;                                         % bias, capacity and resistance
```

```
          Cn;                                   %   stay the same (apart from
          R0];                                  %   slow random wander)

F = eye(5);                                     % Jacobian of the process model
F(1,3) = dt/(3600*Cn);                          %   dSoC/db
F(1,4) = I*dt/(3600*Cn^2);                      %   dSoC/dCn
F(2,2) = a;                                     %   dV1/dV1
F(2,3) = -f.R1*(1 - a);                         %   dV1/db

G = [-dt/(3600*Cn); f.R1*(1 - a); 0; 0; 0];     % how current-sensor noise enters
Q = G * f.sigI^2 * G' + diag(f.q);
P_pred = F * f.P * F' + Q;                      % uncertainty grows

% ---------- 2. Compare: expected vs measured voltage ----------
[ocv, docv] = ocv_lookup(x_pred(1), f.ocv_offset);
I_pred = Im - x_pred(3);
V_pred = ocv - x_pred(2) - x_pred(5)*I_pred;
y = Vm - V_pred;                                % innovation (the surprise)

H = [docv, -1, x_pred(5), 0, -I_pred];          % Jacobian of the measurement model
S = H * P_pred * H' + f.Rv;                     % expected size of the surprise (one
    number)

% ---------- 3. Update: move by K x surprise ----------
K = P_pred * H' / S;                            % Kalman gain (5x1)
x = x_pred + K * y;
IKH = eye(5) - K*H;
P = IKH * P_pred * IKH' + K * f.Rv * K';        % Joseph form: stays symmetric and
   positive
P = (P + P') / 2;

x(1) = min(max(x(1), 0), 1);                    % SoC stays in [0, 1]
x(4) = max(x(4), 0.5);                          % capacity stays positive
x(5) = max(x(5), 1e-4);                         % resistance stays positive

f.x = x;  f.P = P;
info.y = y;  info.S = S;  info.K = K;  info.V_pred = V_pred;
info.nis = y^2 / S;                             % normalised innovation squared
end
```

Listing A.13: `estimators/ekf_step.m`

```
function soc = coulomb_count(soc, Im, Cn, dt)
% COULOMB_COUNT  Baseline: track SoC by adding up the measured current.
%   No voltage, no correction - so any start error or sensor bias stays.
soc = soc - Im * dt / (3600 * Cn);
end
```

Listing A.14: `estimators/coulomb_count.m`

## A.4 Decision rule

```
function dsoc = trip_soc(dist_home, I_cruise, b, Cn, m)
% TRIP_SOC  Charge (as a fraction of SoC) needed to fly home and land.
%
```

```
%   dist_home : straight-line distance to home (m)
%   I_cruise  : cruise current as the SENSOR reads it (A), learned in flight
%   b         : estimated current-sensor bias (A); 0 if unknown
%   Cn        : capacity to divide by (Ah)
%   m         : mission (speed, landing time and current)
%
% The cruise part uses the same drain rate the filter uses, (I - b)/Cn,
% because that combination is what the voltage makes observable (Day 4).
% I_cruise is a MEASURED current, so the bias is removed from it; the
% landing current m.I_land is a nominal TRUE current, so it is used as is.

t_cruise = dist_home / m.v_cruise;                      % s
charge   = (I_cruise - b) * t_cruise + m.I_land * m.T_land;   % A*s
dsoc     = charge / (3600 * Cn);
end
```

Listing A.15: `decision/trip_soc.m`

```
function [ps, margin, dsoc, sig_total] = p_safe_return(f, dist_home, I_cruise, m,
    rp)
% P_SAFE_RETURN  Probability of getting home with the reserve still left.
%
%   f         : EKF (stage C) with estimate f.x and uncertainty f.P
%   dist_home : distance to home (m)
%   I_cruise  : learned cruise current as measured (A)
%   m         : mission
%   rp        : RTH settings: rp.reserve (fraction), rp.trip_frac (fraction)
%
%   ps        : P(safe return) = Phi(margin / sig_total)
%   margin    : SoC estimate - charge needed - reserve
%   dsoc      : charge needed to get home (fraction)
%   sig_total : combined uncertainty of SoC and of the trip (fraction)

soc = f.x(1);  b = f.x(3);  Cn = f.x(4);
dsoc   = trip_soc(dist_home, I_cruise, b, Cn, m);
margin = soc - dsoc - rp.reserve;

sig_soc   = sqrt(f.P(1,1));          % from the EKF: how unsure is the SoC
sig_trip  = rp.trip_frac * dsoc;     % wind and load surprises on the way home
sig_total = sqrt(sig_soc^2 + sig_trip^2);

ps = 0.5 * erfc(-margin / (sqrt(2) * sig_total));   % normal CDF, base MATLAB
end
```

Listing A.16: `decision/p_safe_return.m`

```
function [go_home, low_count] = rth_policy(policy, s, rp)
% RTH_POLICY  The four turn-back rules. Returns true when the drone should go home
    .
%
%   policy : 'A' voltage threshold
%            'B' charge counting + margin rule
%            'C' EKF estimate + margin rule (ignores uncertainty)
%            'D' EKF risk-aware: P(safe return) below a threshold
%   s      : what the drone knows this second (fields used below)
%   rp     : settings (rp.V_thr, rp.V_hold, rp.reserve, rp.p_thr)

low_count = s.low_count;
```

```
switch policy
    case 'A'   % voltage stays below the threshold for V_hold seconds
        if s.Vm < rp.V_thr, low_count = low_count + 1; else, low_count = 0; end
        go_home = low_count >= rp.V_hold;
    case 'B'   % charge counting says the margin is gone
        go_home = s.soc_cc - s.dsoc_cc - rp.reserve < 0;
    case 'C'   % EKF estimate says the margin is gone
        go_home = s.margin < 0;
    case 'D'   % EKF is no longer sure enough of getting home with the reserve
        go_home = s.ps < rp.p_thr;
    otherwise
        error('Unknown policy "%s".', policy);
end
end
```

Listing A.17: `decision/rth_policy.m`

## A.5 Return-to-home missions

```
function c = draw_conditions(p)
% DRAW_CONDITIONS  Random conditions for one Monte Carlo mission.
%   Every policy flies the same conditions, so the comparison is fair.
u = @(a, b) a + (b - a) * rand;

c.soc0  = u(0.60, 1.00);              % starting charge
c.bias  = u(-1.0, 1.0);               % current-sensor bias (A)
c.wind  = u(-0.20, 0.20);             % cruise needs -20%..+20% current
c.Cn    = p.true.Cn * u(0.80, 1.00);  % battery age: 80-100% of rated capacity
c.R0    = p.true.R0 * u(0.85, 1.15);  % the real battery's parameters vary too
c.R1    = p.true.R1 * u(0.85, 1.15);
c.C1    = p.true.C1 * u(0.85, 1.15);
c.R2    = p.true.R2 * u(0.85, 1.15);
end
```

Listing A.18: `sim/draw_conditions.m`

```
function out = run_mission_rth(p, m, c, policy, rp, seed)
% RUN_MISSION_RTH  Fly a mission that is too long for one battery and let a
% policy decide when to turn back.
%
%   p      : battery_params()
%   m      : mission (use make_mission('long'))
%   c      : random conditions from draw_conditions (start SoC, bias, wind, age
    ...)
%   policy : 'A', 'B', 'C' or 'D' (see rth_policy)
%   rp     : RTH settings (reserve, trip_frac, p_thr, V_thr, V_hold)
%   seed   : random seed for the sensor noise
%
%   out.died      : true if the battery died in the air
%   out.soc_land  : true SoC at landing
%   out.drops     : deliveries completed
%   out.t_rth     : second at which the policy turned back (NaN = never)
%   out.rec       : second-by-second record (for plots)
%
% The stage C EKF and charge counting always run, whichever policy decides.
```

```
19
20  % ---- Apply this mission's conditions ----
21  p.sens.bias = c.bias;
22  p.true.Cn = c.Cn;  p.true.R0 = c.R0;  p.true.R1 = c.R1;  p.true.C1 = c.C1;  p.true
        .R2 = c.R2;
23  m.wind = c.wind;
24  dt = p.dt;
25  rng(seed);
26
27  % ---- Before takeoff: average 10 resting readings for the starting guess ----
28  batt = battery_true_init(c.soc0);
29  V_rest = ocv_lookup(c.soc0, p.true.ocv_offset);
30  Vm0 = 0;
31  for i = 1:10
32      [~, Vm_i] = read_sensors(0, V_rest, p.sens);
33      Vm0 = Vm0 + Vm_i / 10;
34  end
35  soc_guess0 = soc_from_ocv(Vm0, p.filt.ocv_offset);
36
37  f      = ekf_init(p, 'C', soc_guess0, 0.03);     % stage C drives the decision
38  soc_cc = soc_guess0;                             % charge counting starts from the
         same guess
39  drone  = drone_init(m);
40  I_cr   = m.I_cruise;                             % learned cruise current (as
        measured)
41
42  % ---- Storage ----
43  N = 4000;
44  rec.t = nan(1,N);  rec.soc = nan(1,N);  rec.soc_hat = nan(1,N);  rec.sig = nan(1,N
        );
45  rec.soc_cc = nan(1,N);  rec.ps = nan(1,N);  rec.margin = nan(1,N);  rec.dsoc = nan
        (1,N);
46  rec.Vm = nan(1,N);  rec.pos = nan(N,2);  rec.mode = strings(1,N);
47
48  wobble = 0;  low_count = 0;
49  out.died = false;  out.t_rth = NaN;
50  for k = 1:N
51      [drone, I_demand] = drone_step(drone, m, dt);
52      if I_demand > 0
53          wobble = 0.9*wobble + 0.3*randn;
54          I = max(I_demand + wobble, 0);
55      else
56          I = 0;
57      end
58
59      [batt, V] = battery_true_step(batt, I, p.true, dt);
60      [Im, Vm]  = read_sensors(I, V, p.sens);
61      f         = ekf_step(f, Im, Vm, dt);
62      soc_cc    = coulomb_count(soc_cc, Im, p.filt.Cn, dt);
63      if strcmp(drone.mode, 'cruise')
64          I_cr = 0.98*I_cr + 0.02*Im;              % slowly learn the real cruise
        current
65      end
66
67      % ---- What the drone knows, and the decision ----
68      dist_home = norm(drone.pos - m.home);
69      [ps, margin, dsoc] = p_safe_return(f, dist_home, I_cr, m, rp);
70      if any(strcmp(drone.mode, {'cruise', 'hover'}))       % decide only while
```

```
    working
        s.Vm = Vm;  s.low_count = low_count;  s.ps = ps;  s.margin = margin;
        s.soc_cc  = soc_cc;
        s.dsoc_cc = trip_soc(dist_home, I_cr, 0, p.filt.Cn, m);   % no bias, label
     capacity
        [go_home, low_count] = rth_policy(policy, s, rp);
        if go_home
            drone.mode = 'return';  drone.t_in_mode = 0;
            out.t_rth = k;
        end
    end

    rec.t(k) = k*dt;  rec.soc(k) = batt.soc;  rec.soc_hat(k) = f.x(1);  rec.sig(k)
     = sqrt(f.P(1,1));
    rec.soc_cc(k) = soc_cc;  rec.ps(k) = ps;  rec.margin(k) = margin;  rec.dsoc(k)
     = dsoc;
    rec.Vm(k) = Vm;  rec.pos(k,:) = drone.pos;  rec.mode(k) = drone.mode;

    if I > 0 && (V < p.true.V_cutoff || batt.soc <= 0)
        out.died = true;  break;                % battery died in the air
    end
    if strcmp(drone.mode, 'landed'), break; end
end

% ---- Trim and summarise ----
n = k;
names = fieldnames(rec);
for i = 1:numel(names)
    v = rec.(names{i});
    if size(v, 2) == N, rec.(names{i}) = v(:, 1:n); else, rec.(names{i}) = v(1:n,
    :); end
end
out.soc_land  = batt.soc;
out.drops     = drone.wp_index - 1;          % a drop counts once its hover is
    finished
e_ekf = rec.soc_hat - rec.soc;               % how well the estimators tracked SoC
e_cc  = rec.soc_cc  - rec.soc;
out.rmse_ekf  = sqrt(mean(e_ekf.^2));
out.rmse_cc   = sqrt(mean(e_cc.^2));
out.inside    = mean(abs(e_ekf) <= 3*rec.sig);
out.dist      = drone.dist_flown;
out.T         = n * dt;
out.rec       = rec;
end
```

Listing A.19: `sim/run_mission_rth.m`

## A.6 Main scripts

```
%% Day 2 - Test the simulator (no filter yet)
% Flies the 'short' delivery mission on the "real" battery and records
% what the sensors would report. Checks that everything looks realistic
% before we add the EKF on Day 3.
%
% Run from the drone_battery_ekf folder.
```

```
7
8  clear; clc; close all;
9  root = fileparts(mfilename('fullpath'));
10 addpath(fullfile(root, 'config'), fullfile(root, 'sim'));
11
12 rng(2);                                  % same random numbers every run
13 p  = battery_params();
14 m  = make_mission('short');
15 dt = p.dt;
16
17 %% Run the flight
18 soc0  = 0.95;                            % battery starts at 95%
19 batt  = battery_true_init(soc0);
20 drone = drone_init(m);
21
22 N = 3600;                                % room for up to 1 hour of logging
23 rec.t = nan(1,N); rec.I = nan(1,N); rec.Im = nan(1,N);
24 rec.V = nan(1,N); rec.Vm = nan(1,N); rec.ocv = nan(1,N);
25 rec.soc = nan(1,N); rec.pos = nan(N,2); rec.mode = strings(1,N);
26
27 wobble = 0;                              % small random load changes (gusts,
       manoeuvres)
28 dead = false;
29 for k = 1:N
30     [drone, I_demand] = drone_step(drone, m, dt);
31     if I_demand > 0
32         wobble = 0.9*wobble + 0.3*randn;     % slowly varying, about +/-0.7 A
33         I = max(I_demand + wobble, 0);
34     else
35         I = 0;
36     end
37
38     [batt, V] = battery_true_step(batt, I, p.true, dt);
39     [Im, Vm]  = read_sensors(I, V, p.sens);
40
41     rec.t(k) = k*dt;   rec.I(k) = I;   rec.Im(k) = Im;
42     rec.V(k) = V;      rec.Vm(k) = Vm; rec.ocv(k) = ocv_lookup(batt.soc, p.true.
       ocv_offset);
43     rec.soc(k) = batt.soc;  rec.pos(k,:) = drone.pos;  rec.mode(k) = drone.mode;
44
45     if I > 0 && V < p.true.V_cutoff
46         dead = true;  break;                 % battery died in the air
47     end
48     if strcmp(drone.mode, 'landed'), break; end
49 end
50 n = k;                                    % number of logged steps
51 f = fieldnames(rec);
52 for i = 1:numel(f)                        % trim unused space
53     if strcmp(f{i}, 'pos'), rec.pos = rec.pos(1:n,:); else, rec.(f{i}) = rec.(f{i
       })(1:n); end
54 end
55 tm = rec.t / 60;                          % minutes
56
57 %% Summary
58 fprintf('Mission ''%s'': %s after %.1f min\n', m.name, ...
59         ternary(dead, 'BATTERY DIED IN THE AIR', 'landed safely'), tm(end));
60 fprintf('Distance flown        : %.2f km\n', drone.dist_flown/1000);
61 fprintf('Charge used           : %.2f Ah (true SoC %.0f%% -> %.1f%%)\n', ...
```

```
        (soc0 - rec.soc(end))*p.true.Cn, 100*soc0, 100*rec.soc(end));
fprintf('Lowest cell voltage    : %.3f V (cutoff %.1f V)\n', min(rec.V), p.true.
    V_cutoff);
sag = rec.ocv - rec.V;
fprintf('Largest voltage sag    : %.0f mV per cell\n', 1000*max(sag));
soc_cc_end = soc0 - sum(rec.Im)*dt/(3600*p.true.Cn);   % charge counting with the
    biased sensor
fprintf('Charge counting error : %+.1f%% at landing (sensor bias + noise)\n', ...
        100*(soc_cc_end - rec.soc(end)));

%% Figure A - OCV curve: real battery vs what the filter believes
s = linspace(0, 1, 201);
[ocv_f, slope_f] = ocv_lookup(s, p.filt.ocv_offset);
ocv_t = ocv_lookup(s, p.true.ocv_offset);
figA = figure('Name', 'OCV curve', 'Color', 'w');
yyaxis left;  plot(100*s, ocv_f, 'b-', 100*s, ocv_t, 'k--', 'LineWidth', 1.3);
ylabel('OCV per cell (V)');
yyaxis right; plot(100*s, 1000*slope_f/100, 'LineWidth', 1); ylabel('Slope (mV per
     1% SoC)');
xlabel('State of charge (%)'); grid on;
legend('Filter''s curve', 'Real battery (5 mV lower)', 'Slope', 'Location', '
    northwest');
title('OCV curve: steep near empty, flatter in the middle');

%% Figure B - Mission map and current profile
figB = figure('Name', 'Mission', 'Color', 'w', 'Position', [100 100 900 650]);
subplot(2,1,1); hold on; grid on; axis equal;
plot(rec.pos(:,1)/1000, rec.pos(:,2)/1000, 'b-', 'LineWidth', 1.3);
plot(m.wp(:,1)/1000, m.wp(:,2)/1000, 'rs', 'MarkerFaceColor', 'r', 'MarkerSize',
    8);
plot(0, 0, 'k^', 'MarkerFaceColor', 'k', 'MarkerSize', 10);
text(m.wp(:,1)/1000 + 0.08, m.wp(:,2)/1000, compose('Drop %d', (1:size(m.wp,1))'))
    ;
xlabel('East (km)'); ylabel('North (km)'); title('Flight path: home \rightarrow 3
    drop-offs \rightarrow home');
legend('Path', 'Drop-offs', 'Home', 'Location', 'bestoutside');

subplot(2,1,2); hold on; grid on;
plot(tm, rec.Im, 'Color', [0.7 0.7 0.7]);
plot(tm, rec.I, 'b', 'LineWidth', 1.2);
xlabel('Time (min)'); ylabel('Current (A)');
legend('Measured (biased, noisy)', 'True', 'Location', 'northeast');
title('Current: takeoff spike, cruise, hover at each drop, landing');

%% Figure C - Voltage sag and true SoC
figC = figure('Name', 'Voltage and SoC', 'Color', 'w', 'Position', [150 150 900
    650]);
subplot(2,1,1); hold on; grid on;
plot(tm, rec.Vm, 'Color', [0.75 0.75 0.75]);
plot(tm, rec.ocv, 'k--', 'LineWidth', 1.2);
plot(tm, rec.V, 'b', 'LineWidth', 1.2);
ylabel('Cell voltage (V)');
legend('Measured', 'OCV (resting voltage)', 'True terminal voltage', 'Location', '
    northeast');
title('The gap between the dashed and blue lines is the voltage sag');

subplot(2,1,2); grid on;
plot(tm, 100*rec.soc, 'k', 'LineWidth', 1.4);
```

```
xlabel('Time (min)'); ylabel('True SoC (%)');
title('True state of charge (the filter will have to estimate this)');

%% Save figures
out = fullfile(root, 'results', 'day2');
if ~exist(out, 'dir'), mkdir(out); end
save_fig(figA, fullfile(out, 'fig_ocv_curve.png'));
save_fig(figB, fullfile(out, 'fig_mission.png'));
save_fig(figC, fullfile(out, 'fig_voltage_soc.png'));
save(fullfile(out, 'day2_log.mat'), 'rec', 'p', 'm');
fprintf('\nFigures saved in %s\n', out);

%% ---------------- Local functions ----------------
function save_fig(fig, file)
if exist('exportgraphics', 'file')
    exportgraphics(fig, file, 'Resolution', 150);
else
    saveas(fig, file);
end
end

function out = ternary(cond, a, b)
if cond, out = a; else, out = b; end
end
```

Listing A.20: `main_day2_simulator.m`

```
%% Day 3 - First EKF: experiments E1 (normal flight) and E2 (wrong start)
% Flies the 'short' mission and compares three ways of tracking SoC:
%   Charge counting  - adds up the measured current (baseline)
%   EKF stage A      - learns SoC and V1
%   EKF stage B      - also learns the resistance R0 (our main filter)
%
% Run from the drone_battery_ekf folder.

clear; clc; close all;
root = fileparts(mfilename('fullpath'));
addpath(fullfile(root, 'config'), fullfile(root, 'sim'), fullfile(root, '
    estimators'));

p = battery_params();
m = make_mission('short');

% E1: battery at 95%, starting guess read from the resting voltage
E1 = struct('soc_true0', 0.95, 'soc_guess0', NaN,  'sig_soc0', 0.03, 'seed', 2);
% E2: battery really at 85%, but everyone assumes it is full
E2 = struct('soc_true0', 0.85, 'soc_guess0', 1.00, 'sig_soc0', 0.20, 'seed', 2);

E1.stage = 'A';  r1A = run_flight(p, m, E1);
E1.stage = 'B';  r1B = run_flight(p, m, E1);
E2.stage = 'A';  r2A = run_flight(p, m, E2);
E2.stage = 'B';  r2B = run_flight(p, m, E2);

%% Results table
fprintf('%-26s %9s %9s %9s %11s %7s %12s\n', 'Run', 'RMSE', 'Final', 'Max', 'In
    +/-3sig', 'NIS', 'Conv. <2%');
show('E1  charge counting', r1A, 'cc');
show('E1  EKF stage A',     r1A, 'ekf');
```

```
show('E1  EKF stage B (R0)', r1B, 'ekf');
show('E2  charge counting', r2A, 'cc');
show('E2  EKF stage A',     r2A, 'ekf');
show('E2  EKF stage B (R0)', r2B, 'ekf');
fprintf('\nStage B resistance estimate at landing (E1): %.2f mOhm  (filter started at %.1f, real R0 = %.1f mOhm)\n', ...
        1000*r1B.x(5,end), 1000*p.filt.R0, 1000*p.true.R0);

%% Figure 1 - Does the model explain the measured voltage? (E1, stage B)
tm = r1B.t / 60;
fig1 = figure('Name', 'E1 voltage', 'Color', 'w', 'Position', [100 100 900 600]);
subplot(2,1,1); hold on; grid on;
plot(tm, r1B.Vm, 'Color', [0.75 0.75 0.75]);
plot(tm, r1B.V_pred, 'b', 'LineWidth', 1.1);
ylabel('Cell voltage (V)'); legend('Measured', 'EKF expected (before update)', 'Location', 'northeast');
title('E1: the EKF''s model follows the measured voltage, including every sag');
subplot(2,1,2); hold on; grid on;
plot(tm, 1000*r1B.y, 'Color', [0.3 0.3 0.3]);
plot(tm,  3000*sqrt(r1B.S), 'r--', tm, -3000*sqrt(r1B.S), 'r--');
ylabel('Innovation (mV)'); xlabel('Time (min)');
legend('Innovation (measured - expected)', '\pm3\surd S (expected size)', 'Location', 'northeast');
title('Innovations look like random noise inside their expected size: the filter is healthy');

%% Figure 2 - SoC tracking (E1 and E2)
fig2 = figure('Name', 'SoC tracking', 'Color', 'w', 'Position', [120 120 900 650]);
plot_soc(subplot(2,1,1), r1A, r1B, 'E1: normal flight (guess read from resting voltage)');
plot_soc(subplot(2,1,2), r2A, r2B, 'E2: battery at 85% but assumed full');

%% Figure 3 - SoC error with the EKF's own +/-3 sigma band
fig3 = figure('Name', 'SoC error', 'Color', 'w', 'Position', [140 140 900 650]);
plot_err(subplot(2,1,1), r1A, r1B, 'E1: error');
plot_err(subplot(2,1,2), r2A, r2B, 'E2: error');

%% Figure 4 - Stage B learns the resistance
fig4 = figure('Name', 'R0 estimate', 'Color', 'w', 'Position', [160 160 900 350]);
hold on; grid on;
sdR = 3000*sqrt(r1B.Pd(5,:));
fill([tm fliplr(tm)], [1000*r1B.x(5,:)+sdR fliplr(1000*r1B.x(5,:)-sdR)], [0.85 0.9 1], 'EdgeColor', 'none');
plot(tm, 1000*r1B.x(5,:), 'b', 'LineWidth', 1.3);
yline(1000*p.true.R0, 'k--', 'Real R0');
yline(1000*p.filt.R0, ':', 'Filter''s starting value');
ylabel('R0 (m\Omega)'); xlabel('Time (min)');
legend('\pm3\sigma', 'Stage B estimate', 'Location', 'southeast');
title('E1: stage B learns that the real battery has more resistance than assumed');

%% Save
out = fullfile(root, 'results', 'day3');
if ~exist(out, 'dir'), mkdir(out); end
save_fig(fig1, fullfile(out, 'fig_e1_voltage_innovation.png'));
save_fig(fig2, fullfile(out, 'fig_soc_tracking.png'));
save_fig(fig3, fullfile(out, 'fig_soc_error.png'));
```

```
save_fig(fig4, fullfile(out, 'fig_r0_estimate.png'));
save(fullfile(out, 'day3_runs.mat'), 'r1A', 'r1B', 'r2A', 'r2B', 'p', 'm');
fprintf('Figures saved in %s\n', out);

%% ---------------- Local functions ----------------
function show(name, r, which)
if strcmp(which, 'cc'), est = r.soc_cc; else, est = r.x(1,:); end
e = est - r.soc;
bad = find(abs(e) >= 0.02, 1, 'last');            % last time the error was 2% or more
if isempty(bad), conv = 0; else, conv = r.t(bad); end
if strcmp(which, 'cc')
    fprintf('%-26s %8.2f%% %+8.2f%% %8.2f%% %11s %7s %12s\n', name, 100*sqrt(mean(e.^2)), 100*e(end), 100*max(abs(e)), '-', '-', '-');
else
    inside = mean(abs(e) <= 3*sqrt(r.Pd(1,:)));
    if conv >= r.t(end), cs = 'never'; else, cs = sprintf('%d s', conv); end
    fprintf('%-26s %8.2f%% %+8.2f%% %8.2f%% %10.0f%% %7.2f %12s\n', name, 100*sqrt(mean(e.^2)), 100*e(end), 100*max(abs(e)), 100*inside, mean(r.nis), cs);
end
end

function plot_soc(ax, rA, rB, ttl)
axes(ax); hold on; grid on;
tm = rB.t / 60;
s3 = 3*sqrt(rB.Pd(1,:));
fill([tm fliplr(tm)], 100*[rB.x(1,:)+s3 fliplr(rB.x(1,:)-s3)], [0.85 0.9 1], 'EdgeColor', 'none');
plot(tm, 100*rB.soc, 'k', 'LineWidth', 1.6);
plot(tm, 100*rB.soc_cc, 'r--', 'LineWidth', 1.2);
plot(rA.t/60, 100*rA.x(1,:), 'Color', [0.95 0.6 0.1], 'LineWidth', 1.1);
plot(tm, 100*rB.x(1,:), 'b', 'LineWidth', 1.2);
ylabel('SoC (%)'); title(ttl);
legend('Stage B \pm3\sigma', 'True', 'Charge counting', 'EKF stage A', 'EKF stage B', 'Location', 'northeast');
end

function plot_err(ax, rA, rB, ttl)
axes(ax); hold on; grid on;
tm = rB.t / 60;
s3 = 300*sqrt(rB.Pd(1,:));
fill([tm fliplr(tm)], [s3 -fliplr(s3)], [0.85 0.9 1], 'EdgeColor', 'none');
plot(tm, 100*(rB.soc_cc - rB.soc), 'r--', 'LineWidth', 1.1);
plot(rA.t/60, 100*(rA.x(1,:) - rA.soc), 'Color', [0.95 0.6 0.1], 'LineWidth', 1.1);
plot(tm, 100*(rB.x(1,:) - rB.soc), 'b', 'LineWidth', 1.2);
yline(0, 'k:');
ylabel('Error (% SoC)'); xlabel('Time (min)'); title(ttl);
legend('Stage B \pm3\sigma', 'Charge counting', 'EKF stage A', 'EKF stage B', 'Location', 'best');
end

function save_fig(fig, file)
if exist('exportgraphics', 'file')
    exportgraphics(fig, file, 'Resolution', 150);
else
    saveas(fig, file);
end
```

```
end
```

Listing A.21: main_day3_ekf.m

```
%% Day 4 - Experiments E3 to E8: stress-testing the filter
%   E3  current sensor reads 0.8 A too high
%   E4  aged battery: only 85% of its rated capacity
%   E5  tuning: what happens when Q and R are set badly
%   E6  stress test: a much bigger unmodelled slow RC effect
%   E7  does uncertainty grow where the OCV curve is flat?
%   E8  summary: charge counting vs stages A, B, C over 20 random flights
%
% Run from the drone_battery_ekf folder. E8 flies 300 missions (about 1-3 min).

clear; clc; close all;
root = fileparts(mfilename('fullpath'));
addpath(fullfile(root, 'config'), fullfile(root, 'sim'), fullfile(root, '
    estimators'));

p0 = battery_params();
m  = make_mission('short');

% Normal flight settings; every scenario starts from these
base = struct('soc_true0', 0.95, 'soc_guess0', NaN, 'sig_soc0', 0.03, ...
              'seed', 2, 'n_rest', 10);   % n_rest: average 10 resting readings
    before takeoff

%% Scenarios
sc = {};
sc{end+1} = scenario('E1', 'Normal flight',          p0, base);

o = base;  o.soc_true0 = 0.85;  o.soc_guess0 = 1.00;  o.sig_soc0 = 0.20;
sc{end+1} = scenario('E2', 'Wrong start (85%, assumed full)', p0, o);

p = p0;  p.sens.bias = 0.8;
sc{end+1} = scenario('E3', 'Sensor bias 0.8 A',      p, base);

p = p0;  p.true.Cn = 0.85 * p0.true.Cn;  o = base;  o.soc_true0 = 1.00;
sc{end+1} = scenario('E4', 'Aged battery (85% capacity)', p, o);

p = p0;  p.true.R2 = 0.004;  p.true.C2 = 1e5;        % slow RC 2.7x bigger (tau
    400 s)
sc{end+1} = scenario('E6', 'Strong model mismatch',  p, base);

get_sc = @(code) sc{find(cellfun(@(s) strcmp(s.code, code), sc), 1)};

%% E3 and E4 - can stage C learn the bias and the capacity?
s3 = get_sc('E3');  r3B = run_stage(s3, m, 'B');  r3C = run_stage(s3, m, 'C');
s4 = get_sc('E4');  r4B = run_stage(s4, m, 'B');  r4C = run_stage(s4, m, 'C');

fprintf('==== E3: sensor bias 0.8 A ====\n');
print_row('Charge counting', flight_metrics(r3B, 'cc'));
print_row('EKF stage B',     flight_metrics(r3B, 'ekf'));
print_row('EKF stage C',     flight_metrics(r3C, 'ekf'));
fprintf('Stage C bias estimate at landing: %+.2f A (true %+.2f A)\n\n', r3C.x(3,
    end), s3.p.sens.bias);

fprintf('==== E4: aged battery (true capacity %.2f Ah, filter assumes %.2f Ah)
```

```
    ====\n', s4.p.true.Cn, s4.p.filt.Cn);
print_row('Charge counting', flight_metrics(r4B, 'cc'));
print_row('EKF stage B',     flight_metrics(r4B, 'ekf'));
print_row('EKF stage C',     flight_metrics(r4C, 'ekf'));
fprintf('Stage C at landing: capacity %.2f Ah (true %.2f), bias %+.2f A (true %+.2
    f)\n', ...
        r4C.x(4,end), s4.p.true.Cn, r4C.x(3,end), s4.p.sens.bias);
fprintf('True SoC at landing %.1f%%; charge counting believed %.1f%%\n\n', 100*r4B
    .soc(end), 100*r4B.soc_cc(end));

figE3 = figure('Name', 'E3 bias', 'Color', 'w', 'Position', [60 60 900 650]);
plot_errors(subplot(2,1,1), {r3B, r3C}, {'EKF stage B', 'EKF stage C'}, 'E3: SoC
    error with a 0.8 A sensor bias');
plot_param(subplot(2,1,2), r3C, 3, 1, s3.p.sens.bias, 'True bias', 'Bias (A)', ...
           'E3: stage C cannot pin down the bias within one flight');

figE4 = figure('Name', 'E4 aged', 'Color', 'w', 'Position', [80 80 900 650]);
plot_errors(subplot(2,1,1), {r4B, r4C}, {'EKF stage B', 'EKF stage C'}, 'E4: SoC
    error with an aged battery (85% capacity)');
plot_param(subplot(2,1,2), r4C, 4, 1, s4.p.true.Cn, 'True capacity', 'Capacity (Ah
    )', ...
           'E4: stage C moves its capacity estimate toward the truth');
yline(s4.p.filt.Cn, ':', 'Rated (label) capacity', 'HandleVisibility', 'off');

%% E5 - tuning: turn the Q and R knobs
s1 = get_sc('E1');
tune = [1 1; 0.01 1; 100 1; 1 0.01; 1 100];          % [Q scale, R scale]
tune_names = {'Q x1, R x1 (chosen)', 'Q x0.01', 'Q x100', 'R x0.01', 'R x100'};
rT = cell(1, size(tune,1));
fprintf('==== E5: tuning (stage B, normal flight) ====\n');
for i = 1:size(tune,1)
    o = s1.opts;  o.stage = 'B';  o.q_scale = tune(i,1);  o.r_scale = tune(i,2);
    rT{i} = run_flight(s1.p, m, o);
    print_row(tune_names{i}, flight_metrics(rT{i}, 'ekf'));
end
fprintf('\n');

figE5 = figure('Name', 'E5 tuning', 'Color', 'w', 'Position', [100 100 900 650]);
subplot(2,1,1); hold on; grid on;
cols = lines(numel(rT));
for i = 1:numel(rT)
    plot(rT{i}.t/60, 100*(rT{i}.x(1,:) - rT{i}.soc), 'Color', cols(i,:), '
    LineWidth', 1.2);
end
yline(0, 'k:', 'HandleVisibility', 'off');
ylabel('SoC error (%)'); xlabel('Time (min)'); legend(tune_names, 'Location', '
    best');
title('E5: SoC error for different tunings');
subplot(2,1,2);
MT = cellfun(@(r) flight_metrics(r, 'ekf'), rT, 'UniformOutput', false);
MT = [MT{:}];
yyaxis left;  bar(100*[MT.rmse]);  ylabel('RMSE (% SoC)');
yyaxis right; plot(1:numel(MT), 100*[MT.inside], 'o-', 'LineWidth', 1.5); ylabel('
    Truth inside \pm3\sigma (%)'); ylim([0 105]);
set(gca, 'XTick', 1:numel(MT), 'XTickLabel', tune_names); grid on;
title('Accuracy (bars) and honesty (line) for each tuning');

%% E6 - stress test: strong model mismatch
```

```
s6 = get_sc('E6');
r6A = run_stage(s6, m, 'A');  r6B = run_stage(s6, m, 'B');  r6C = run_stage(s6, m, 'C');
fprintf('==== E6: strong model mismatch (slow RC R2 = %.1f mOhm) ====\n', 1000*s6.p.true.R2);
print_row('Charge counting', flight_metrics(r6A, 'cc'));
print_row('EKF stage A',     flight_metrics(r6A, 'ekf'));
print_row('EKF stage B',     flight_metrics(r6B, 'ekf'));
print_row('EKF stage C',     flight_metrics(r6C, 'ekf'));
fprintf('\n');

figE6 = figure('Name', 'E6 stress', 'Color', 'w', 'Position', [120 120 900 380]);
plot_errors(gca, {r6A, r6B, r6C}, {'EKF stage A', 'EKF stage B', 'EKF stage C'}, ...
            'E6: SoC error when the real battery differs strongly from the model');

%% E7 - uncertainty vs the flat part of the OCV curve
r7B = run_stage(s1, m, 'B');  r7C = run_stage(s1, m, 'C');
figE7 = figure('Name', 'E7 flat OCV', 'Color', 'w', 'Position', [140 140 900 420]);
keep = r7B.t > 60;  keepC = r7C.t > 60;              % skip the first minute (start-up)
yyaxis left; hold on; grid on;
plot(100*r7B.soc(keep),  100*sqrt(r7B.Pd(1,keep)),  'b-', 'LineWidth', 1.3);
plot(100*r7C.soc(keepC), 100*sqrt(r7C.Pd(1,keepC)), 'm-', 'LineWidth', 1.3);
ylabel('\sigma_{SoC} (% SoC)');
yyaxis right;
s = linspace(0.15, 0.95, 200);  [~, sl] = ocv_lookup(s);
plot(100*s, 1000*sl/100, 'Color', [0.85 0.45 0.1], 'LineWidth', 1);
ylabel('OCV slope (mV per 1% SoC)');
set(gca, 'XDir', 'reverse');  xlabel('True SoC (%)  \leftarrow flight goes this way');
legend('Stage B uncertainty', 'Stage C uncertainty', 'OCV slope', 'Location', 'northwest');
title('E7: where the curve is flat (low slope), stage C becomes less sure');

%% E8 - summary over 20 random flights per scenario
seeds  = 1:20;
stages = {'A', 'B', 'C'};
nS = numel(sc);  nSeed = numel(seeds);
R = struct('rmse', nan(nS,4,nSeed), 'inside', nan(nS,4,nSeed), 'final', nan(nS,4,nSeed), ...
           'nis', nan(nS,4,nSeed), 'nees', nan(nS,4,nSeed));   % column 1 = charge counting, 2-4 = stages
fprintf('==== E8: 20 flights x %d scenarios x 3 stages ====\n', nS);
tic;
for i = 1:nS
    for j = 1:nSeed
        for k = 1:numel(stages)
            o = sc{i}.opts;  o.seed = seeds(j);  o.stage = stages{k};
            r = run_flight(sc{i}.p, m, o);
            if k == 1, R = store(R, i, 1, j, flight_metrics(r, 'cc')); end
            R = store(R, i, k+1, j, flight_metrics(r, 'ekf'));
        end
    end
    fprintf('  %s done (%.0f s)\n', sc{i}.code, toc);
end
```

```
148
149 meth = {'Charge counting', 'Stage A', 'Stage B', 'Stage C'};
150 fprintf('\nMean over %d flights:  RMSE (truth inside +/-3 sigma)\n', nSeed);
151 fprintf('%-34s', 'Scenario');  fprintf('%-18s', meth{:});  fprintf('\n');
152 rows = {};
153 for i = 1:nS
154     fprintf('%-34s', [sc{i}.code ' ' sc{i}.name]);
155     for c = 1:4
156         rm = 100*mean(R.rmse(i,c,:));  in = 100*mean(R.inside(i,c,:));
157         if c == 1, fprintf('%-18s', sprintf('%.2f%%', rm));
158         else,      fprintf('%-18s', sprintf('%.2f%% (%3.0f%%)', rm, in)); end
159         rows(end+1, :) = {sc{i}.code, sc{i}.name, meth{c}, rm, in, ...
160                           100*mean(R.final(i,c,:)), mean(R.nis(i,c,:)), mean(R.
    nees(i,c,:))}; %#ok<SAGROW>
161     end
162     fprintf('\n');
163 end
164 T = cell2table(rows, 'VariableNames', {'code', 'scenario', 'method', 'rmse_pct',
    ...
165                'inside3sigma_pct', 'final_err_pct', 'mean_nis', 'mean_nees'});
166
167 figE8 = figure('Name', 'E8 summary', 'Color', 'w', 'Position', [160 160 950 650]);
168 labels = cellfun(@(s) s.code, sc, 'UniformOutput', false);
169 subplot(2,1,1);
170 mcol = [0.85 0.2 0.2; 0.95 0.6 0.1; 0 0.3 0.9; 0.75 0 0.75];   % same colours as
    the other figures
171 hb = bar(100*mean(R.rmse, 3));  grid on;  set(gca, 'XTickLabel', labels);
172 for c = 1:4, hb(c).FaceColor = mcol(c,:); end
173 ylabel('Mean RMSE (% SoC)');  legend(meth, 'Location', 'northoutside', '
    Orientation', 'horizontal');
174 title(sprintf('E8: accuracy over %d random flights per scenario (lower is better)'
    , nSeed));
175 subplot(2,1,2);
176 hb = bar(100*mean(R.inside(:,2:4,:), 3));  grid on;  set(gca, 'XTickLabel', labels
    );  ylim([0 105]);
177 for c = 1:3, hb(c).FaceColor = mcol(c+1,:); end
178 yline(99.7, 'k--', 'ideal 99.7%', 'HandleVisibility', 'off');
179 ylabel('Truth inside \pm3\sigma (%)');  legend(meth(2:4), 'Location', '
    northoutside', 'Orientation', 'horizontal');
180 title('Honesty: how often the truth was inside the filter''s own band (higher is
    better)');
181
182 %% Save
183 out = fullfile(root, 'results', 'day4');
184 if ~exist(out, 'dir'), mkdir(out); end
185 save_fig(figE3, fullfile(out, 'fig_e3_bias.png'));
186 save_fig(figE4, fullfile(out, 'fig_e4_aged.png'));
187 save_fig(figE5, fullfile(out, 'fig_e5_tuning.png'));
188 save_fig(figE6, fullfile(out, 'fig_e6_stress.png'));
189 save_fig(figE7, fullfile(out, 'fig_e7_flat_ocv.png'));
190 save_fig(figE8, fullfile(out, 'fig_e8_summary.png'));
191 writetable(T, fullfile(out, 'e8_summary.csv'));
192 save(fullfile(out, 'day4_runs.mat'), 'r3B', 'r3C', 'r4B', 'r4C', 'rT', 'r6A', 'r6B
    ', 'r6C', 'r7B', 'r7C', 'R', 'T', 'sc');
193 fprintf('\nFigures and e8_summary.csv saved in %s\n', out);
194
195 %% ---------------- Local functions ----------------
196 function s = scenario(code, name, p, opts)
```

```
s.code = code;  s.name = name;  s.p = p;  s.opts = opts;
end

function r = run_stage(s, m, stage)
o = s.opts;  o.stage = stage;
r = run_flight(s.p, m, o);
end

function R = store(R, i, c, j, M)
R.rmse(i,c,j) = M.rmse;  R.inside(i,c,j) = M.inside;  R.final(i,c,j) = M.final;
R.nis(i,c,j)  = M.nis;   R.nees(i,c,j)   = M.nees;
end

function print_row(name, M)
if isnan(M.inside)
    fprintf('  %-22s RMSE %5.2f%%  final %+6.2f%%\n', name, 100*M.rmse, 100*M.
    final);
else
    fprintf('  %-22s RMSE %5.2f%%  final %+6.2f%%  inside +/-3sig %3.0f%%  NIS %.2
    f  NEES %.2f\n', ...
            name, 100*M.rmse, 100*M.final, 100*M.inside, M.nis, M.nees);
end
end

function plot_errors(ax, runs, names, ttl)
axes(ax); hold on; grid on;
last = runs{end};  tm = last.t/60;  s3 = 300*sqrt(last.Pd(1,:));
fill([tm fliplr(tm)], [s3 -fliplr(s3)], [0.9 0.85 1], 'EdgeColor', 'none');
plot(runs{1}.t/60, 100*(runs{1}.soc_cc - runs{1}.soc), 'r--', 'LineWidth', 1.1);
cols = [0.95 0.6 0.1; 0 0 1; 0.75 0 0.75];
cols = cols(end-numel(runs)+1:end, :);
for i = 1:numel(runs)
    plot(runs{i}.t/60, 100*(runs{i}.x(1,:) - runs{i}.soc), 'Color', cols(i,:), '
    LineWidth', 1.2);
end
yline(0, 'k:', 'HandleVisibility', 'off');
ylabel('SoC error (%)'); xlabel('Time (min)'); title(ttl);
legend([{[names{end} ' \pm3\sigma'], 'Charge counting'}, names], 'Location', 'best
    ');
end

function plot_param(ax, r, idx, scale, truth, truth_lbl, ylab, ttl)
axes(ax); hold on; grid on;
tm = r.t/60;  v = scale*r.x(idx,:);  s3 = 3*scale*sqrt(r.Pd(idx,:));
fill([tm fliplr(tm)], [v+s3 fliplr(v-s3)], [0.9 0.85 1], 'EdgeColor', 'none');
plot(tm, v, 'm', 'LineWidth', 1.3);
yline(truth, 'k--', truth_lbl, 'HandleVisibility', 'off');
ylabel(ylab); xlabel('Time (min)'); title(ttl);
legend('Stage C \pm3\sigma', 'Stage C estimate', 'Location', 'best');
end

function save_fig(fig, file)
if exist('exportgraphics', 'file')
    exportgraphics(fig, file, 'Resolution', 150);
else
    saveas(fig, file);
end
end
```

Listing A.22: `main_day4_experiments.m`

```
%% Day 5 - The return-to-home decision: four policies compared
% The drone flies the 'long' route (6 drop-offs, too far for one battery)
% and must decide when to turn back. Four policies decide:
%   A  voltage threshold (what many simple drones do)
%   B  charge counting + "do I still have enough to get home?"
%   C  EKF estimate + the same question, ignoring uncertainty
%   D  EKF risk-aware: turn back when P(home with reserve) < 95%
%
% Part 1: one demo mission, all four policies
% Part 2: Monte Carlo - 500 random missions x 4 policies
% Part 3: threshold sweep - safety vs wasted battery
%
% Run from the drone_battery_ekf folder. Full run: about 5-10 minutes.
% For a quick test set N_MC = 50 and N_SWEEP = 30 first.

clear; clc; close all;
root = fileparts(mfilename('fullpath'));
addpath(fullfile(root, 'config'), fullfile(root, 'sim'), ...
        fullfile(root, 'estimators'), fullfile(root, 'decision'));

N_MC    = 500;      % missions in the Monte Carlo study
N_SWEEP = 200;      % missions in the threshold sweep

p = battery_params();
m = make_mission('long');

% Return-to-home settings
rp.reserve   = 0.10;    % always land with at least 10% SoC
rp.trip_frac = 0.10;    % trip-cost uncertainty: 10% of the trip (wind, load)
rp.p_thr     = 0.95;    % policy D: turn back when P(safe) < 95%
rp.V_thr     = 3.5;     % policy A: turn back when cell voltage < 3.5 V ...
rp.V_hold    = 5;       %   ... for 5 seconds in a row

policies = {'A', 'B', 'C', 'D'};
pnames   = {'A: voltage threshold', 'B: charge counting', 'C: EKF estimate only',
    'D: EKF risk-aware'};
pcol     = [0.95 0.6 0.1; 0.85 0.2 0.2; 0 0.3 0.9; 0.75 0 0.75];

%% Part 1 - one demo mission: aged battery, sensor reads 0.5 A low, headwind
c_demo = struct('soc0', 0.95, 'bias', -0.5, 'wind', 0.10, 'Cn', 0.85*p.true.Cn,
    ...
                'R0', p.true.R0, 'R1', p.true.R1, 'C1', p.true.C1, 'R2', p.true.R2
    );
demo = cell(1, 4);
fprintf('==== Part 1: demo mission (aged battery 85%%, bias -0.5 A, wind +10%%)
    ====\n');
for j = 1:4
    demo{j} = run_mission_rth(p, m, c_demo, policies{j}, rp, 7);
    o = demo{j};
    fprintf('  %-22s turned back at %4.1f min | deliveries %d | landed with %5.1f
    %% | %s\n', ...
            pnames{j}, o.t_rth/60, o.drops, 100*o.soc_land, ternary(o.died, '
    BATTERY DIED', 'safe'));
end

```

```
% Figure 1: how the decision is made (policy D's flight)
d = demo{4}.rec;  tm = d.t/60;  tD = demo{4}.t_rth/60;
fig1 = figure('Name', 'Demo decision', 'Color', 'w', 'Position', [60 60 950 800]);
subplot(3,1,1); hold on; grid on;
plot(tm, 100*d.ps, 'm', 'LineWidth', 1.4);
yline(100*rp.p_thr, 'k--', '95% threshold', 'HandleVisibility', 'off');
for j = 1:4
    if ~isnan(demo{j}.t_rth)
        xline(demo{j}.t_rth/60, '-', policies{j}, 'Color', pcol(j,:), 'LineWidth', 1.2, 'HandleVisibility', 'off');
    end
end
ylim([0 105]); ylabel('P(safe return) (%)');
title('Policy D turns back as soon as it is no longer 95% sure of getting home with the reserve');

subplot(3,1,2); hold on; grid on;
fill([tm fliplr(tm)], 100*[d.soc_hat+3*d.sig fliplr(d.soc_hat-3*d.sig)], [0.92 0.85 1], 'EdgeColor', 'none');
plot(tm, 100*d.soc, 'k', 'LineWidth', 1.5);
plot(tm, 100*d.soc_hat, 'm', 'LineWidth', 1.2);
plot(tm, 100*d.soc_cc, 'r--', 'LineWidth', 1.1);
plot(tm, 100*(d.dsoc + rp.reserve), 'Color', [0.2 0.6 0.2], 'LineWidth', 1.3);
xline(tD, 'm:', 'HandleVisibility', 'off');
ylabel('SoC (%)');
legend('EKF \pm3\sigma', 'True SoC', 'EKF estimate', 'Charge counting', 'Needed to get home + reserve', 'Location', 'northeast');
title('Turning back is safe while the estimate stays clearly above the green line');

subplot(3,1,3); hold on; grid on;
for j = 1:4
    plot(demo{j}.rec.t/60, 100*demo{j}.rec.soc, 'Color', pcol(j,:), 'LineWidth', 1.3);
end
yline(100*rp.reserve, 'k--', '10% reserve', 'HandleVisibility', 'off');
yline(0, 'k-', 'HandleVisibility', 'off');
ylabel('True SoC (%)'); xlabel('Time (min)'); legend(pnames, 'Location', 'northeast');
title('What really happened to the battery under each policy');

% Figure 2: map of the four flights
fig2 = figure('Name', 'Demo map', 'Color', 'w', 'Position', [80 80 950 450]);
hold on; grid on; axis equal;
plot(m.wp(:,1)/1000, m.wp(:,2)/1000, 's', 'Color', [0.5 0.5 0.5], 'MarkerFaceColor', [0.8 0.8 0.8], 'MarkerSize', 8, 'HandleVisibility', 'off');
plot(0, 0, 'k^', 'MarkerFaceColor', 'k', 'MarkerSize', 10, 'HandleVisibility', 'off');
for j = 1:4
    P = demo{j}.rec.pos;
    plot(P(:,1)/1000, P(:,2)/1000 + 0.03*(j-2.5), 'Color', pcol(j,:), 'LineWidth', 1.4);   % small offset so lines do not overlap
end
for j = 1:4
    o = demo{j};
    if ~isnan(o.t_rth)
        q = o.rec.pos(o.t_rth, :);
        plot(q(1)/1000, q(2)/1000 + 0.03*(j-2.5), 'o', 'Color', pcol(j,:), '
```

```
    MarkerFaceColor', pcol(j,:), 'HandleVisibility', 'off');
    end
    if o.died
        q = o.rec.pos(end, :);
        plot(q(1)/1000, q(2)/1000, 'kx', 'MarkerSize', 14, 'LineWidth', 2.5, '
    HandleVisibility', 'off');
        text(q(1)/1000 + 0.1, q(2)/1000 - 0.15, 'battery died', 'Color', pcol(j,:)
    );
    end
end
xlabel('East (km)'); ylabel('North (km)'); legend(pnames, 'Location', '
    northeastoutside');
title('Demo mission: dots = turn-back points, grey squares = drop-offs, triangle =
     home');

%% Part 2 - Monte Carlo: N_MC random missions x 4 policies
rng(2026);
cond = cell(N_MC, 1);
for i = 1:N_MC, cond{i} = draw_conditions(p); end

died = false(N_MC, 4);  land = nan(N_MC, 4);  drops = nan(N_MC, 4);
acc = nan(N_MC, 3);                                  % EKF RMSE, CC RMSE, EKF
    inside +/-3 sigma (policy D flights)
fprintf('\n==== Part 2: Monte Carlo, %d missions x 4 policies ====\n', N_MC);
tic;
for i = 1:N_MC
    for j = 1:4
        o = run_mission_rth(p, m, cond{i}, policies{j}, rp, 10000 + i);   % same
    noise for every policy
        died(i,j) = o.died;  land(i,j) = o.soc_land;  drops(i,j) = o.drops;
        if j == 4, acc(i,:) = [o.rmse_ekf, o.rmse_cc, o.inside]; end
    end
    if mod(i, 50) == 0, fprintf('  %d / %d missions (%.0f s)\n', i, N_MC, toc);
    end
end
S = summarise(died, land, drops, rp.reserve);

fprintf('\n%-24s %10s %16s %16s %12s\n', 'Policy', 'Died', 'Reserve broken', '
    Unused battery', 'Deliveries');
for j = 1:4
    fprintf('%-24s %9.1f%% %15.1f%% %15.1f%% %12.2f\n', pnames{j}, 100*S.died(j),
    100*S.breach(j), 100*S.unused(j), S.drops(j));
end
fprintf('\nEstimator accuracy over the %d policy-D flights: EKF (stage C) RMSE %.2
    f%%, charge counting RMSE %.2f%%, truth inside EKF +/-3 sigma %.0f%%\n', ...
        N_MC, 100*mean(acc(:,1)), 100*mean(acc(:,2)), 100*mean(acc(:,3)));
aged = cellfun(@(c) c.Cn < 0.9*p.true.Cn, cond);
fprintf('\nBattery died, aged batteries only (capacity < 90%%, %d missions):  ',
    sum(aged));
fprintf('%s %.1f%%   ', policies{1}, 100*mean(died(aged,1)), policies{2}, 100*mean
    (died(aged,2)), ...
        policies{3}, 100*mean(died(aged,3)), policies{4}, 100*mean(died(aged,4)));
fprintf('\n');

fig3 = figure('Name', 'Monte Carlo', 'Color', 'w', 'Position', [100 100 950 650]);
vals = {100*S.died, 100*S.breach, 100*S.unused, S.drops};
ttl  = {'Battery died in the air (%)', 'Landed below the 10% reserve, or died (%)'
    , ...
```

```
        'Unused battery at landing, above reserve (%)', 'Deliveries completed (mean)'};
for q = 1:4
    subplot(2,2,q);
    hb = bar(vals{q}, 'FaceColor', 'flat');  hb.CData = pcol;  grid on;
    set(gca, 'XTickLabel', policies);
    lo = min(0, min(vals{q}));  hi = max(vals{q});
    ylim([1.25*lo - 0.5*(lo < 0), 1.2*hi + 0.5]);       % headroom so value labels do not hit the title
    title(ttl{q});
    for j = 1:4
        text(j, vals{q}(j), sprintf('%.1f', vals{q}(j)), 'HorizontalAlignment', 'center', 'VerticalAlignment', 'bottom');
    end
end
sgtitle(sprintf('Monte Carlo: %d random missions (start charge, sensor bias, wind, battery age, battery parameters)', N_MC));

%% Part 3 - threshold sweep: safety vs wasted battery
D_thr = [0.80 0.90 0.95 0.99 0.999];
A_thr = [3.40 3.45 3.50 3.55 3.60];
nS = min(N_SWEEP, N_MC);
SD = sweep(p, m, cond(1:nS), rp, 'D', 'p_thr', D_thr);
SA = sweep(p, m, cond(1:nS), rp, 'A', 'V_thr', A_thr);
SB = summarise(died(1:nS,2), land(1:nS,2), drops(1:nS,2), rp.reserve);
SC = summarise(died(1:nS,3), land(1:nS,3), drops(1:nS,3), rp.reserve);

fprintf('\n==== Part 3: threshold sweep (%d missions) ====\n', nS);
fprintf('Policy D (risk-aware):\n');
for k = 1:numel(D_thr)
    fprintf('  P(safe) < %5.1f%%: died %4.1f%%, reserve broken %4.1f%% (promised <= %4.1f%%), unused %4.1f%%, deliveries %.2f\n', ...
            100*D_thr(k), 100*SD.died(k), 100*SD.breach(k), 100*(1-D_thr(k)), 100*SD.unused(k), SD.drops(k));
end
fprintf('Policy A (voltage):\n');
for k = 1:numel(A_thr)
    fprintf('  V < %.2f V: died %4.1f%%, reserve broken %4.1f%%, unused %4.1f%%, deliveries %.2f\n', ...
            A_thr(k), 100*SA.died(k), 100*SA.breach(k), 100*SA.unused(k), SA.drops(k));
end

fig4 = figure('Name', 'Trade-off', 'Color', 'w', 'Position', [120 120 800 550]);
hold on; grid on;
plot(100*SA.unused, 100*SA.breach, 'o-', 'Color', pcol(1,:), 'LineWidth', 1.5, 'MarkerFaceColor', pcol(1,:));
plot(100*SB.unused, 100*SB.breach, 's', 'Color', pcol(2,:), 'MarkerFaceColor', pcol(2,:), 'MarkerSize', 10);
plot(100*SC.unused, 100*SC.breach, 'd', 'Color', pcol(3,:), 'MarkerFaceColor', pcol(3,:), 'MarkerSize', 10);
plot(100*SD.unused, 100*SD.breach, 'o-', 'Color', pcol(4,:), 'LineWidth', 1.5, 'MarkerFaceColor', pcol(4,:));
for k = 1:numel(A_thr), text(100*SA.unused(k)+0.4, 100*SA.breach(k)+1.5, sprintf('%.2f V', A_thr(k)), 'Color', pcol(1,:)); end
for k = 1:numel(D_thr), text(100*SD.unused(k)+0.4, 100*SD.breach(k)+1.5, sprintf('%g%%', 100*D_thr(k)), 'Color', pcol(4,:)); end
xlabel('Unused battery at landing (% SoC)  \rightarrow more wasted range');
```

```
ylabel('Missions that broke the reserve or died (%)  \rightarrow less safe');
legend('A: voltage threshold', 'B: charge counting', 'C: EKF estimate only', 'D: EKF risk-aware', 'Location', 'northeast');
title('Safety vs wasted battery: bottom-left is best');

%% Save
out = fullfile(root, 'results', 'day5');
if ~exist(out, 'dir'), mkdir(out); end
save_fig(fig1, fullfile(out, 'fig_demo_decision.png'));
save_fig(fig2, fullfile(out, 'fig_demo_map.png'));
save_fig(fig3, fullfile(out, 'fig_monte_carlo.png'));
save_fig(fig4, fullfile(out, 'fig_tradeoff.png'));
T = table(pnames', 100*S.died', 100*S.breach', 100*S.unused', S.drops', ...
          'VariableNames', {'policy', 'died_pct', 'reserve_broken_pct', 'unused_pct', 'deliveries'});
writetable(T, fullfile(out, 'monte_carlo_summary.csv'));
TS = table([repmat({'D'}, numel(D_thr), 1); repmat({'A'}, numel(A_thr), 1)], [D_thr'; A_thr'], ...
           100*[SD.died'; SA.died'], 100*[SD.breach'; SA.breach'], 100*[SD.unused'; SA.unused'], [SD.drops'; SA.drops'], ...
           'VariableNames', {'policy', 'threshold', 'died_pct', 'reserve_broken_pct', 'unused_pct', 'deliveries'});
writetable(TS, fullfile(out, 'threshold_sweep.csv'));
for j = 1:4, demo{j} = rmfield(demo{j}, 'rec'); end   % keep the .mat small
save(fullfile(out, 'day5_results.mat'), 'died', 'land', 'drops', 'acc', 'cond', 'S', 'SD', 'SA', 'demo', 'rp');
fprintf('\nFigures and CSV files saved in %s\n', out);

%% ---------------- Local functions ----------------
function S = summarise(died, land, drops, reserve)
% Per policy (column): died, reserve broken (died or landed below reserve),
% unused battery above the reserve (landed missions only), deliveries.
n = size(died, 2);
S.died = mean(died, 1);
S.breach = mean(died | land < reserve, 1);
S.unused = nan(1, n);
for j = 1:n, S.unused(j) = mean(land(~died(:,j), j) - reserve); end
S.drops = mean(drops, 1);
end

function S = sweep(p, m, cond, rp, policy, field, values)
n = numel(cond);  K = numel(values);
died = false(n, K);  land = nan(n, K);  drops = nan(n, K);
for k = 1:K
    r = rp;  r.(field) = values(k);
    for i = 1:n
        o = run_mission_rth(p, m, cond{i}, policy, r, 10000 + i);
        died(i,k) = o.died;  land(i,k) = o.soc_land;  drops(i,k) = o.drops;
    end
    fprintf('  sweep %s %s = %g done\n', policy, field, values(k));
end
S = summarise(died, land, drops, rp.reserve);
end

function out = ternary(cond, a, b)
if cond, out = a; else, out = b; end
end
```

```
function save_fig(fig, file)
if exist('exportgraphics', 'file')
    exportgraphics(fig, file, 'Resolution', 150);
else
    saveas(fig, file);
end
end
```

Listing A.23: `main_day5_rth.m`

## A.7 Learning exercise

```
%% Day 1 - The EKF by hand (SoC only)
% Goal: understand predict, update and the Kalman gain K with real numbers.
% Run one section at a time (Ctrl+Enter). Change numbers and re-run.
%
% Simplifications for Day 1 (all removed later):
%   - one state only: SoC
%   - no R1-C1 polarisation, no bias state, known capacity

clear; clc; close all;

%% Part 1: one predict + update step (matches the hand calculation)
dt    = 10;       % s   (10 s so the SoC change is easy to see)
Cn    = 5;        % Ah  battery capacity
R0    = 0.008;    % ohm internal resistance
Q     = 1e-6;     % process noise: how much we distrust the model (SoC^2)
R     = 0.01^2;   % measurement noise: 10 mV voltage sensor (V^2)

% Near 79% SoC the OCV curve is roughly a straight line through
% (70%, 3.93 V) with slope 0.7 V per unit SoC (= 7 mV per 1% SoC).
ocv_lin   = @(s) 3.93 + 0.7*(s - 0.70);
slope_lin = @(s) 0.7;

soc = 0.80;       % current guess: 80%
P   = 0.05^2;     % current uncertainty: +/-5% (1 sigma)
Im  = 20;         % measured current (A)
Vm  = 3.80;       % measured voltage (V)

[soc_new, P_new, d] = ekf_step_1d(soc, P, Im, Im, Vm, dt, Cn, R0, Q, R, ocv_lin,
    slope_lin);
print_step(soc, P, d);

%% Part 2: turn the knobs - how does K react?
% Same step, three situations. Watch K, the new SoC and the new sigma.
cases = { 'Baseline (10 mV sensor, +/-5% guess)', 0.05^2, 0.01^2;
          'Noisy sensor (50 mV)',                 0.05^2, 0.05^2;
          'Confident guess (+/-1%)',              0.01^2, 0.01^2 };

fprintf('\n%-38s %8s %10s %10s\n', 'Situation', 'K', 'new SoC', 'new sigma');
for i = 1:size(cases, 1)
    [s_i, P_i, d_i] = ekf_step_1d(soc, cases{i,2}, Im, Im, Vm, dt, Cn, R0, Q,
    cases{i,3}, ocv_lin, slope_lin);
    fprintf('%-38s %8.3f %9.2f%% %9.2f%%\n', cases{i,1}, d_i.K, 100*s_i, 100*sqrt(
    P_i));
```

```
end
fprintf(['\nLesson: K is big when we are unsure of our guess and trust the sensor;\n' ...
         'K is small when the sensor is noisy or we are already confident.\n']);

%% Part 3: a 10-minute discharge - EKF vs charge counting
% Truth: battery really starts at 75%. Both estimators wrongly think 80%.
% The current sensor also reads 0.3 A too high (bias).
rng(1);                                  % same random numbers every run

soc_tab = [0 5 10 20 30 40 50 60 70 80 90 100] / 100;
ocv_tab = [3.30 3.45 3.55 3.63 3.68 3.73 3.78 3.85 3.93 4.00 4.08 4.18];
ocv_fun   = @(s) interp1(soc_tab, ocv_tab, min(max(s, 0), 1), 'linear');
slope_fun = @(s) (ocv_fun(min(s + 0.005, 1)) - ocv_fun(max(s - 0.005, 0))) ...
                 / (min(s + 0.005, 1) - max(s - 0.005, 0));

dt   = 1;  N = 600;                      % 1 s steps, 10 minutes
I    = 15 * ones(1, N);                  % true current (A)
bias = 0.3;  sigI = 0.2;  sigV = 0.01;   % sensor errors

% --- Simulate the truth and the sensors ---
soc_true = zeros(1, N);  soc_true(1) = 0.75;
for k = 2:N
    soc_true(k) = soc_true(k-1) - I(k-1)*dt/(3600*Cn);
end
Im = I + bias + sigI*randn(1, N);                          % measured current
Vm = ocv_fun(soc_true) - R0*I + sigV*randn(1, N);          % measured voltage

% --- Run the EKF and charge counting ---
Q = (dt/(3600*Cn))^2 * sigI^2 + 1e-9;    % current noise pushed through the model
R = sigV^2;
soc_ekf = zeros(1, N);  P_ekf = zeros(1, N);  K_ekf = zeros(1, N);
soc_cc  = zeros(1, N);
soc_ekf(1) = 0.80;  P_ekf(1) = 0.05^2;  soc_cc(1) = 0.80;
for k = 2:N
    [soc_ekf(k), P_ekf(k), d] = ekf_step_1d(soc_ekf(k-1), P_ekf(k-1), Im(k-1), Im(k), Vm(k), ...
                                            dt, Cn, R0, Q, R, ocv_fun, slope_fun);
    K_ekf(k)  = d.K;
    soc_cc(k) = soc_cc(k-1) - Im(k-1)*dt/(3600*Cn);
end

% --- Plot ---
t = (0:N-1) / 60;                        % minutes
sig3 = 3*sqrt(P_ekf);
figure('Name', 'Day 1: EKF vs charge counting', 'Color', 'w');

subplot(3,1,1); hold on; grid on;
fill([t fliplr(t)], 100*[soc_ekf+sig3 fliplr(soc_ekf-sig3)], [0.8 0.88 1], ...
     'EdgeColor', 'none', 'DisplayName', 'EKF \pm3\sigma');
plot(t, 100*soc_true, 'k',  'LineWidth', 1.5, 'DisplayName', 'True SoC');
plot(t, 100*soc_ekf,  'b',  'LineWidth', 1.2, 'DisplayName', 'EKF');
plot(t, 100*soc_cc,   'r--','LineWidth', 1.2, 'DisplayName', 'Charge counting');
ylabel('SoC (%)'); legend('Location', 'northeast');
title('EKF corrects a wrong start; charge counting never does');

subplot(3,1,2); grid on;
plot(t, K_ekf, 'b'); ylabel('Kalman gain K');
```

```
title('K is large at first (unsure), then settles');

subplot(3,1,3); grid on;
plot(t, 100*sqrt(P_ekf), 'b'); ylabel('\sigma_{SoC} (%)'); xlabel('Time (min)');
title('Uncertainty shrinks fast, then stays small');

fprintf('\nFinal error  EKF: %+.2f%%   charge counting: %+.2f%%\n', ...
        100*(soc_ekf(end) - soc_true(end)), 100*(soc_cc(end) - soc_true(end)));

%% ---------------- Local functions ----------------
function [soc, P, d] = ekf_step_1d(soc, P, Im_prev, Im_now, Vm, dt, Cn, R0, Q, R,
    ocv_fun, slope_fun)
% One EKF step for a single state (SoC).
    % 1. Predict: charge counting moves the guess, uncertainty grows
    d.soc_pred = soc - Im_prev*dt/(3600*Cn);
    d.P_pred   = P + Q;                          % F = 1 for this model

    % 2. Compare: what voltage do we expect at the predicted SoC?
    d.V_pred = ocv_fun(d.soc_pred) - R0*Im_now;
    d.y      = Vm - d.V_pred;                    % innovation (the surprise)
    d.H      = slope_fun(d.soc_pred);            % tangent slope dOCV/dSoC

    % 3. Update: move by K x surprise, uncertainty shrinks
    d.S = d.H^2 * d.P_pred + R;                  % expected size of the surprise
    d.K = d.P_pred * d.H / d.S;                  % Kalman gain
    soc = d.soc_pred + d.K * d.y;
    P   = (1 - d.K*d.H) * d.P_pred;
end

function print_step(soc0, P0, d)
    fprintf('Start:    SoC = %.2f%%  (sigma %.2f%%)\n', 100*soc0, 100*sqrt(P0));
    fprintf('Predict:  SoC = %.2f%%  (sigma %.2f%%)\n', 100*d.soc_pred, 100*sqrt(d
    .P_pred));
    fprintf('Expected voltage %.4f V, innovation %+.4f V\n', d.V_pred, d.y);
    fprintf('H = %.2f V per unit SoC,  S = %.6f V^2,  K = %.3f\n', d.H, d.S, d.K);
    fprintf('Update:   SoC = %.2f%%  (sigma %.2f%%)\n', ...
            100*(d.soc_pred + d.K*d.y), 100*sqrt((1 - d.K*d.H)*d.P_pred));
end
```

Listing A.24: `learning/day1_ekf_by_hand.m`